

BOOK

Introduction to Database Systems

A Comprehensive Course: Relational Databases & NoSQL

Hussein Younis Computer Engineering Department

Last updated 8 November 2026

Chapter1 Introduction to Database Systems

1 The exponential growth of digital data

Global data creation expands rapidly from 15.5 zettabytes in 2015 to a forecast 221 zettabytes in 2026.

> **One zettabyte (ZB) expressed in bytes and bits**
>
> $$1\ \text{ZB} = 10^{21}\ \text{bytes} = 1,000,000,000,000,000,000,000\ \text{bytes}$$
>
> $$1\ \text{ZB} = 8 \times 10^{21}\ \text{bits} = 8,000,000,000,000,000,000,000\ \text{bits}$$

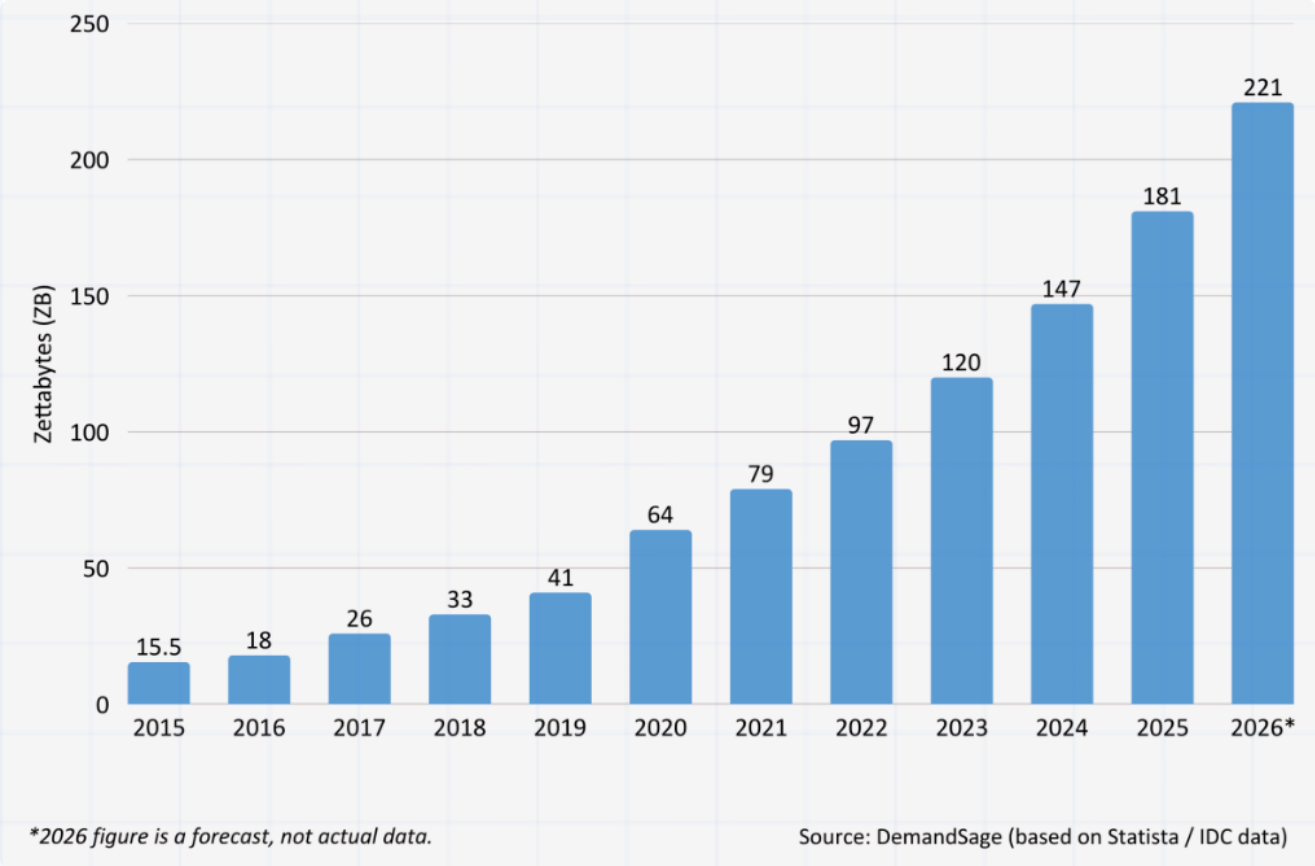


Figure 1: How Much Data Exists in the World?

Key Growth Highlights:

- **2015 Baseline:** 15.5 zettabytes of data created or replicated worldwide.
- **2026 Forecast:** 221 zettabytes of total global data volume.
- **Daily Volume:** Reaches **402.74 million terabytes** (or about 29 terabytes per second).
- **Annual Pace:** Data volume expands by roughly **22% year-over-year**.

Main Drivers of Expansion:

- **Unstructured Data:** Makes up **80% to 90%** of all data, growing three times faster than structured data.
- **Artificial Intelligence:** AI model training, machine learning, and interactive tools generate massive volumes of continuous information.
- **Connected Devices:** The expansion of the Internet of Things (IoT), edge computing, and high-resolution media like virtual reality and autonomous vehicle sensors drive high daily loads.

2 Fundamental Concepts

2.1 What is Data?

📖 DEFINITION 1: **Data**

Data are raw facts, values, observations, or symbols collected from the real world. By themselves, they may be incomplete or hard to interpret.

ⓘ **Note**

The data in DBMS can be broadly classified into two types; one is the collection of information needed by the organization and the other is “metadata” which is the information about the database.

📝 EXAMPLE 1:

Examples of Data:

- Numbers: 25, 100
- Text: "John", "New York"
- Images: A photograph
- Measurements: 30°C, 5.5 cm

2.2 What is Information?

📖 DEFINITION 2: **Information**

Information is data that has been processed, organized, and placed in context so that it becomes meaningful and useful.

Note

- Data becomes information when it is organized and interpreted.
- Data answers the question "What was observed?" while information answers the question "What does it mean?"

EXAMPLE 2:

Examples of Information:

- The average temperature for the week was 22 degrees Celsius.
- Ahmed, Sara, and Ali attended the meeting.

3 Data Classification in Database Systems

Database management systems (DBMS) categorize data across multiple structural, technical, and governance layers to optimize physical storage, query performance, and security.

3.1 Classification by Structure & Schema Format

Structured, semi-structured, and unstructured data are **classifications based on the degree of organization** in the data. They describe how data is formatted and whether it conforms to a predefined model.

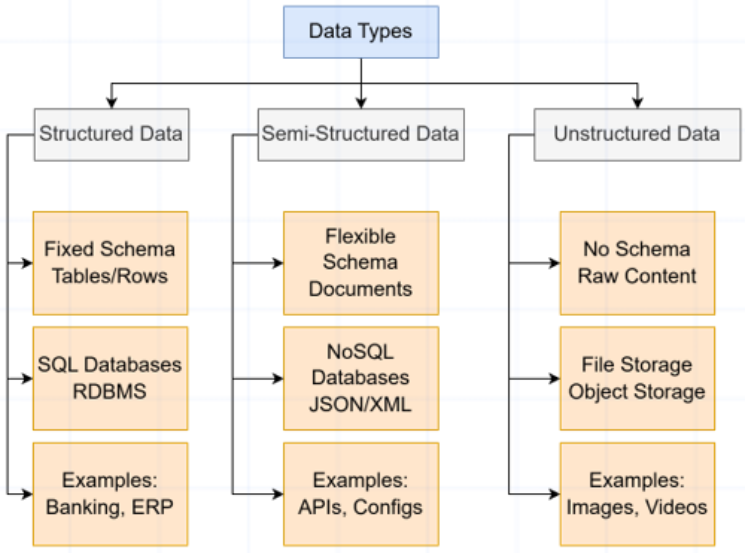


Figure 2: Overview of structured, semi-structured, and unstructured data.

3.1.1 Structured Data

Structured data has a predefined schema with fixed data types, organized into rows and columns (tables). Each field has a specific type and constraint, making it highly organized and easily searchable.

Key Characteristics:

- **Predefined Schema(Schema-on-Write):** Follows a fixed model (like a relational database, SQL table, or Excel spreadsheet) where every record shares the same fields and data types. any data is entered, with strict data types enforced (e.g., `INT`, `VARCHAR`).
- **Tabular Format:** Data is organized neatly in rows (records) and columns (attributes), like a spreadsheet.
- **Machine Readable:** Easily parsed by algorithms, software applications, and search engines.
- **High Efficiency:** Lends itself well to fast searching, sorting, aggregation, and mathematical calculations.

Common Examples:

- **Database Tables:** Customer names, phone numbers, account balances, and transaction timestamps.
- **Financial & Business Records:** Spreadsheets tracking inventory counts, payroll, and sales figures.

Table 1: a relational database table (how it looks to humans)

EmployeeID	FirstName	LastName	Department	HireDate	Salary
1001	Hussein	Younis	Engineering	2022-03-15	95000
1002	Ahmed	Yassen	Marketing	2021-08-01	78000
1003	Ammar	saif	Sales	2023-01-10	82000

3.1.2 Semi-Structured Data

Semi-structured data is information that does not conform to a rigid relational database schema, but still contains internal markers, tags, or organizational structures to separate data elements.

Unlike fully structured data (Schema-on-Write), it allows for flexibility because fields can be added, omitted, or nested on the fly without breaking the system. It bridges the gap between structured tables and completely unstructured text.

Key Characteristics:

- **Flexible Schema:** Records do not need to have the exact same fields. One record might have five attributes, while the next has ten.
- **Self-Describing:** The data contains both the description of the data (metadata) and the data itself within the same file (e.g., tags or keys like `"first_name": "hussein"`).
- **Hierarchical & Nested:** Data can easily contain child elements, arrays, or sub-objects nested within a single record.
- **Schema-on-Read:** The structure is interpreted when the data is read (queried), not when it is stored.
- **No Fixed Tables:** Data is stored in documents or objects rather than rigid rows.

Common Formats & Examples:

- **JSON (JavaScript Object Notation):** The most popular format for web APIs and modern NoSQL databases.

- **XML (eXtensible Markup Language):** A tag-based format widely used in legacy systems, enterprise configurations, and web services.
- **NoSQL Databases:** Platforms like **MongoDB** or **DynamoDB** that store data as flexible, semi-structured documents.
- **Emails:** The body text is unstructured, but the email metadata (**To, From, Subject, Date**) provides semi-structured organization.

```json
[
  {
    "userId": 201,
    "name": "Ahmed Mansour",
    "location": "Nablus",
    "contact": {
      "email": "ahmed@example.com",
      "phone": "+970-59-XXXXXXX"
    },
    "active": true
  },
  {
    "userId": 202,
    "name": "Fatima Nazzal",
    "location": "Ramallah",
    "skills": ["Data Analysis", "Python", "API Integration"]
  }
]
```

```xml
<catalog>
  <product id="P-501">
    <name>Wireless Mouse</name>
    <price currency="USD">29.99</price>
    <category>Electronics</category>
  </product>
  <product id="P-502">
    <name>Smart Thermostat</name>
    <price currency="USD">149.00</price>
    <specifications>
      <connectivity>Wi-Fi</connectivity>
      <compatibleWith>Alexa, Google Assistant</compatibleWith>
    </specifications>
  </product>
</catalog>
```

3.1.3 Unstructured Data

Unlike structured database tables or semi-structured JSON files, it cannot be neatly organized into rows and columns without heavy processing. It makes up the vast majority of the data generated in the world today—estimated at around **80% to 90% of all enterprise data**.

Key Characteristics:

- **No Inherent Structure:** There are no fixed fields, keys, or tags to identify data elements out of the box.
- **Heavy Text and Media Focus:** It consists primarily of raw human language, audio, or visual files.

- **Requires AI/ML to Parse:** To extract meaning, systems must use technologies like Natural Language Processing (NLP), computer vision, or speech-to-text.

Examples Using Local and Transliterated Contexts:

- **Text Files:** A digital copy of a book detailing the history of **Nablus**, or a text file containing the poetry of Mahmoud Darwish.
- **Audio Files:** A voice note sent by **Ahmed** over WhatsApp discussing a business proposal.
- **Video Footage:** Traffic or security camera footage overlooking the city center of **Ramallah** or **Hebron**.
- **Images:** Satellite photographs of agricultural fields near **Jenin**, or a JPEG of a scanned hand-written invoice.
- **Communications:** The raw text bodies of emails, chat messages, or social media posts.

4 How Data Was Saved: A Brief History

Before Edgar F. Codd published his landmark relational model paper in 1970 and before the relational database management systems (RDBMS) emerged, the database landscape was dominated by older generations of database models—most notably the **hierarchical database model** and the **network database model**.

4.1 Phase 1: The Filing Cabinet Era (Pre-Computer Era)

Before computers, Data was stored physically in paper folders, filing cabinets, and structured paper forms. Retrieval and organization relied entirely on manual sorting and clerical oversight. The terminology of this era persists in modern computer systems: physical folders became **files**, form entries became **records**, and blank spaces became **fields**.

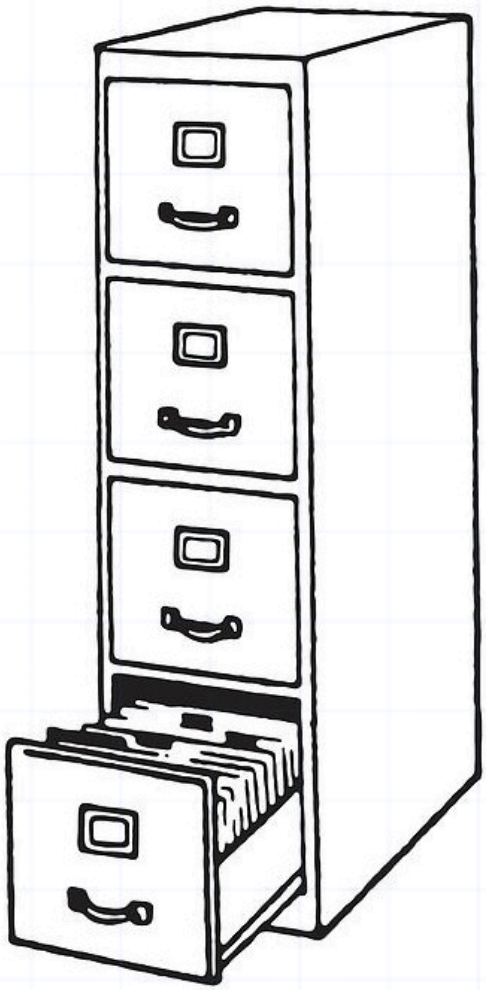


Figure 3: A physical filing cabinet, representing the pre-computer era of record keeping.

Problems and Limitations:

- Extreme physical vulnerability (susceptible to fire, misplacement, and physical degradation).
- Highly constrained scalability, requiring linear manual searching through massive physical archives.
- Complete absence of cross-referencing mechanisms or concurrent multi-user modification safeguards.

4.2 Phase 2: The Flat-File Era (1950s–1960s)

Emerged with the advent of commercial electronic data processing in the 1950s and dominated through the 1960s. Organizations digitized filing cabinets by storing records as simple sequential text files (flat files) on magnetic tape or early hard disks, processed via custom procedural programs written in COBOL or assembly.

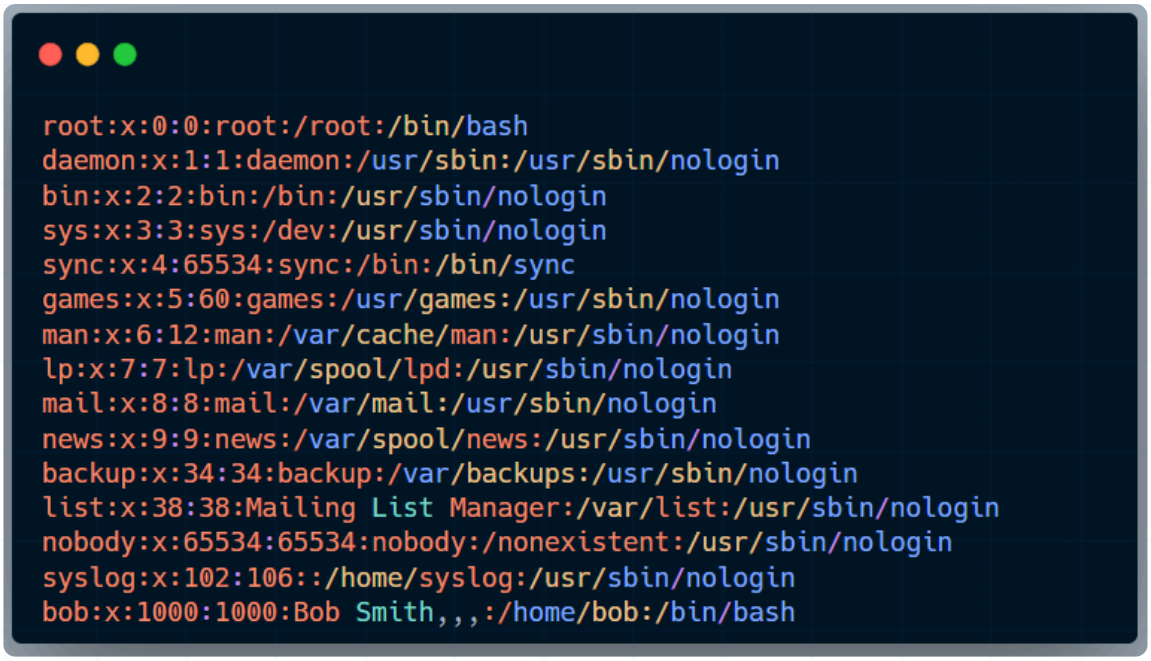


Figure 4: A real-world example of the flat-file era from modern Unix/Linux systems—the /etc/passwd configuration file, which stores user account records as sequential plain-text lines separated by colons (:).

🎬 SCENARIO 1: Notepad Scenario

Imagine building a simple student-grade system for an academic department using three independent text files:

```text
students.txt
001, Ahmed Khalidi, Computer_Eng
002, Sara Hasan,    Computer_Eng
003, Omar Zayed,    Information_Sys

grades.txt
001, Database, 92
001, Networks, 85
002, Database, 78
003, Database, 90

departments.txt
Computer_Eng,    Dr_Mansour, Building_A
Information_Sys, Dr_Younis,  Building_B
```

While clean and simple at first glance, this flat-file approach collapses under scale due to six critical structural problems:

1. **Linear Search Overhead ($O(n)$):** Finding records requires scanning files line by line from top to bottom, becoming agonizingly slow as datasets grow into millions of rows.
2. **Data Inconsistency:** Redundant data leads to conflicting updates when a change in one file is not propagated to duplicated fields elsewhere.
3. **Lost Updates from Concurrency:** Simultaneous edits by multiple users cause the last save to silently overwrite and destroy earlier work.
4. **Complex Manual Joins:** Cross-referencing related data across multiple files requires writing dozens of lines of tedious procedural parsing code.

5. **Zero Type Safety or Validation:** Plain text files allow malformed entries, typos, and empty sentinels that crash analytics scripts.
6. **File Corruption on Crashes:** Power outages mid-write leave files permanently truncated and inconsistent without a transaction recovery log.

4.3 Phase 3: Hierarchical and Network Database Models

To overcome flat-file limitations in the late 1960s and early 1970s, early database models introduced structured record linkage before the relational model took over:

- **The Hierarchical Model:** Organized data in a strict tree-like parent-child structure (such as IBM's Information Management System [IMS] developed for the Apollo program), where each child node could have only one parent.
- **The Network Model:** Standardized by the Conference on Data Systems Languages (CODASYL), allowing child records to connect to multiple parent nodes using graph-like pointer structures to support many-to-many relationships.
- **Limitations:** Both models required programmers to navigate physical pointers manually and lacked data independence, meaning changing the physical schema required rewriting application traversal logic.

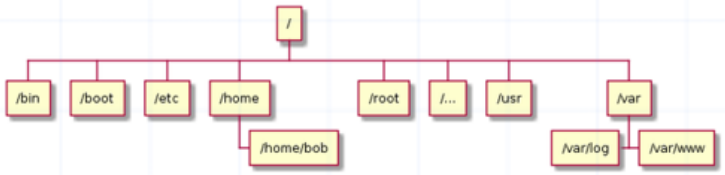


Figure 5: A hierarchical tree structure representing parent-child directory pathways.

4.4 Phase 4: The Relational Revolution and SQL

In 1970, **E.F. Codd** at IBM proposed the **relational model**.

- **Core Mechanism:** Data is stored mathematically as tables (relations) connected by primary and foreign keys, completely decoupling the logical structure of data from its physical storage.
- **The SQL Evolution:** To make the relational model practical, developers created **Structured Query Language (SQL)**. SQL allowed users to specify *what* data they wanted declaratively while the DBMS handled *how* to fetch and optimize the query.
- **Impact:** Commercial relational database management systems (RDBMS) solved the core flaws of flat files and early hierarchical models, establishing the dominant paradigm for enterprise software.

5 What Is a Database?

📖 DEFINITION 3: **Database**

A **database** is an organized collection of interrelated data that models some aspect of the real world.

📖 DEFINITION 4: **Database Management System (DBMS)**

Database Management System (DBMS) is a specialized software architecture that serves as an abstraction layer between physical data storage and user applications. A DBMS provides high-level declarative interfaces for data definition, query optimization, concurrency control, transaction processing, data integrity enforcement, and crash recovery.

ⓘ **Note**

The combination of a database and its DBMS is called a **database system**. In everyday speech, people often say "database" when they mean the entire system.

⚠ **Common Misconception**

Question: "Is a database software?"
Answer: No—a **database** is the *data* itself. The **DBMS** is the *software* that manages the database.

5.1 Data Abstraction

The major purpose of a database system is to provide users with an abstract view of the system. The system hides certain details of how data is stored and maintained, ensuring that internal complexity is hidden from database users.

Unlike raw file systems where developers must manage physical storage paths, byte offsets, and access indices, a Database Management System (DBMS) shields users from these low-level implementation details.

To achieve this, database systems typically organize data abstraction into three conceptual levels (known as the **Three-Schema Architecture**):

- **Physical Level:** The lowest level of abstraction, describing *how* the data is actually stored on disk (e.g., file allocations, B-tree indexing blocks, compression algorithms, and byte pointers).
- **Logical Level:** The middle level, describing *what* data is stored in the database and the relationships among them (e.g., tables, columns, constraints, and data types) without concerning itself with physical storage locations.
- **View Level:** The highest level of abstraction, consisting of tailored, restricted windows into the database designed for specific user groups (e.g., hiding sensitive salary or administrative columns from student or customer portals).

By maintaining strict separation across these layers, the DBMS ensures **data independence**—the ability to modify physical storage structures or indexing strategies without requiring any changes to application programs or logical table designs.

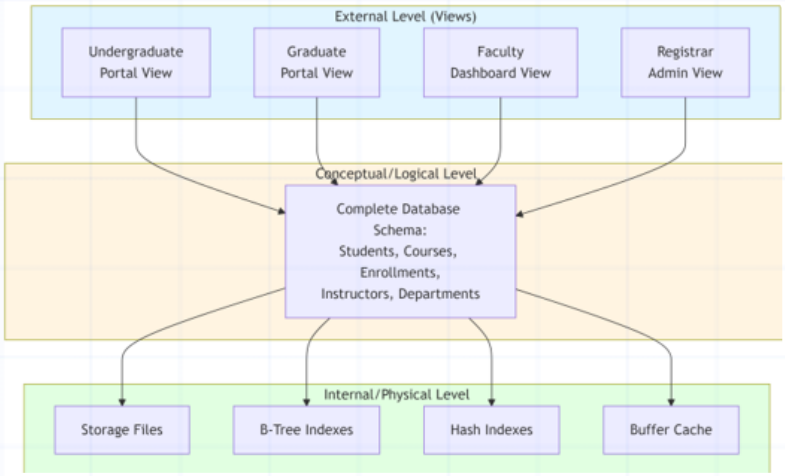


Figure 6: The Three-Schema Architecture of Data Abstraction, illustrating how user-specific external views map onto a unified logical database schema, which is in turn managed across internal physical storage structures and caching layers.

5.2 Key properties of a database

- **Self-describing**: The database stores a description of its own structure — called the **catalog** (or *data dictionary*) — alongside the data itself. Unlike a text file, the database knows what columns exist, what types they have, and what constraints apply.
- **Insulation between programs and data**: Application programs do not need to know *how* data is physically stored. Changing storage details does not require rewriting application code (**data abstraction**).
- **Support for multiple views**: Different users see different tailored views of the same underlying data. The payroll clerk sees salary columns; the student portal sees only name and grade.
- **Sharing and multiuser transaction processing**: Multiple users and programs access data concurrently while the DBMS guarantees that their actions do not interfere with each other.

5.3 Database Languages

> ⓘ **How Do We Talk to the Database?**
>
> You don't edit database files—you *ask* for what you need.
>
> These questions are written in a special language called SQL

A Database Management System (DBMS) provides structured languages for every interaction. While traditional relational database systems rely heavily on standardized SQL (Structured Query Language), different database models —particularly NoSQL systems—utilize alternative approaches tailored to their specific architectures.

Table 2: *The Four Pillars of Relational Database Languages*

Language	Abbreviation	Purpose & Detailed Scope	SQL Examples
Data Definition Language	DDL	**Data definition:** Create, modify, and remove definitions of the data's structure. These operations change properties that affect how the database accepts and stores data, which is more critical in rigid database architectures than in flexible ones.	`CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`
Data Manipulation Language	DML	**Update and Retrieval:** Insert, modify, and delete data rows (changing the actual managed data), or retrieve stored data as-is, filtered, or transformed into a useful format using rich query mechanisms.	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
Data Control Language	DCL	**Administration & Security:** Manage user access privileges, permissions, and security controls.	`GRANT`, `REVOKE`
Transaction Control Language	TCL	**Transaction Management:** Manage transaction boundaries, commit changes, or roll back uncommitted work to maintain atomicity.	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

5.3.1 Interacting with NoSQL Databases

Unlike relational databases that rely on a universal declarative language like SQL, **NoSQL databases use APIs and query languages specific to their data model**.

Because NoSQL families (such as document stores, key-value stores, column-family stores, and graph databases) reject rigid tabular schemas in favor of flexible structures, they require specialized query mechanisms:

- **Document Stores (e.g., MongoDB):** Utilize rich programmatic JSON/BSON-based query methods and driver APIs rather than standard relational `JOIN` statements (e.g., `db.students.find({ major: "Computer Science" })`).
- **Key-Value Stores (e.g., Redis, DynamoDB):** Rely on minimal programmatic APIs consisting of direct operational primitives like `GET(key)`, `PUT(key, value)`, and `DELETE(key)`.
- **Graph Databases (e.g., Neo4j):** Implement specialized graph traversal query languages (such as Cypher) designed specifically to navigate nodes and edges rather than relational rows and columns.

5.4 The Database Is Just Files — The DBMS Is the Container

Both a simple file system and a database ultimately store your records as **bytes in files** on a disk. When you create a database in MySQL, it simply creates a folder on your drive, and when you create a table, it writes files like `employee.ibd` inside that folder.

However, the fundamental difference lies in what sits on top of those raw files:

- **In a file system:** The files are "dead bytes." The operating system only gives you basic read and write system calls, leaving you to write custom code for searching, updating, concurrency, and security.
- **In a database:** The **Database Management System (DBMS)** adds an intelligent runtime layer around those files.

By wrapping the files in this container, the DBMS provides the advanced mechanisms missing from raw files—such as a **buffer pool** for fast memory caching, **transaction management** with Write-Ahead Logs (WAL) for crash safety, **locking and MVCC** for concurrent users, and an automated **query optimizer** that replaces manual file-parsing code.

Table 3: *What the DBMS adds on top of the raw files*

Concern	What "Just Files" Give You	What the DBMS Gives You
OS Integration	Raw read/write system calls	Buffered, page-aligned I/O; direct I/O for Write-Ahead Logs; `fsync` for guaranteed durability
Memory / Cache	Nothing — every read hits disk storage	**Buffer pool** — hot data pages stay in RAM (thousands of times faster than disk access)
Concurrency	OS advisory file locks (all-or-nothing)	Row, page, and table locks, Multi-Version Concurrency Control (MVCC), isolation levels, and deadlock detection
User Management	OS file permissions only	Fine-grained `CREATE USER`, `GRANT`, and `REVOKE` controls at the database, table, column, or row level
Security	Dead bytes on disk readable by anyone with disk access	Authentication, encryption-at-rest, audit logs, and TLS for client connections
Crash Safety	A half-written, corrupted file after a power outage	Write-Ahead Log (WAL) + automated rollback/redo mechanisms providing ACID durability
Query Processing	Custom hand-written scan, join, and sort logic	SQL parser → optimizer → executor that selects the most efficient execution plan automatically
Self-Description	Manual documentation or memory	**Data dictionary** — the database describes its own structure
Networking	Local file access only	TCP network listeners, connection pooling, and standard client drivers (JDBC/ODBC)

EXAMPLE 3: The Paradigm Shift: Manual Code vs. Declarative SQL

To understand why a Database Management System (DBMS) transforms data engineering, consider how a developer extracts specific records from a raw flat file versus how they query a relational database.

1. Manual File Parsing (The File System Approach)

When working directly with flat files, developers must write low-level procedural code to manage disk access, string manipulation, and filtering logic manually.

```python
# You must manually write code to find data
for line in open("students.csv"):
    parts = line.split(",")
    if parts[2] == "Computer Science":   # GPA column?
        print(parts[0])                # Hope this is the name!
```

Explanation: In this procedural approach, the programmer carries the entire execution burden. They must explicitly open the file, iterate through every single row, handle string splitting, and assume the column indices correspond to the expected schema. Any minor typo, misaligned delimiter, or missing column value causes runtime errors, incorrect filtering, or corrupted results.

2. Declarative Querying (The Relational Database Approach)

In contrast, a relational database allows users and applications to interact with data using a high-level, declarative language.

```sql
-- DBMS figures out the most efficient way to retrieve data
SELECT name FROM Students
WHERE major = 'Computer Science'
  AND gpa ≥ 3.5
ORDER BY gpa DESC;
```

Explanation: Here, the query is entirely **declarative**—the user specifies *what* data is needed rather than *how* to fetch it. The DBMS parses the statement, consults the system catalog, passes the query through an automated optimizer to choose the most efficient execution plan (such as utilizing a B+ tree index instead of a full table scan), manages memory caching via the buffer pool, and returns the accurate result safely.

5.5 Database Users and Administrators

Database users interact with the system in various ways depending on their technical expertise and job requirements.

Table 4: *Overview of Database Roles*

Category & Role	Description	Core Focus / Responsibilities	Typical Tool
1. Naive / Parametric Users	End users who interact with the system indirectly via pre-built applications. Totally unaware of DBMS internals.	Performing standard day-to-day transactions (e.g., checking grades, placing orders).	Web forms, mobile apps, customer portals
2. Application Programmers	Software developers who build backend/frontend services that interface with the database.	Writing code that executes SQL queries via embedded commands, drivers, or ORMs.	Python, Java, PHP, IDEs
3. Sophisticated Users	Power users familiar with the database who bypass standard interfaces.	Writing ad-hoc queries for deep data analysis, reporting, and research.	SQL clients, Jupyter Notebooks, BI tools
4. Specialized Users	Domain experts who create custom database applications or analytical pipelines.	Developing unique, domain-specific tasks (e.g., spatial analysis, machine learning integration).	Custom scripts, specialized frameworks
5. Database Designers (Data Modelers)	Professionals who bridge business requirements and technical database structure.	Requirements analysis, conceptual ER design, logical normalization, and physical layout.	ER modeling tools, data dictionaries
6. Database Administrators (DBAs)	Operations experts who carry ultimate responsibility for the production environment.	Schema definition, storage optimization, security/authorization, maintenance, and backups.	Command-line interfaces, Admin Consoles
7. Database Managers	High-level organizational leaders overseeing the entire data function.	Strategic planning, team supervision, budgeting, infrastructure procurement, and policy governance.	Project management tools, executive reports

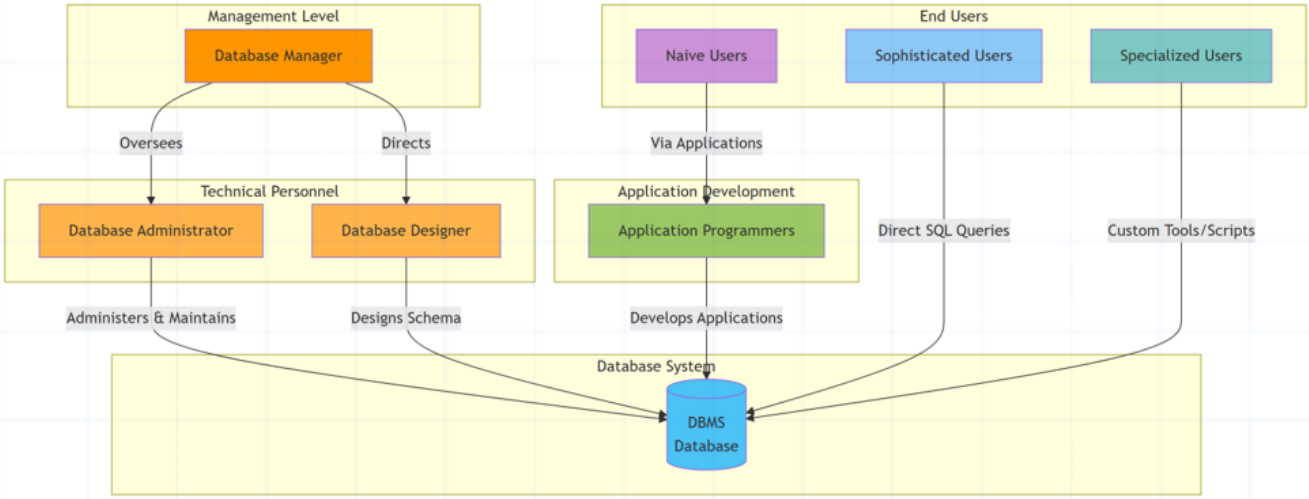


Figure 7: Overview of database system actors, organizational hierarchy, and their respective interactions with the database management system (DBMS).

5.5.1 Core Responsibilities of the Database Administrator (DBA)

The DBA bears ultimate operational responsibility for the health, availability, security, and integrity of the database production environment. Their core duties include:

- **Schema Definition:** Establishing the foundational database structure using Data Definition Language (DDL).
- **Storage Structure Decisions:** Configuring physical parameters, selecting optimal storage engines, and determining indexing strategies for query performance.
- **Authorization Management:** Enforcing security policies by granting and revoking privileges across users and roles to protect sensitive assets.
- **Routine Maintenance:** Overseeing automated backup schedules, monitoring resource utilization, and tuning slow-running queries.
- **Schema Evolution:** Safely modifying database structures to accommodate changing business requirements with minimal disruption.

5.5.2 Core Responsibilities of the Database Designer

The Database Designer (or Data Modeler) bridges business requirements and technical implementation by focusing on logical and physical data organization:

- **Requirements Analysis:** Interviewing stakeholders, identifying entities, attributes, and relationships, and documenting business rules and constraints.
- **Conceptual Design:** Constructing Entity-Relationship (ER) diagrams to map out high-level concepts, entity types, cardinalities, and participation constraints.
- **Logical Design:** Translating the conceptual model into a relational schema (tables), applying normalization rules (1NF, 2NF, 3NF, BCNF) to reduce redundancy, and defining keys.
- **Physical Design:** Selecting appropriate data types, building performance-tuned indexes, partitioning large datasets, and defining secure views.
- **Schema Evolution:** Planning structural updates and data migrations while keeping documentation current.

5.5.3 Core Responsibilities of the Database Manager

The Database Manager provides high-level organizational leadership, aligning database infrastructure with broader enterprise goals:

- **Strategic Planning:** Formulating long-term data strategies, evaluating emerging technologies, and planning for organizational scalability.
- **Team Management:** Directing DBAs and designers, prioritizing engineering tasks, and conducting performance evaluations.
- **Budget & Resource Allocation:** Managing infrastructure budgets, procuring hardware and software licenses, and justifying technology investments.
- **Policy & Governance:** Establishing organizational naming conventions, operational standards, and regulatory compliance protocols.

- **Stakeholder Communication:** Reporting system statuses to executives and coordinating with cross-functional software teams.

5.5.4 Core Responsibilities of the Database Users

Users interact with the system through distinct abstractions tailored to their technical expertise:

- **Application Programmers:** Build backend and frontend software services that interface with the database via drivers or ORMs to deliver business logic to end-users.
- **Sophisticated Users (Power Users):** Possess advanced database literacy, writing direct query-language statements for deep data exploration and reporting.
- **Naive Users (Casual / End Users):** Interact indirectly through graphical user interfaces, web portals, or mobile apps without any direct exposure to database schemas or query languages.
- **Specialized Users:** Build custom applications or analytical pipelines tailored to unique, domain-specific tasks (e.g., spatial analysis or machine learning integration).

6 Database Classification and Paradigms

While E.F. Codd's relational model remains the foundation of enterprise data management, the modern technological landscape spans a rich ecosystem of specialized database families tailored to distinct structural, scale, and performance needs.

6.1 Relational Databases

- **When Initiated & Used:** 1970s (E.F. Codd's paper), commercially viable in the 1980s. The dominant database model for over 40 years.
- **Problem it Solved:** It solved the rigidity and redundancy of previous models by using tables, relationships (keys), and a high-level, declarative query language (SQL).
- **Characteristics:**
  - Data organized into tables (relations) with rows (tuples) and columns (attributes).
  - Uses primary and foreign keys to define relationships.
  - Relies on ACID (Atomicity, Consistency, Isolation, Durability) transactions for strict data integrity.
- **Advantage:** High data integrity, flexible querying (JOINs), data independence, and a standardized language (SQL). Excellent for complex transactions.
- **Disadvantage:** Can be difficult to scale horizontally (across many servers). The rigid schema can be a poor fit for unstructured or semi-structured data.
- **Example:** PostgreSQL, MySQL, Oracle Database, Microsoft SQL Server.









6.2 NoSQL Databases

> ⓘ **Not only SQL**
>
> The acronym **NoSQL** does *not* stand for "No SQL" (meaning the complete absence of SQL or query languages). Instead, it officially stands for **"Not Only SQL."** This emphasizes that NoSQL databases complement relational databases rather than completely replace them, often combining alternative query mechanisms, specialized APIs, or even SQL-like layers alongside non-relational data models.

NoSQL ("Not only SQL") databases emerged in the late 2000s alongside the rise of big web applications (such as Google, Amazon, and Facebook) and cloud computing.

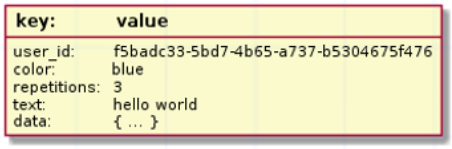
- **When Initiated & Used:** Late 2000s. Emerged to address the demands of large-scale distributed web applications.
- **Problem it Solved:** It addressed the need for horizontal scalability, handling massive volumes of unstructured and semi-structured data, and supporting flexible schemas where traditional relational models struggled.
- **Characteristics:**
  - "Not Only SQL." A broad category of non-relational databases.
  - Schema-flexible or schema-less design.
  - Built for horizontal scaling and high availability across distributed clusters.
  - Often utilize BASE (Basically Available, Soft state, Eventual consistency) properties instead of strict ACID guarantees.
- **Advantages:** High horizontal scalability, high performance for specific data models, ability to process massive volumes of diverse data, and developer-friendly flexibility for modern applications.
- **Disadvantages:** Weaker consistency guarantees compared to traditional RDBMS, lack of a universal standardized query language, and limited support for complex multi-table transactions.

6.2.1 Key-Value Databases

- **When Initiated & Used:** Mid-to-late 2000s. Gained prominence with systems like Amazon's Dynamo.
- **Problem it Solved:** It solved the need for extremely fast, simple, and scalable lookups for data that is accessed by a known identifier.
- **Characteristics:**
  - A giant, distributed hash map. Data is stored as a collection of key-value pairs.

- The value is opaque (blob) to the database; it doesn't know or care what's inside.

- **Advantages:** Incredibly fast, simple, and highly scalable. Perfect for caching and session management.
- **Disadvantages:** Limited querying capability. You can only query by the key. Not suitable for relationships or filtering based on the value's content.
- **Examples:** Redis, Amazon DynamoDB (Core functionality), Memcached.







6.2.2 Document Databases

- **When Initiated & Used:** Mid-to-late 2000s, popularized alongside web-scale applications and agile development methodologies.
- **Problem it Solved:** It solved the "impedance mismatch" between rigid relational tables and complex, hierarchical application objects (like nested JSON payloads) by storing data as self-describing documents.
- **Characteristics:**
  - Data is stored as semi-structured documents (typically JSON, BSON, or XML) rather than fixed-schema rows and columns.
  - Documents can contain nested objects and arrays, keeping related data together without requiring expensive multi-table `JOIN` operations.
  - Schema-flexible or schema-less design, allowing different documents in the same collection to have different fields.
- **Advantages:** Excellent alignment with modern programming object models, fast retrieval of complete hierarchical records, flexible schema evolution, and straightforward horizontal scaling (sharding).
- **Disadvantages:** Complex multi-document transactions can be more challenging to manage than in traditional RDBMS, and ad-hoc aggregations across deeply nested structures require specialized query operators.
- **Examples:** MongoDB, Couchbase, Amazon DocumentDB.

```json
{
  "_id": "customer_001",
  "name": "Alice Smith",
  "email": "alice@example.com",
  "orders": [
    {"id": 42, "date": "2026-02-06", "total": 129.99},
    {"id": 43, "date": "2026-02-10", "total": 89.50}
  ]
}
```

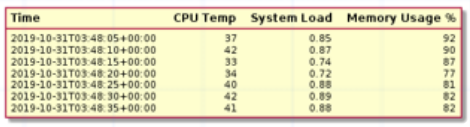
6.2.3 Graph Databases

- **When Initiated & Used:** Gained significant traction in the late 2000s and 2010s as highly connected datasets (social graphs, recommendation engines, fraud detection) overwhelmed traditional relational JOIN performance.
- **Problem it Solved:** It solved the exponential performance degradation of multi-table `JOIN` operations in relational databases when traversing deep, highly interconnected networks of relationships.
- **Characteristics:**
  - Data is modeled as a collection of **nodes** (entities), **edges** (relationships), and **properties** (attributes on both).
  - Relationships are treated as first-class citizens stored physically as direct pointers (index-free adjacency) rather than computed dynamically via keys at runtime.
  - Specialized declarative traversal languages (such as Cypher) designed specifically to query paths, networks, and patterns.
- **Advantages:** Blazing-fast traversal of deep relationships regardless of dataset size, highly intuitive modeling for connected domains, and exceptional performance for recommendation systems and network analysis.
- **Disadvantages:** Can be challenging to scale horizontally across distributed clusters for general-purpose transactional workloads, and requires a shift in thinking away from traditional tabular normalization.
- **Examples:** Neo4j, Amazon Neptune, ArangoDB.



6.3 Time-Series Databases

- **When Initiated & Used:** Emerged prominently in the early-to-mid 2010s to address the explosion of IoT devices, cloud infrastructure monitoring, and high-frequency financial trading.
- **Problem it Solved:** It solved the inability of traditional relational and standard NoSQL databases to efficiently ingest, compress, and query millions of high-velocity, append-only timestamped events without running out of disk space or slowing down historical range queries.
- **Characteristics:**
  - Optimized for append-only write workloads with strict time-based ordering (timestamps are primary).
  - Employs specialized compression techniques (such as delta-of-delta encoding and Gorilla compression) to shrink numerical time-series data down to a fraction of its raw size.
  - Features automated data lifecycle management, including downsampling (rolling up high-resolution old data into hourly or daily averages) and data retention policies (dropping old data automatically).
  - Includes specialized windowing and aggregation query capabilities (e.g., sliding-window averages, rate-of-change functions).
- **Advantages:** Blazing-fast write and read performance for time-stamped metrics, massive storage cost savings via specialized compression, and native handling of time-window analytics.
- **Disadvantages:** Poor performance for frequent random updates or deletes (since data is typically append-only), limited suitability for general-purpose transactional modeling, and specialized query languages that require a learning curve.
- **Examples:** InfluxDB, TimescaleDB, Prometheus, Amazon Timestream.



Time	CPU Temp	System Load	Memory Usage %
2019-10-31T03:48:05+00:00	37	0.85	92
2019-10-31T03:48:10+00:00	42	0.87	90
2019-10-31T03:48:15+00:00	33	0.74	87
2019-10-31T03:48:20+00:00	34	0.72	77
2019-10-31T03:48:25+00:00	40	0.88	81
2019-10-31T03:48:30+00:00	42	0.89	82
2019-10-31T03:48:35+00:00	41	0.88	82

6.4 NewSQL Databases

- **When Initiated & Used:** Emerged in the early-to-mid 2010s to bridge the gap between traditional relational databases and NoSQL distributed architectures.
- **Problem it Solved:** It solved the classic trade-off where developers had to choose between the horizontal scalability of NoSQL (which sacrificed ACID guarantees and complex querying) and the strict ACID compliance of traditional RDBMS (which struggled to scale horizontally across distributed clusters).
- **Characteristics:**
  - Combines the full power of relational structures and declarative SQL with distributed scale-out architectures.
  - Guarantees strict ACID transactions globally across distributed cluster nodes.
  - Employs advanced consensus algorithms (like Raft or Paxos) and precise time synchronization hardware/software (such as TrueTime in Google Spanner) to coordinate multi-node writes safely.
- **Advantages:** Offers the best of both worlds—horizontal scalability and high availability characteristic of NoSQL, combined with the strong consistency, schema integrity, and query power of SQL.

- **Disadvantages:** Complex underlying infrastructure requiring sophisticated distributed coordination, higher operational overhead, and potential latency overhead for cross-node distributed transactions.
- **Examples:** Google Spanner, CockroachDB, TiDB.

6.5 Multi-Model Databases

- **When Initiated & Used:** Gained momentum in the mid-to-late 2010s and 2020s as modern enterprise applications increasingly demanded diverse data access patterns within a single backend.
- **Problem it Solved:** It solved the friction and operational overhead of **polyglot persistence**—where applications had to manage multiple distinct databases (such as MySQL for relational data, MongoDB for documents, and Redis for caching) by integrating multiple data models into a single unified DBMS engine.
- **Characteristics:**
  - A single database engine that natively supports multiple data models (e.g., relational tables, documents, key-value pairs, graphs, or time-series) concurrently.
  - Allows developers to query different models using appropriate APIs or unified query languages within the same transactional context.
  - Eliminates the need for complex data synchronization pipelines between disparate database systems.
- **Advantages:** Reduced operational complexity, simplified infrastructure management, flexibility to choose the best data representation for different modules of the same app, and unified ACID transaction guarantees across heterogeneous data types.
- **Disadvantages:** Can be a "jack of all trades, master of none" if specific sub-engines lack the depth or specialized optimization of dedicated single-model databases, and can introduce complex tuning overhead.
- **Examples:** ArangoDB, Couchbase, Microsoft Azure Cosmos DB, PostgreSQL (with extensions like JSONB and AGE).

7 DB-Engines Ranking

Table 5: The top 10 database management systems ranking from DB-Engines for October 2026

Rank	DBMS	Database Model	Score (Oct 2026)
1	Oracle	Relational, Multi-model	1119.79
2	MySQL	Relational, Multi-model	837.96
3	Microsoft SQL Server	Relational, Multi-model	694.47
4	PostgreSQL	Relational, Multi-model	688.75
5	MongoDB	Document, Multi-model	374.64
6	Snowflake	Relational	221.30
7	Databricks	Multi-model	171.24
8	Redis	Key-value, Multi-model	156.09
9	IBM Db2	Relational, Multi-model	106.29
10	Apache Cassandra	Wide column, Multi-model	98.86

Chapter2 Entity-Relationship (ER) Model

CHAPTER OVERVIEW

Entity-Relationship (ER) modeling serves as the conceptual design phase in database development, acting as a crucial bridge between real-world requirements and software implementation. By translating unstructured business descriptions into precise visual schemas, ER diagrams establish a shared blueprint for business stakeholders and database engineers before any physical tables or SQL code are created. This chapter covers the database design lifecycle, the core components of conceptual data modeling (entities, attributes, and relationships), Chen notation visual symbols, key and structural constraints, and variations in relationship degrees and cardinalities.

🎯 Learning Outcomes

At the end of this chapter, you will be able to:

1. **Explain the DB Design Lifecycle & Conceptual Schema Design:** Trace the four phases of database design (Requirements Engineering, Conceptual Design, Logical Design, and Physical Design) and define the purpose of an ER diagram as an implementation-independent model.
2. **Differentiate ER Diagram Notations:** Compare visual notation dialects—including academic **Chen notation**, formal **Min-Max structural constraints**, and industry-standard **Crow's Foot notation**—and identify their primary use cases.
3. **Identify & Classify Entity Types:** Differentiate between **strong entities** (having independent existence and primary key attributes) and **weak entities** (relying on an owner entity and a partial key via identifying relationships).
4. **Classify & Map Attribute Types:** Analyze data properties and represent them in Chen notation as **simple**, **composite**, **single-valued**, **multi-valued**, **derived**, **stored**, **key**, or **partial key** attributes.
5. **Define & Model Relationship Properties:** Construct and interpret relationships across varying degrees (**unary/recursive**, **binary**, **ternary**), applying attributes to relationships and enforcing structural constraints such as **cardinality ratios** (1:1, 1:N, M:N) and **participation rules** (total vs. partial).

> The conceptual schema is the key to database design; all else is implementation detail.
>
> — Peter Chen, 1976

1 Scenario: "What Do You See in This Restaurant?"

🤔 Warmup Question

Imagine we are sitting at a table in a busy restaurant, and you look at me and ask, "*Can you describe what you see?*"

Click to expand

Naturally, I wouldn't start reciting code or listing software specifications. I would describe the scene in plain text:

- "*I see customers sitting at tables.*"
- "*There's a server taking an order from a table.*"
- "*The kitchen is cooking dishes listed on the menu.*"
- "*When the meal is done, the server brings a bill to the customer.*"

This human ability to observe a real-world environment and describe its key elements is the exact starting point of **Data Modeling**.

However, computer systems and database developers cannot build a software system directly from plain text descriptions. Plain English can be ambiguous, vague, or open to misinterpretation. To bridge the gap between human language and actual database implementation, we need a structured, standardized way to express these plain text requirements. This is where **Conceptual Modeling** and **Entity-Relationship Diagrams (ERDs)** come in. They take those casual real-world observations and translate them into a clear visual blueprint of **Entities** (`Customer` , `Order` , `Menu_Item`), **Attributes** (`Customer_Name` , `Price`), and **Relationships** (*A Customer places an Order*).

Describing a scene as a human is fundamentally the same mental process as **Data Modeling** in computer science.

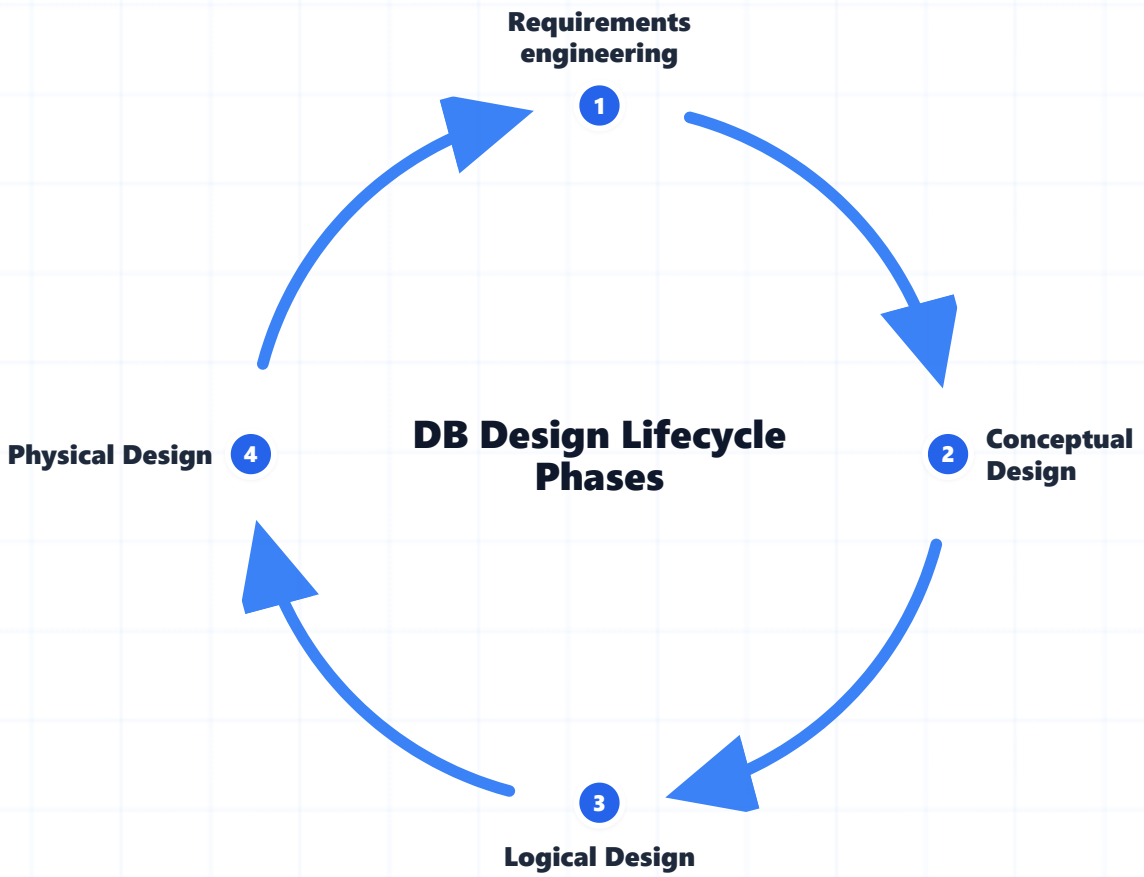
> 📖 DEFINITION 1: **Data Model**
>
> Concepts for describing data objects and their relationships (meta model).

> 📝 EXAMPLE 1: **Real-World Example: E-Commerce Store**
>
> Imagine building an online store. Before creating databases or writing code, you define the core concepts of the business:
>
> - **Entities (The Main Objects):**
> - `Customer` : The person making a purchase.
> - `Order` : The transaction placed by a customer.
> - `Product` : The item being sold.
> - **Attributes (Key Characteristics):**
> - `Customer` : Name, Email Address.
> - `Order` : Order Date, Total Amount.
> - `Product` : Product Name, Price.

- **Relationships & Cardinalities (How They Connect):**
  - A `Customer` **places** an `Order` (One customer can place **many** orders, but an order belongs to **one** customer).
  - An `Order` **contains** a `Product` (One order can contain **many** products, and a product can appear in **many** orders).

2 DB Design Lifecycle Phases

No database should be designed from thin air. A well-established **design lifecycle** guides the process from understanding what users need to deploying a production system.



1 Requirements engineering

- Collect and analyze data and application requirements
- **Key Questions**: What data must be stored? What

2 Conceptual Design

- Model data semantics and st ructure, independent of logi cal data model

3 Logical Design

- Model data with implementa tion primitives of concrete d ata model.

queries must be answered?
- **Deliverable:** Data requirements document.

- **Key Questions**: What entities and relationships exist in the domain?
- **Deliverable:** ER diagram.

- **Key Questions**: How do entities translate to tables? Are they normalized?
- **Deliverable:** Relational schema.

4 **Physical Design**

- Model user-level data organization in a specific DBMS (and data model) Account for deployment environment and performance requirements.
- **Key Questions**: Which indexes? What storage engine? How to partition?
- **Deliverable:** Annotated schema + storage spec.

3 Conceptual Data Modeling: Translating Ideas into Structure

Before drawing an Entity-Relationship Diagram (ERD), you must first define the **Conceptual Data Model** itself. While an ERD is the visual chart or diagram, conceptual data modeling is the high-level design process that identifies *what* core concepts exist in a domain and *how* they connect.

📖 DEFINITION 2: **Conceptual Data Model**

is the general language, rules, and notation framework used to model data conceptually (such as the **Entity-Relationship Model (ERM)** or **UML**).

📖 DEFINITION 3: **Conceptual Schema**

A description of the *structure* of a database at a level that is independent of any specific DBMS, storage format, or programming language. It answers: *"What data exists, and how is it connected?"*— not *"How is it stored?"*

3.1 The Purpose of Conceptual Data Modeling

1. **High-Level Abstraction:** It captures essential business concepts without being tied to specific technology, software engines, or database types (SQL vs. NoSQL).
2. **Stakeholder Bridge:** It serves as a shared vocabulary between technical developers and non-technical business managers to ensure everyone agrees on basic terms.
3. **Scope Definition:** It sets clear boundaries on what information the system will track and what it will ignore.

3.2 Core Building Blocks

Every conceptual model relies on three fundamental elements:

- **Entities (Objects of Interest):** Real-world subjects, places, or events you need to store information about.
  - *Example:* `Customer`, `Order`, `Product`, `Employee`.

- **Attributes (Properties & Characteristics):** Essential traits that describe each entity.
  - *Example:* A `Customer` entity has attributes like `Name` and `EmailAddress`.
- **Relationships (Business Rules & Connections):** Statements describing how entities interact with one another.
  - *Example:* A `Customer` **places** an `Order`.

3.3 The Conceptual Design Process

To move from raw business requirements to a formal model, follow these initial steps:

- **Extract Nouns & Verbs from Requirements:** Read through your business rules. Nouns usually map to **Entities** or **Attributes**, while verbs map to **Relationships**.
  - *Requirement Text: "A student enrolls in multiple courses."*
  - *Entities:* `Student`, `Course`
  - *Relationship:* **Enrolls In**
- **Determine Cardinality & Business Constraints:** Decide how many instances of one entity can relate to another (e.g., 1-to-1, 1-to-Many, Many-to-Many).
- **Draft the Visual Representation:** Once these rules and entities are clearly defined, you translate them into visual notation using an **ER Diagram (ERD)**.

1 **Extract Nouns & Verbs from Requirements**

Read through your business rules. Nouns usually map to **Entities** or **Attributes**, while verbs map to **Relationships**.

- *Requirement Text: "A student enrolls in multiple courses."*
- *Entities:* `Student`, `Course`
- *Relationship:* **Enrolls In**

2 **Determine Cardinality & Business Constraints**

Decide how many instances of one entity can relate to another (e.g., 1-to-1, 1-to-Many, Many-to-Many).

3 **Draft the Visual Representation**

Once these rules and entities are clearly defined, you translate them into visual notation using an **ER Diagram (ERD).**

4 What Is an ER Diagram?

ER modeling was introduced by **Peter Chen in 1976**. Its core contribution was giving database designers a single, shared language to communicate with both business stakeholders (who understand the problem domain) and software engineers (who will implement the solution).

> 📖 DEFINITION 4: **Entity-Relationship (ER) Diagram**
>
> A graphical representation of a conceptual schema—a high-level, implementation-independent description of the data a system must store.

> 📖 DEFINITION 5: **Conceptual Schema**
>
> A description of the *structure* of a database at a level that is independent of any specific DBMS, storage format, or programming language. It answers: *"What data exists, and how is it connected?"*— not *"How is it stored?"*

4.1 ER Notations Overview

An ER diagram is a graphical language, and like any language it comes in different dialects. Three notations are widely used; you will encounter all three in textbooks and professional tools.

Table 6: *Comparison of standard Entity-Relationship (ER) diagram notations, including creator origins and primary use cases.*

Notation	Introduced by	Where you see it
Chen	Peter Chen, 1976	University textbooks, academic exams, this course
Min-Max (Structural Constraints)	Various authors	Formal specifications needing exact minimum and maximum counts
Crow's Foot	Industry convention	Design tools — MySQL Workbench, Lucidchart, draw.io, Visio, ERwin

All three notations express the same concepts — entities, attributes, relationships, cardinality, and participation — using different visual symbols. You do not need to master all three right now. The important thing is to know they exist so you can recognise a diagram regardless of the notation it uses.

4.2 Chen Notation

4.2.1 Chen Notation — Symbol Reference

Chen notation is the academic standard you will use for all diagrams in this course. The table below is your reference for every symbol.

Table 7: *Chen Notation — Shape Reference*

Concept	Shape	Description
Strong Entity	Single rectangle	An object with independent existence and its own key
Weak Entity	Double rectangle	An object that depends on another entity for identification
Relationship	Single diamond	An association between two or more entities
Identifying Relationship	Double diamond	The relationship that identifies a weak entity
Attribute	Ellipse (oval)	A property of an entity or relationship
Key Attribute	Ellipse with underlined name	Uniquely identifies each entity instance
Multivalued Attribute	Double ellipse	Can hold multiple values per instance
Derived Attribute	Dashed ellipse	Computed from another stored attribute
Composite Attribute	Ellipse → child ellipses	Built from smaller sub-attributes
Total Participation	Double line	Every instance of the entity must participate
Partial Participation	Single line	Some instances may not participate
Cardinality label	1, N, or M on the line	Placed near each entity; shows the maximum

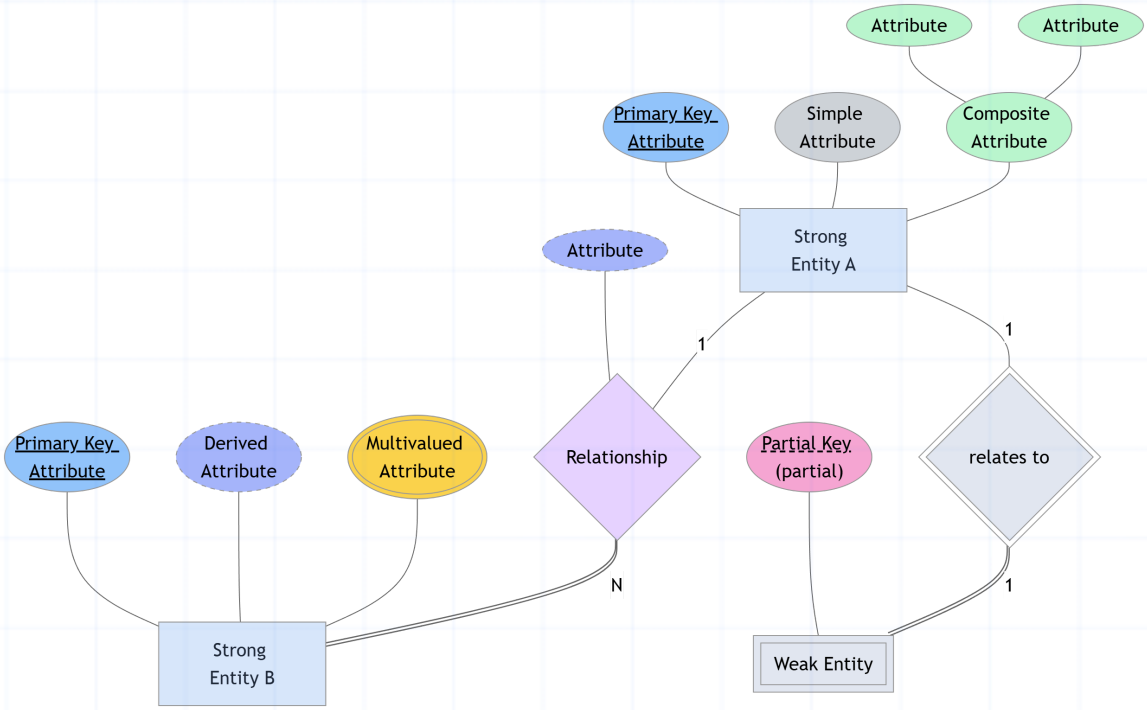


Diagram 1: Chen Notation — Shape Reference

4.2.2 Entities

4.2.2.1 Strong Entity

A strong entity type (or simply entity type) is a real-world object or concept that:

1. Has an independent existence — it does not depend on another object to exist in the database.
2. Has a set of attributes that describe it.
3. Has at least one attribute (or combination) that uniquely identifies each instance — its key attribute.

Notation: A **single rectangle** labeled with the entity name (all caps by convention).

> 📝 EXAMPLE 2: **STUDENT entity with its attributes in Chen notation**
>
> `Student` is the entity (rectangle), `ID` is the key attribute (underlined), and `NAME`, `GENDER`, `DOB` are simple attributes (ovals connected by lines).

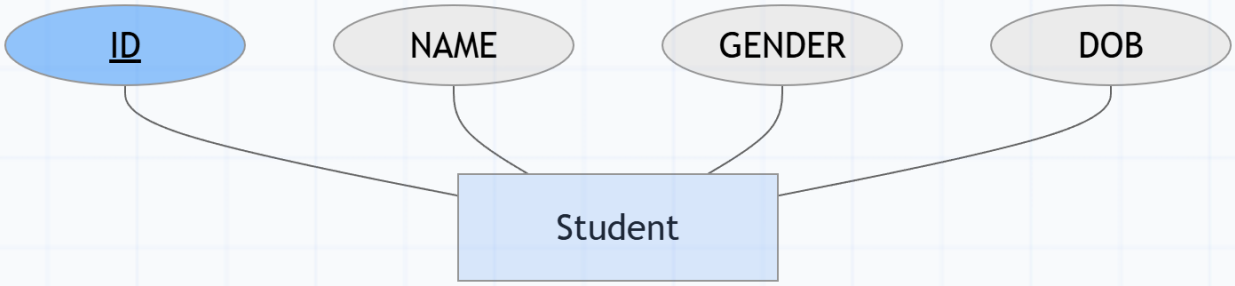


Diagram 2: Entity STUDENT with attributes ID (key), NAME, GENDER, and DOB

4.2.2.2 Weak Entity

A **weak entity type** has **no key attribute of its own**— it cannot be uniquely identified without reference to another entity (its **owner**).

> 📖 DEFINITION 6: **Partial Key**
>
> The attribute(s) of a weak entity that, combined with the owner entity's key, uniquely identify a weak entity instance.

Table 8: *Comparison of Key Features and Notation Differences Between Strong and Weak Entities*

Feature	Strong Entity	Weak Entity
Has own key?	Yes	No — only a partial key
Identified by	Own primary key	Owner's key + partial key
Rectangle notation	Single	Double
Identifying relationship	Single diamond	Double diamond
Participation in identifying rel.	May vary	Always total

Key characteristics include:

1. **Identification**: Identified by combining the owner entity's primary key with its own partial key (discriminator).
2. **Notation**: Drawn as a double rectangle in Chen notation.
3. **Relationship**: Connected to its owner through an identifying relationship (drawn as a double diamond).
4. **Participation**: Always has total participation in its identifying relationship, meaning a weak entity instance cannot exist in the database without an owner.

EXAMPLE 3:

Room is a weak entity identified by Hotel (owner) + roomNumber (partial key, dashed underline). The HAS relationship is the identifying relationship (double diamond in full Chen notation).

- `Hotel` is the **strong entity**— it has its own key (`hotelID` , underlined).
- `Room` is the **weak entity**— `roomNumber` alone is not unique across all hotels (two hotels can both have room 101). It is identified by the combination `(hotelID, roomNumber)` .
- `HAS` is the **identifying relationship**(1:N — one hotel has many rooms).

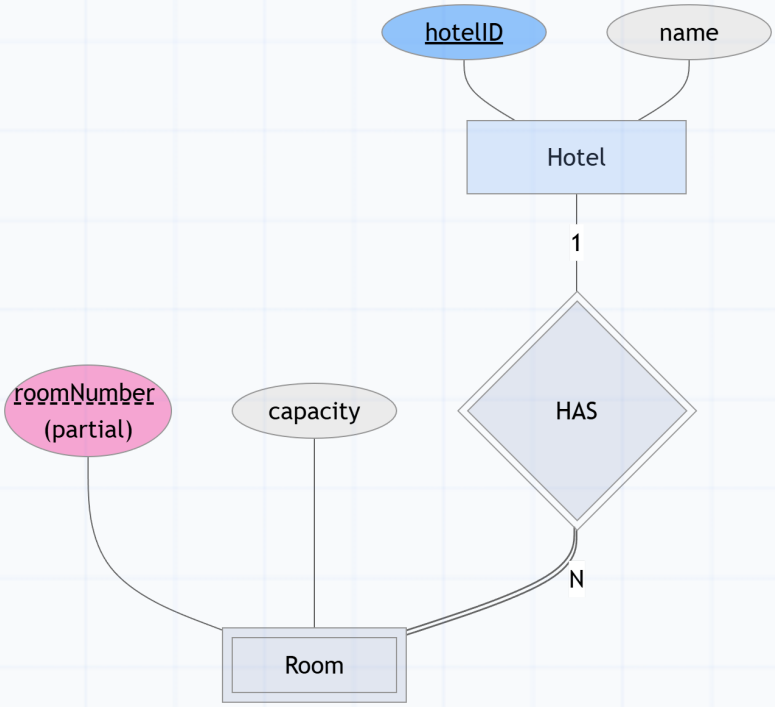


Diagram 3: Hotel (strong) and Room (weak entity) in Chen notation

EXAMPLE 4: Local Order Numbers (Order is a WEAK Entity)

- **The Setup:** Each location has its own Point of Sale (POS) system. Location A generates Order 1, 2, 3... Location B *also* generates Order 1, 2, 3...
- **The Problem:** If I say "Look at Order #1", you would ask, "*Which location's Order #1?*"
- **The Database Reality:** The `Order_Number` alone is not unique. To uniquely identify the record, the database **must** use a composite key: `(Location_ID, Local_Order_Number)` .

- **The Conclusion:** Because the `Order` relies on the `Location`'s ID to form its primary key, it is **Identification Dependent**. Therefore, **Order is a Weak Entity**. (The `Order_Number` acts as the *partial key* or *discriminator*).

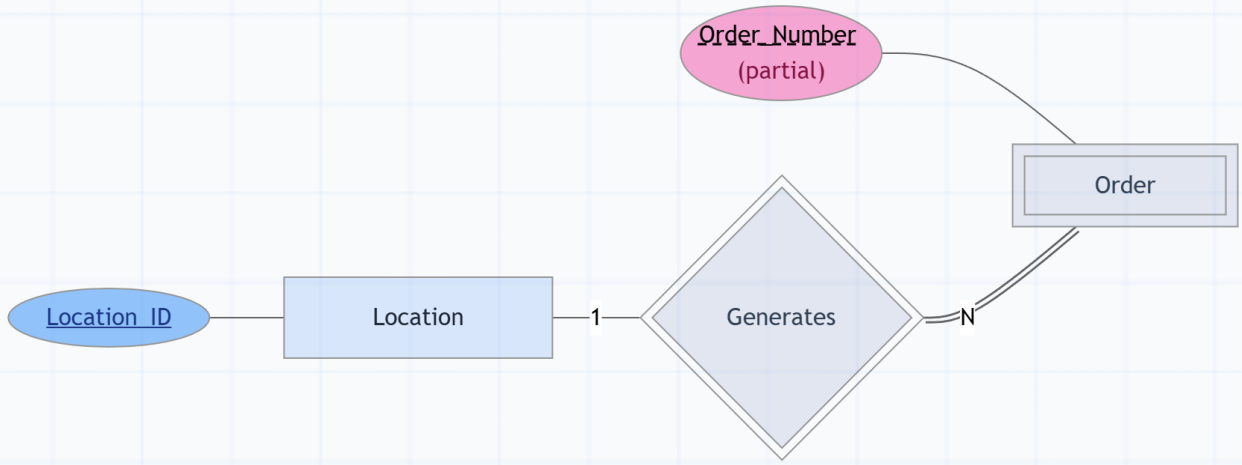


Diagram 4: Local Order Numbers (Order is a WEAK Entity)

📝 EXAMPLE 5: **Centralized Order Numbers (Order is a STRONG Entity)**

- **The Setup:** All locations connect to one central server to generate order IDs. Location A gets Order #1001, Location B gets Order #1002. The numbers never repeat across locations.
- **The Reality:** If I say "Look at Order #1001", you can find it immediately. You don't need to know the location to find the order, because `Order #1001` only exists once in the entire system.
- **The Database Reality:** The Primary Key is simply `Order_ID`. It is globally unique.
- **The Conclusion:** Because the `Order` has its own independent, globally unique identifier, it is **Identification Independent**. Therefore, **Order is a Strong Entity**.
- *(Note: The order still **must** belong to a location, so it has Total Participation / Existence Dependency, but as we learned with the passport, that does NOT make it weak!)*

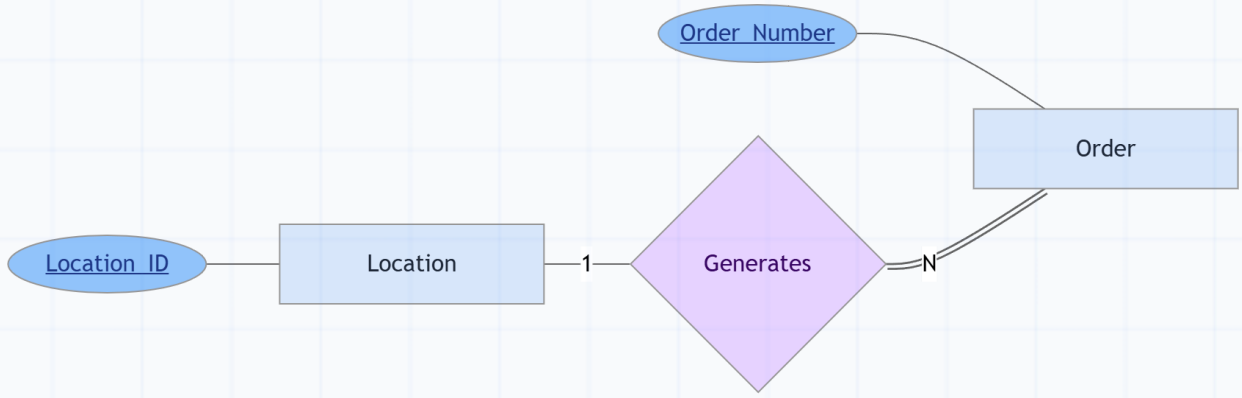


Diagram 5: Centralized Order Numbers (Order is a STRONG Entity)

4.2.3 Attributes

📖 DEFINITION 7: **Attribute**

A property that describes an entity type or relationship type.

Table 9: *Summary of ER Diagram Attribute Types, Descriptions, Chen Notation Visual Representations, and Examples*

Type	Description	Chen Shape	Example
Simple	Atomic, cannot be divided further	Plain oval	Salary, Gender
Composite	Built from smaller sub-attributes	Oval feeding child ovals	Name → (Fname, Minit, Lname)
Multivalued	Can hold multiple values per entity instance	Double oval	Phone_Numbers, Dept_Locations
Derived	Computed from another stored attribute; not physically stored	Dashed oval	Age derived from Bdate
Stored	Permanently stored; may be a source for derived attributes	Plain oval	Bdate
Key	Uniquely identifies each entity instance	Oval with underlined text	Ssn, Dnumber
Partial key	Identifies weak entity instances within the owner; not globally unique	Oval with dashed underline	Name (of DEPENDENT)

4.2.3.1 Simple (Atomic) Attribute

📖 DEFINITION 8: **Simple (Atomic) Attribute**

An attribute that cannot be divided into smaller meaningful parts.

📝 EXAMPLE 6: **Simple attributes — Product entity: every oval holds one indivisible value**

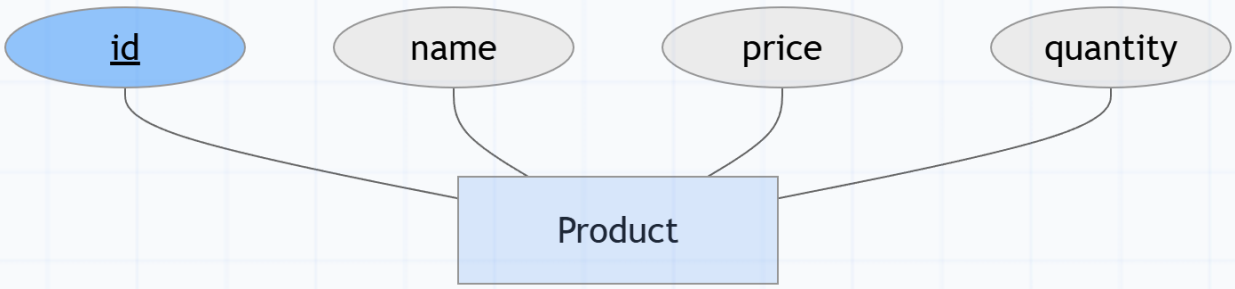


Diagram 6: Simple attributes — Product entity: every oval holds one indivisible value

4.2.3.2 Composite Attribute

📖 DEFINITION 9: **Composite Attribute**

An attribute made up of smaller, meaningful sub-attributes.

📝 EXAMPLE 7: **Composite attributes — Employee FullName and Address each decompose into sub-attributes**

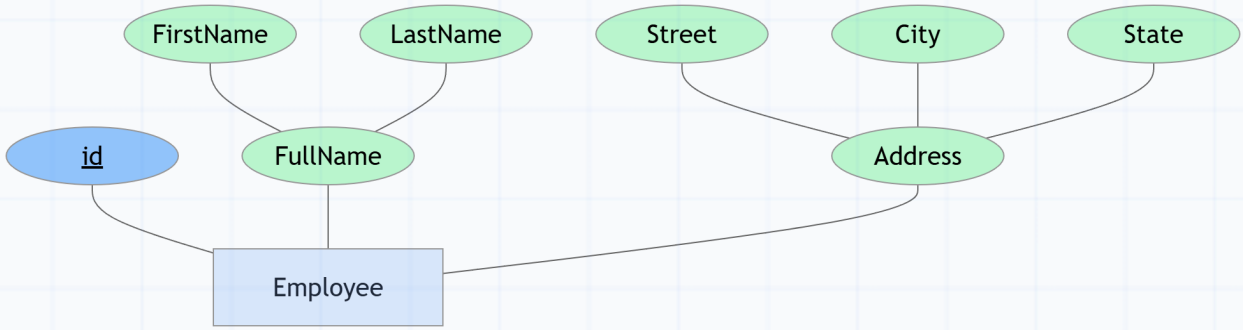


Diagram 7: Composite attributes — Employee FullName and Address each decompose into sub-attributes

4.2.3.3 Single-Valued Attribute

📖 DEFINITION 10: **Single-Valued Attribute**

An attribute that holds exactly one value for each entity instance.

📝 EXAMPLE 8: **Single-valued attributes — each User has exactly one id, one name and one email**

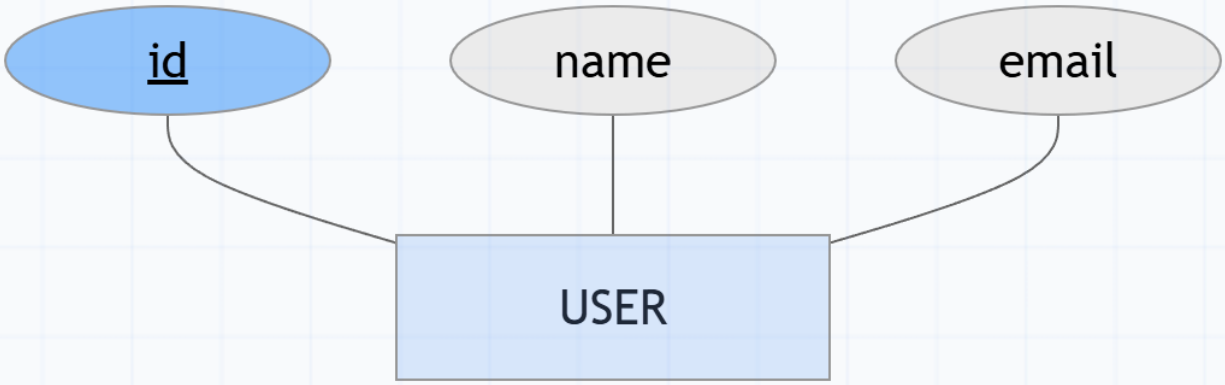


Diagram 8: Single-valued attributes — each User has exactly one id, one name and one email

4.2.3.4 Multi-Valued Attribute

📖 DEFINITION 11: **Multi-Valued Attribute**

An attribute that can hold more than one value for the same entity instance.

EXAMPLE 9: Multi-valued attributes — Addresses and Skills drawn as double ovals because one employee may have several

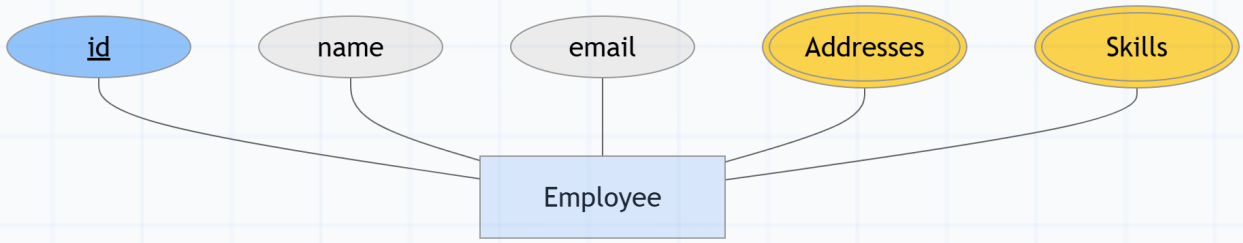


Diagram 9: Multi-valued attributes — Addresses and Skills drawn as double ovals because one employee may have several

4.2.3.5 Derived Attribute

DEFINITION 12: Derived Attribute

An attribute computed from another stored attribute rather than physically stored in the database.

EXAMPLE 10: Derived attribute — Age (dashed oval) is computed from the stored DOB attribute

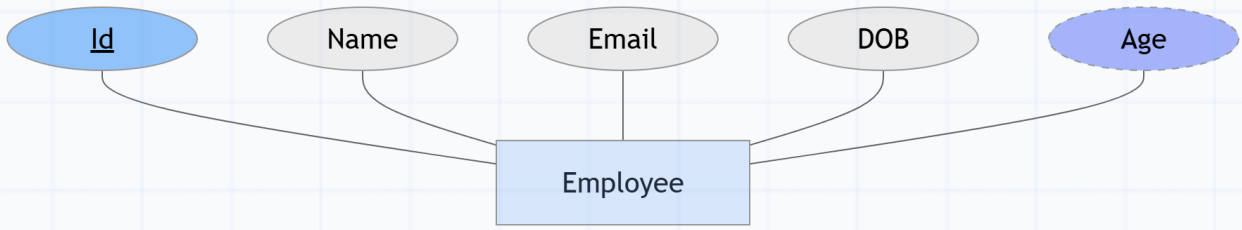


Diagram 10: Derived attribute — Age (dashed oval) is computed from the stored DOB attribute

In the example, `Age` (dashed oval) is derived from the stored `DOB` attribute.

4.2.3.6 Key Attribute

DEFINITION 13: Key Attribute

An attribute that uniquely identifies each entity instance.

The ER Key Rule

Every strong entity type must have exactly one key attribute(or one composite key attribute group) that uniquely identifies each instance. You cannot draw a valid ER diagram for a strong entity without specifying its key.

Weak entities do not have their own key — instead they have a **partial key** that, combined with the owner entity's key, forms a composite identifier.

What about Foreign Keys?

Foreign keys do **not** appear in ER diagrams. They are a relational concept — a column in one table that references the primary key of another table. Foreign keys are introduced in Chapter 5 and created automatically by the ER-to-relational mapping rules.

4.2.3.6.1 Key Attribute (Strong Entity)

A **key attribute** is drawn as an oval with its name **underlined**. It guarantees that no two instances of the entity have the same value for that attribute.

- One entity can have **only one key attribute**(or one composite key group) at the ER level.
- If two candidate identifiers exist (e.g., both `SSN` and `Email` are unique), you underline **one** in the ER diagram — the one you choose as the primary identifier. The other becomes an alternate key at the relational level.
- The key attribute in the ER diagram maps directly to the **primary key** column(s) when the entity becomes a relational table.

4.2.3.6.2 Partial Key (Weak Entity)

A **partial key** is drawn with a **dashed underline**. It is *not* globally unique — it is only unique within the set of instances owned by the same owner entity.

📖 DEFINITION 14: **Partial Key**

The attribute(s) of a weak entity that, combined with the owner entity's key, uniquely identify a weak entity instance.

Note

- The full identifier of a weak entity instance is: **owner's primary key + partial key**.
- This combination maps to a **composite primary key** in the relational table.

4.2.4 Relationships

📖 DEFINITION 15: **Relationship**

An association or connection among two or more entity types. In Chen notation, it is represented visually by a **single diamond** labeled with a verb or verb phrase describing the association.

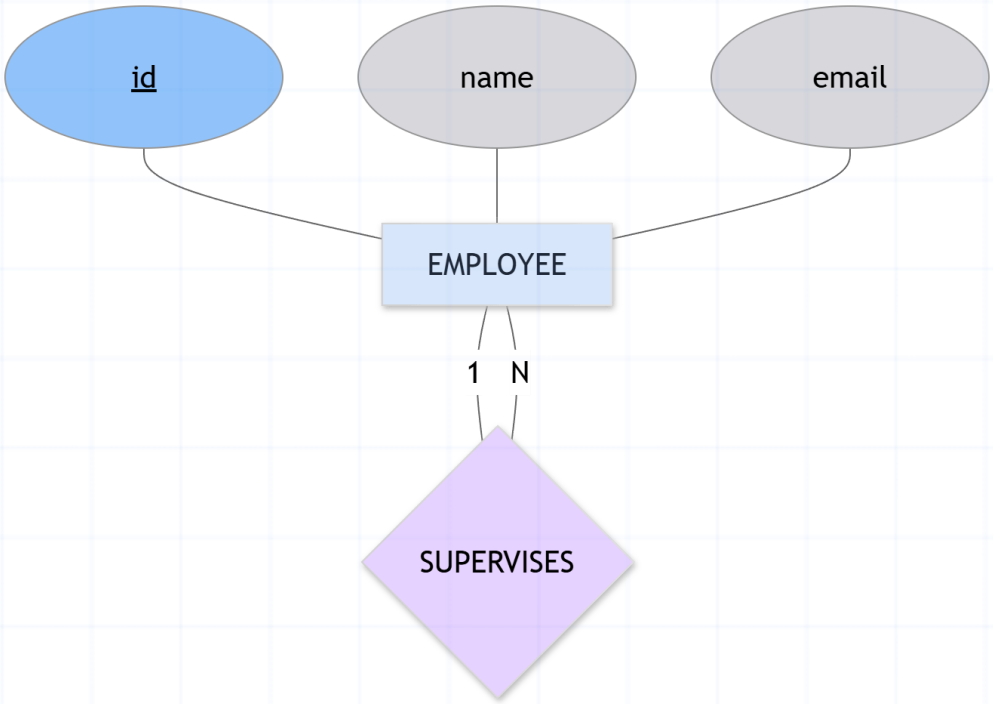
4.2.4.1 Degree of a Relationship

📖 DEFINITION 16: **Degree of a Relationship**

> The number of participating entity types in a relationship (e.g., Unary/1, Binary/2, Ternary/3).

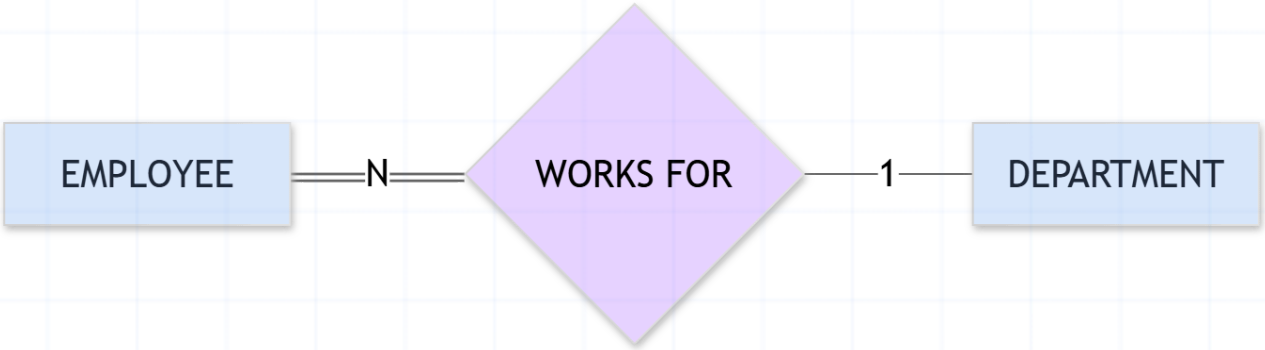
The three degrees are:

- **Unary (Recursive):** A single entity type associated with itself (e.g., an `EMPLOYEE` supervising another `EMPLOYEE`).



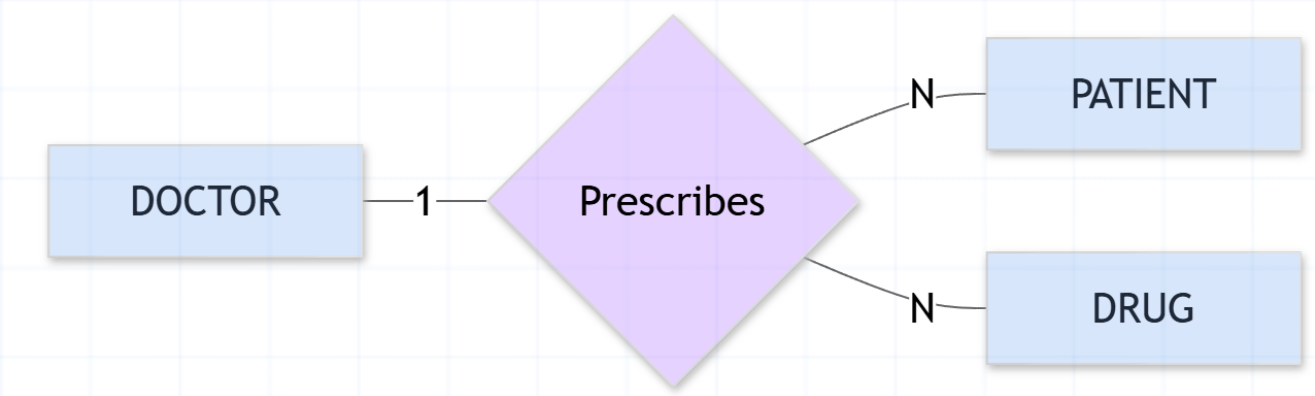
A unary (recursive) relationship where a single entity type (EMPLOYEE) interacts with itself. One employee acts as a supervisor for N supervisee employees.

- **Binary:** Connects two entity types (e.g., `EMPLOYEE` works for `DEPARTMENT`).



A binary relationship connecting two distinct entity types (EMPLOYEE and DEPARTMENT), where many employees work for one department.

- **Ternary:** Involves three entity types simultaneously.



A 1:N:N ternary relationship (Prescribes) connecting three entity types: DOCTOR, DRUG, and PATIENT.

> **Note**
>
> Most real-world relationships are **binary**. Before using a ternary relationship, ask whether it can be decomposed into two binary ones without losing information.

4.2.4.2 Cardinality Ratios

> 📖 DEFINITION 17: **Cardinality Ratio**
>
> Specifies how many instances of one entity can be associated with one instance of another entity (1:1, 1:N, M:N).

4.2.4.2.1 One-to-One (1:1)

Each instance of entity A is associated with **at most one** instance of entity B, and vice versa.

> 📝 EXAMPLE 11: **1:1 relationship — each Person has at most one Passport; each Passport belongs to exactly one Person**
>
> **Requirements:**
>
> - Each person has **at most one** passport — not everyone has a passport.
> - Each passport belongs to **exactly one** person — every passport has an owner.

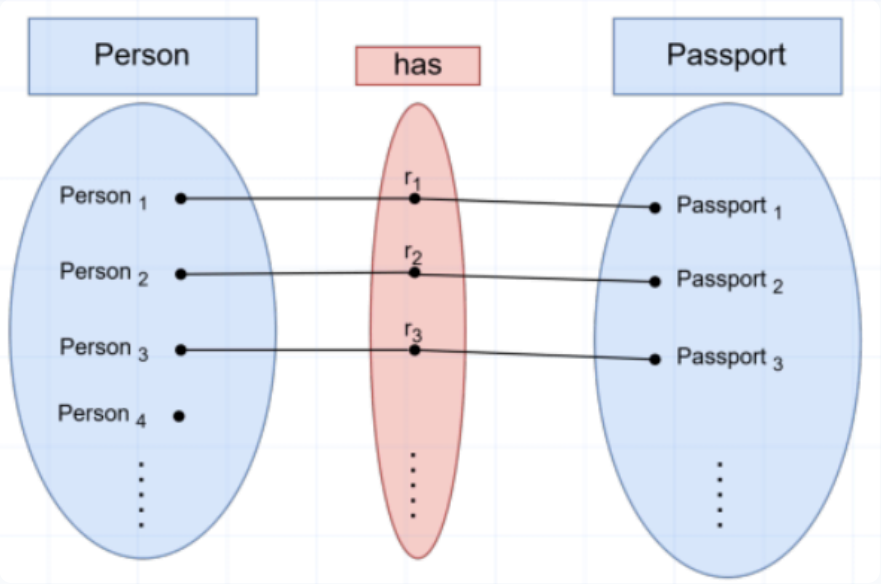


Figure 8: 1:1 relationship — each Person has at most one Passport; each Passport belongs to exactly one Person

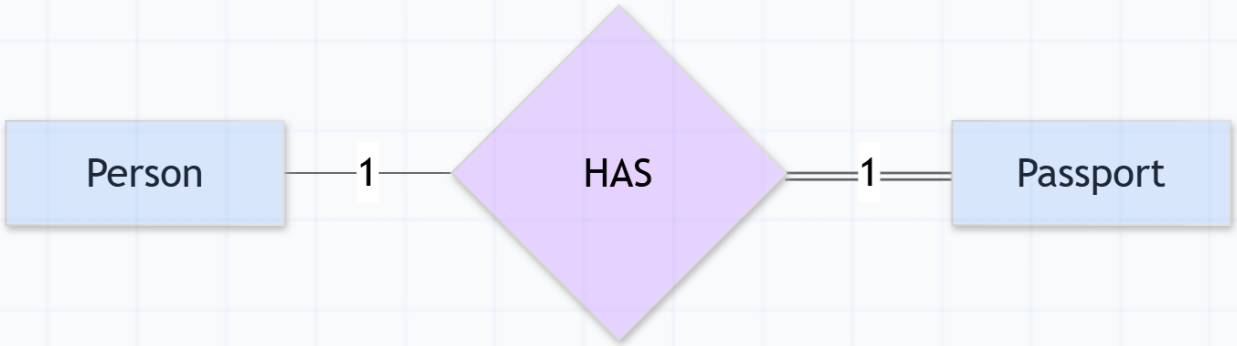


Diagram 11: 1:1 relationship — each Person has at most one Passport; each Passport belongs to exactly one Person

Reading: "A person may have zero or one passport; a passport must be owned by exactly one person."

Why Passport is modeled as a strong entity rather than a weak entity

The reason a **Passport** is modeled as a **strong entity** rather than a weak entity is precisely because it has its own unique identifier (the **Passport Number**) that can uniquely identify each passport record on its own, without needing to borrow an identifier from the `Person` entity.

Strong Entity vs. Weak Entity (It's about the Primary Key)

- **Strong Entity**: Has its own **primary key** (a unique identifier). It can be uniquely identified in the database independently.
- **Weak Entity**: **Does not have its own primary key**. It can only be uniquely identified by combining its partial identifier (discriminator) with the primary key of another entity (the owner entity).
- *Application to Passport*: A `Passport Number` is globally unique (or unique per country). Therefore, `Passport` has its own primary key and is a **strong entity**.

4.2.4.2.2 One-to-Many (1:N)

One instance of entity A is associated with **many** instances of entity B, but each B instance is associated with at most one A.

EXAMPLE 12: 1:N relationship — one Department employs many Employees; each Employee belongs to exactly one Department

Requirements:

- A department can exist **without** employees (e.g., a newly created department with no staff yet) →**partial** participation of Department in the relationship.
- An employee **must** be assigned to a department →**total** participation of Employee in the relationship.

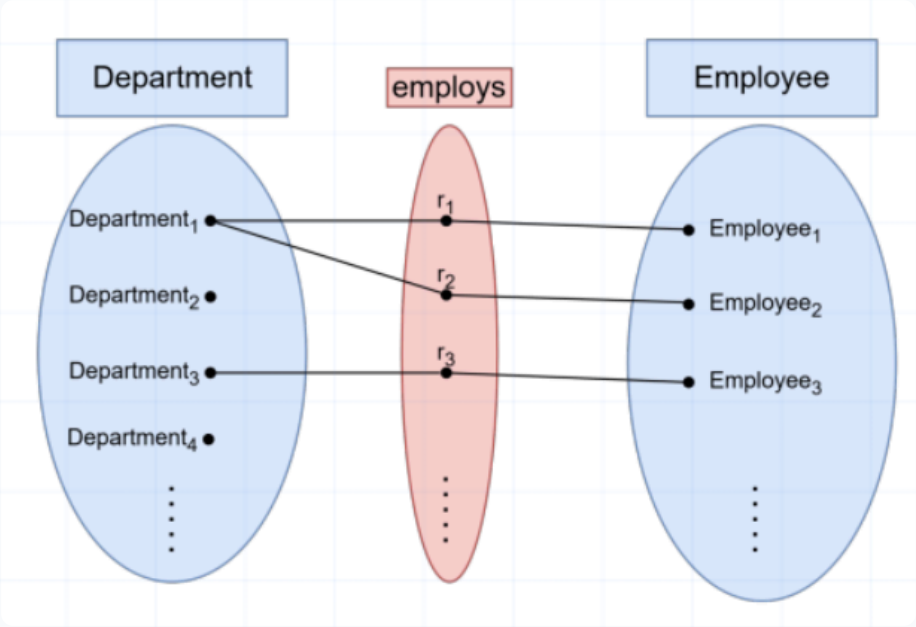


Figure 9: 1:N relationship — one Department employs many Employees; each Employee belongs to exactly one Department

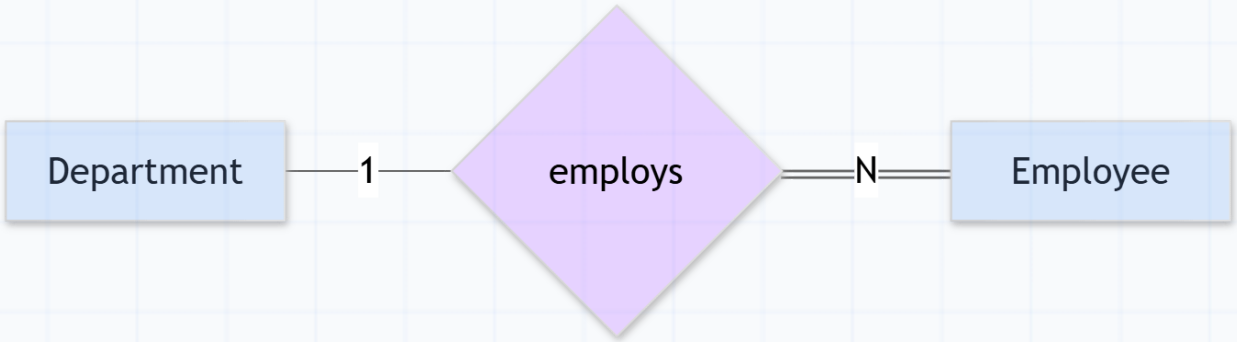


Diagram 12: 1:N relationship — one Department employs many Employees; each Employee belongs to exactly one Department

Reading:

- **Left → Right (Department → Employee):**One department employs zero or more employees.
- **Right → Left (Employee → Department):**Each employee works for exactly one department.

4.2.4.2.3 Many-to-Many (M:N)

Each instance of entity A can be associated with **many** instances of entity B, and each B can be associated with many A's.

EXAMPLE 13: M:N relationship — many Students enroll in many Courses; many Courses have many Students

Requirements:

- One student can enroll in multiple courses, and one course can have multiple students enrolled.
- A student can exist in the system without currently being enrolled in any course (e.g., a newly admitted student).
- A course can exist in the system without any enrolled students (e.g., a newly created course section).

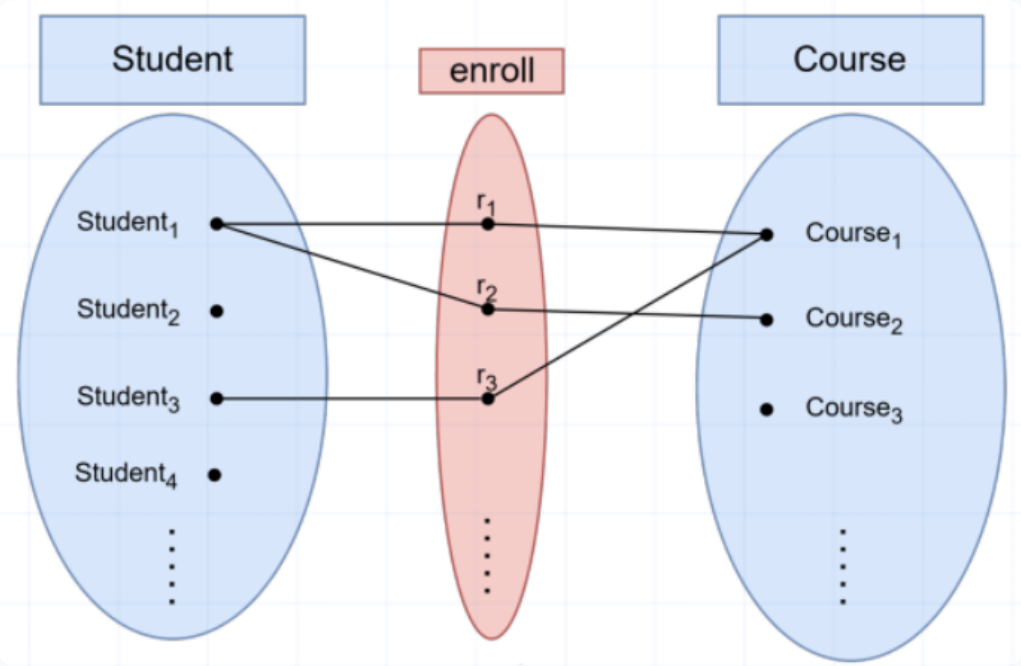


Figure 10: M:N relationship — many Students enroll in many Courses; many Courses have many Students

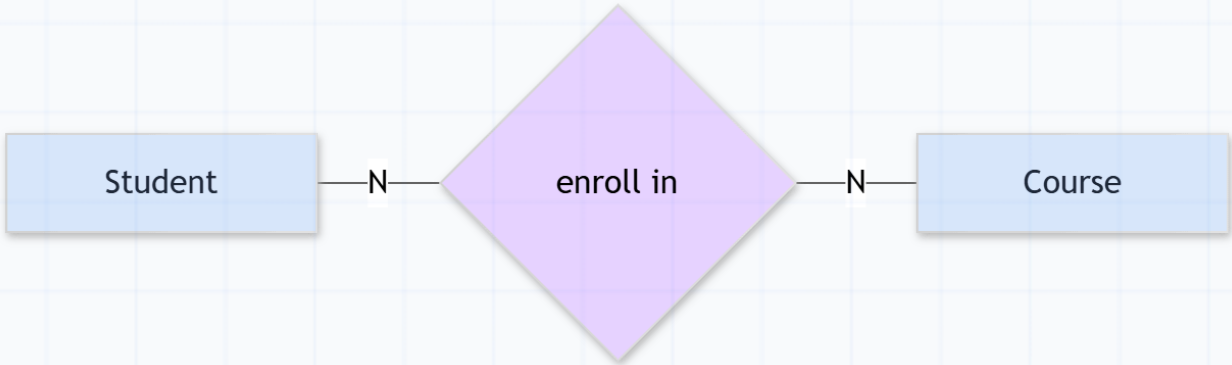


Diagram 13: M:N relationship — many Students enroll in many Courses; many Courses have many Students

Reading:

- **Left → Right (Student → Course):**A student may enroll in zero or more courses.
  Right → Left (Course → Student):A course may have zero or more students.

4.2.4.3 Participation Constraints

📖 **DEFINITION 18: Participation Constraint**

Specifies whether every instance of an entity must participate in a relationship (Total) or if participation is optional (Partial).

Table 10: *Participation Constraints Visual and Conceptual Summary*

Line type	Name	Meaning
Single line —	Partial participation	Some instances may not participate
Double line ═	Total participation	Every instance must participate

📝 **EXAMPLE 14: Department managed By Employee**

- **Department** side →**double line (total)**: every department *must* be managed by exactly one employee.
- **Employee** side →**single line (partial)**: not every employee is a manager — an employee *may* manage a department, but doesn't have to.

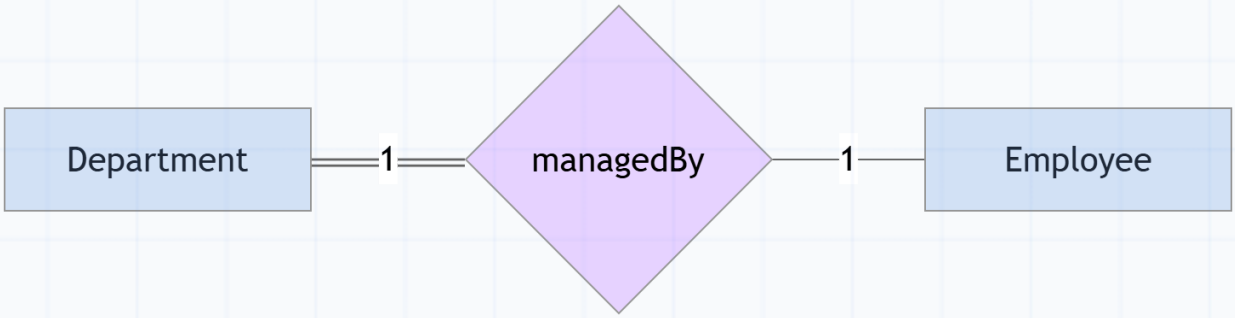


Diagram 14: Department managed By Employee

ⓘ **Note**

Total participation means the entity **cannot exist** in the database without being connected to this relationship. A department record cannot be inserted unless it has a managing employee assigned to it.

4.2.4.4 Relationship Attributes

📖 **DEFINITION 19: Relationship Attribute**

Data or a value that belongs specifically to the pairing of entities rather than to either entity alone.

📝 **EXAMPLE 15: WORKS_ON relationship attribute**

`Hours` in *WORKS_ON* is a **relationship attribute**: it is the number of hours *this employee* spends on *this project*.

- Storing it with `EMPLOYEE` is wrong: hours on which project?
- Storing it with `PROJECT` is wrong: hours by which employee?
- It belongs to the *combination* `(Employee, Project)` — i.e., the relationship.

In Chen notation, relationship attributes are drawn as ovals attached to the diamond.

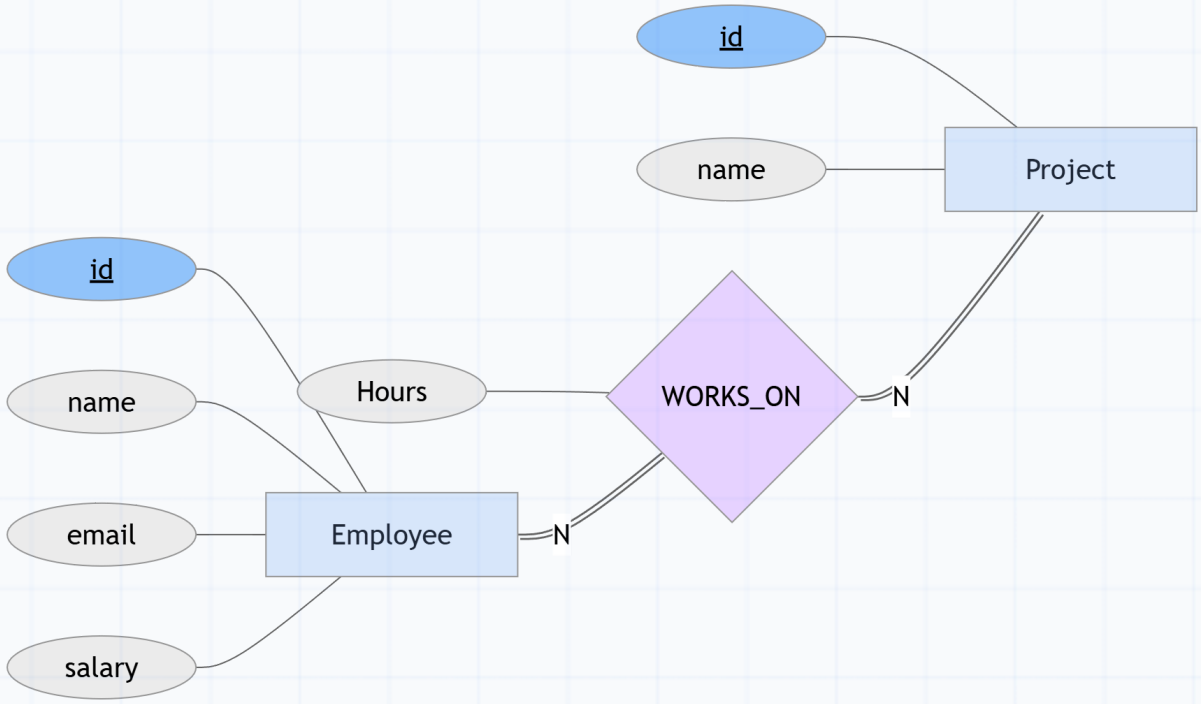


Diagram 15: WORKS_ON relationship attribute — Hours is attached to the WORKS_ON diamond because it belongs to the (Employee, Project) pairing, not to either entity alone

4.2.4.5 Recursive (Unary) Relationships

A relationship where the **same entity type** participates more than once, in different **roles**.

EXAMPLE 16: **Recursive relationship : Supervisor is a role of Employee**

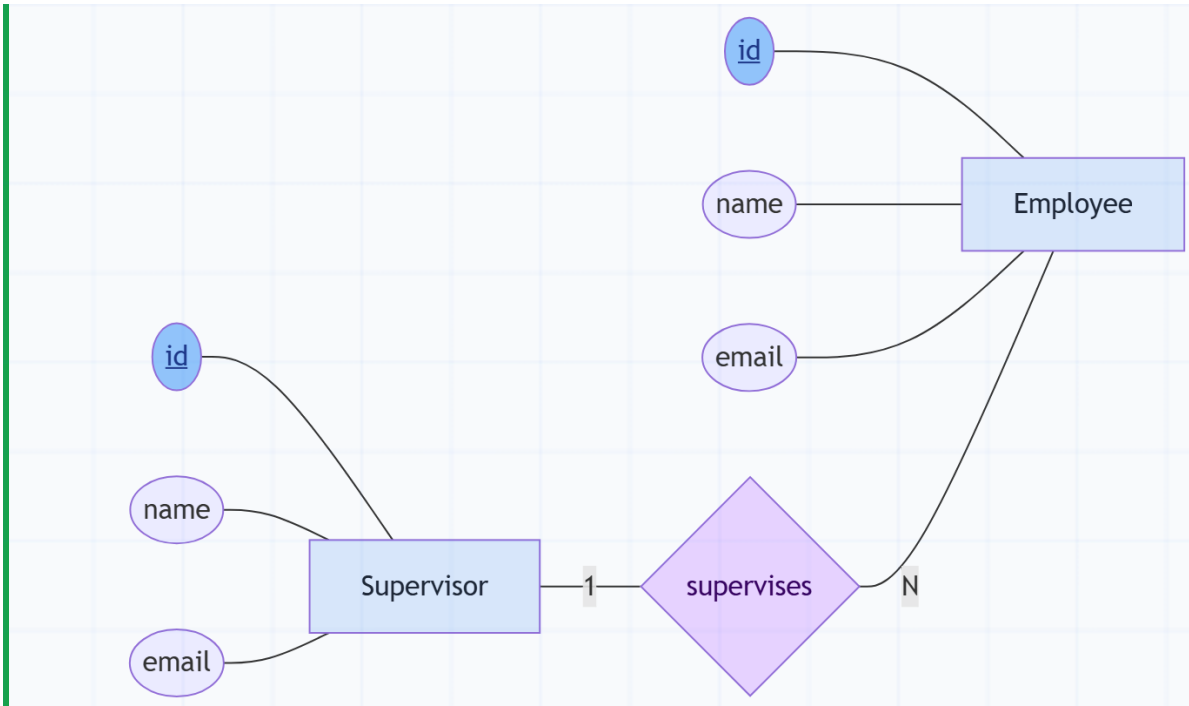


Diagram 16: Supervisor is a role of Employee (wrong)

Why the first diagram is wrong: `Supervisor` is not a different kind of thing from `Employee` — a supervisor *is* an employee who happens to manage others. Drawing two separate entity boxes implies two different tables in the database, which is incorrect.

> **Note**
>
> **Partial Participation on Both Ends:** Use single lines for both connections because an employee can exist without supervising anyone (regular workers), and a top-level manager can exist without having a supervisor above them.

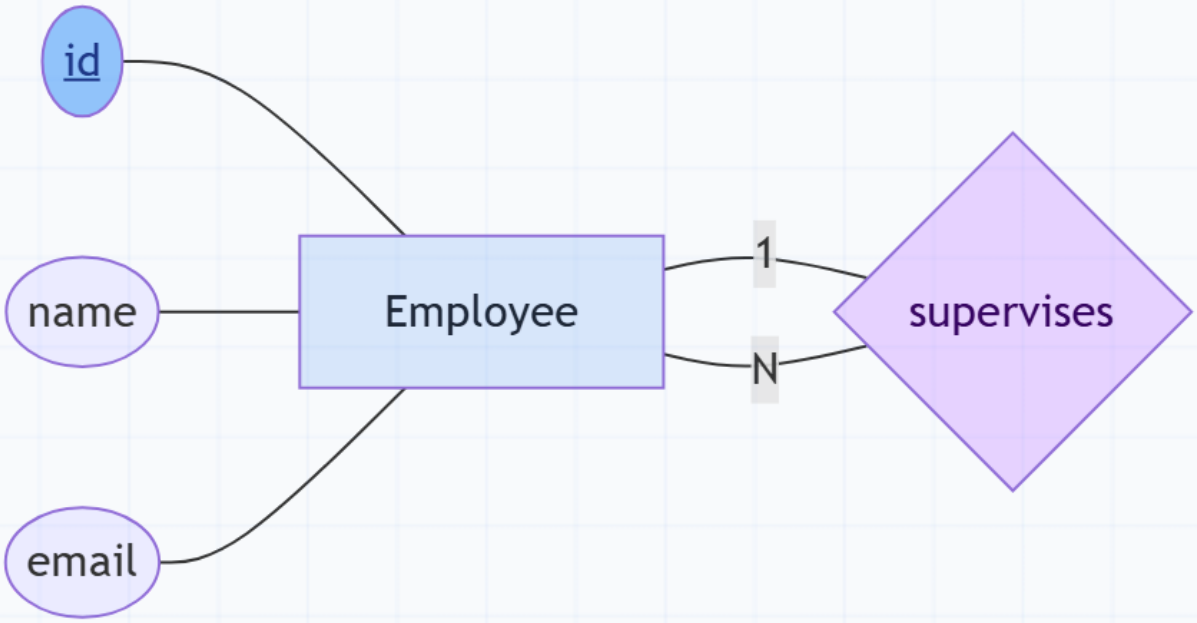


Diagram 17: Supervisor is a role of Employee (correct)

> **Why the second diagram is correct:** A single `Employee` entity with a **recursive** `SUPERVISION` relationship back to itself, with two labelled roles: `Supervisor` (1 side) and `Supervisee` (N side). One entity maps to one table — the role is just a label on the line.

4.3 Worked Example: The Company Database

> **Note**
>
> This section builds the **Company Database ER diagram** incrementally — one requirement at a time. For each step, the relevant sentence from the requirements is highlighted, followed by the updated diagram and an explicit note on any assumption made.

4.3.1 Requirements

The company has several **departments**. Each department has a unique **name** and a **number**. A department is managed by exactly one **employee**, who is called the **manager**. The manager has a **start date** for their managerial role. A department may have several **locations**.

Each employee has a unique **social security number (SSN)**, a **first name**, **middle initial**, **last name**, **date of birth**, **address**, **gender**, and **salary**. An employee's **age** can be derived from their date of birth and does not need to be stored separately. Each employee works in exactly one department but may work on several **projects**. We track the **number of hours** per week each employee works on each project. Each employee may be supervised by at most one other employee.

Each project has a unique **name**, a **number**, and a single **location**. Each project is controlled by exactly one department.

Employees may have **dependents**. Each dependent has a **name**, **gender**, **date of birth**, and **relationship** to the employee.

4.3.2 Step 1 — EMPLOYEE Entity

> **From Requirements**
>
> Each employee has a unique social security number (SSN), a first name, middle initial, last name, date of birth, address, gender, and salary. An employee's age can be derived from their date of birth and does not need to be stored separately.

We start with the most described entity: EMPLOYEE.

- `SSN` →**key attribute**(underlined oval) — uniquely identifies each employee
- `Name` →**composite attribute**— decomposes into `Fname` , `Minit` , `Lname`
- `Bdate` → stored attribute — the value is physically kept in the database

- `Age` →**derived attribute**(dashed oval) — computed from `Bdate`; never stored
- `Address`, `Gender`, `Salary` → simple, single-valued attributes

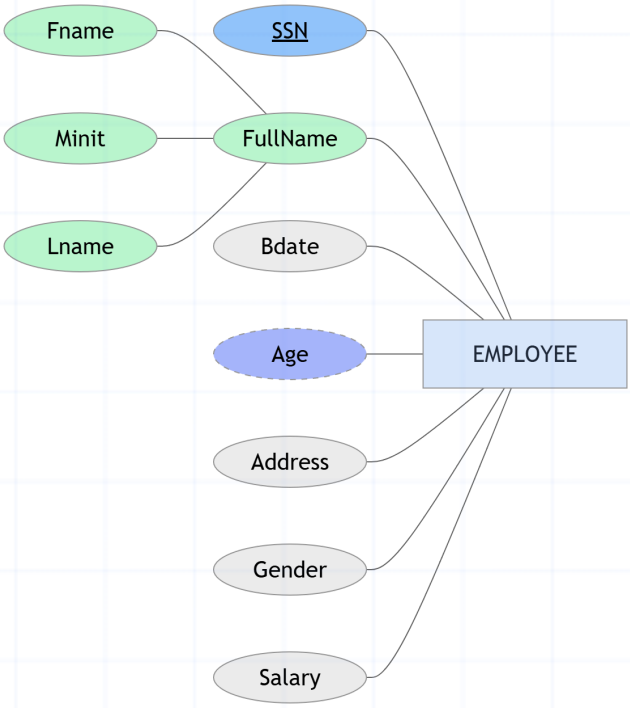


Diagram 18: Step 1 — EMPLOYEE entity with all its attributes

4.3.3 Step 2 — DEPARTMENT Entity + MANAGES Relationship

> **From Requirements**
>
> The company has several departments. Each department has a unique name and a number. A department may have several locations. A department is managed by exactly one employee, who is called the manager. The manager has a start date for their managerial role.

We start with the most described entity: EMPLOYEE.

- `DEPARTMENT` is a strong entity; `Number` is its key attribute
- `Locations` →**multivalued attribute**(double oval) —*"may have several"*
- `Start_date` →**relationship attribute**on MANAGES — it belongs to the (Employee, Department) pairing, not to either entity alone
- **Department side of MANAGES:***"exactly one"*→**total**participation, max = 1 (double line, label 1)
- **Employee side of MANAGES:**The requirement does not say every employee is a manager. "Manager" is a*role*played by some employees, not a separate entity type.

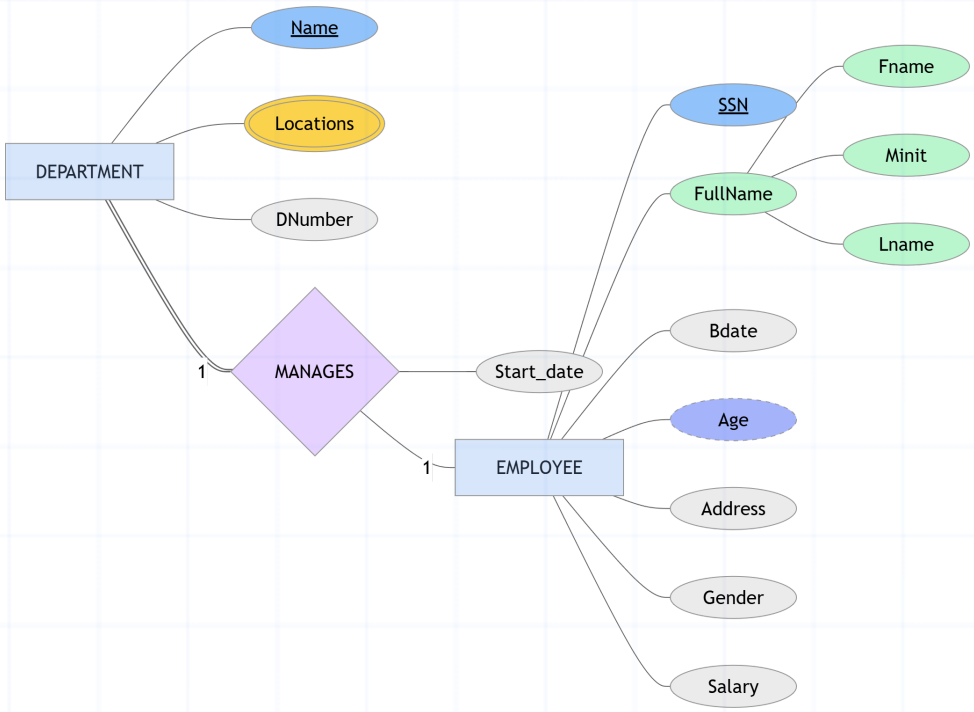


Diagram 19: Step 2 — DEPARTMENT Entity + MANAGES Relationship

4.3.4 Step 3 — WORKS_FOR Relationship

> **From Requirements**
>
> Each employee works in exactly one department.

WORKS_FOR connects EMPLOYEE and DEPARTMENT with cardinality N:1.

- **Employee side:** *"exactly one"* →**total** participation (double line), max = 1
- **Department side:** The requirement only states where employees go; it says nothing about whether a department must have employees.

> **Assumption — WORKS_FOR (Department side)**
>
> **Stated:** Every employee must belong to exactly one department → Employee side is **total**.
>
> **Assumption:** The diagram shows a **single line** on the Department side, meaning a department can exist with zero employees assigned to it → Department side is **partial**, min = 0, max = N.
>
> *This is a design choice. You could require every department to have at least one employee by making their side total (double line).*

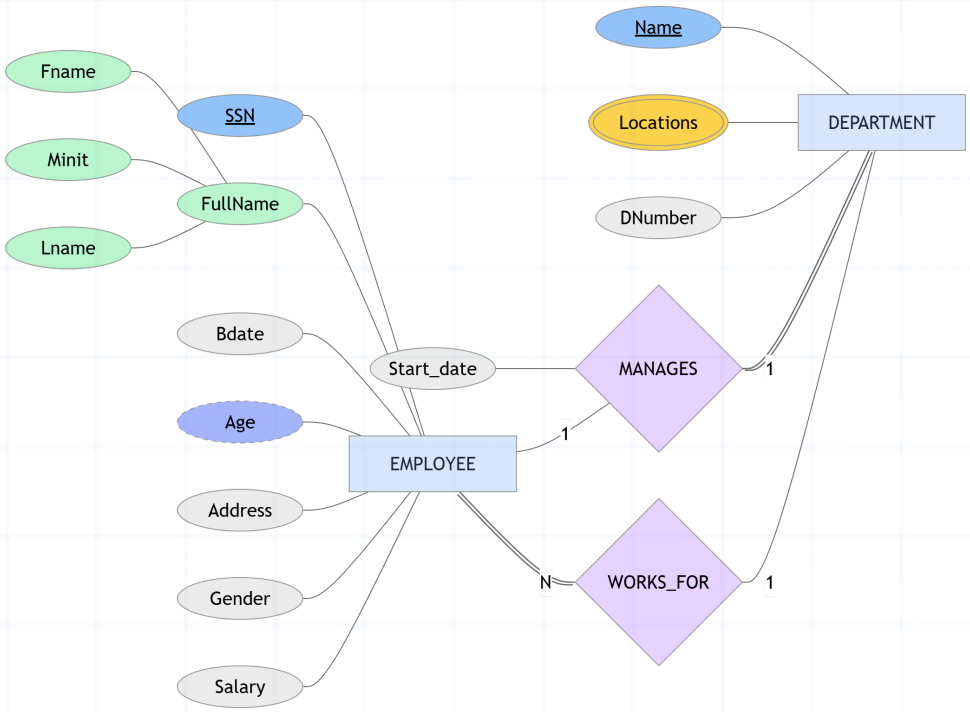


Diagram 20: Step 3 — WORKS_FOR Relationship

4.3.5 Step 4 — PROJECT Entity

> **From Requirements**
>
> Each project has a unique name, a number, and a single location.

PROJECT is a strong entity with three attributes.

- `Number` →**key attribute**(underlined oval)
- `Name` → simple attribute (unique per requirements, but Number is the diagrammatic key)
- `Location` → simple, single-valued attribute —*"a single location "*confirms no double oval needed

> **Assumption — WORKS_FOR (Department side)**
>
> **Stated:** Every employee must belong to exactly one department → Employee side is **total**.
>
> **Assumption:** The diagram shows a **single line** on the Department side, meaning a department can exist with zero employees assigned to it → Department side is **partial**, min = 0, max = N.
>
> *This is a design choice. You could require every department to have at least one employee by making their side total (double line).*

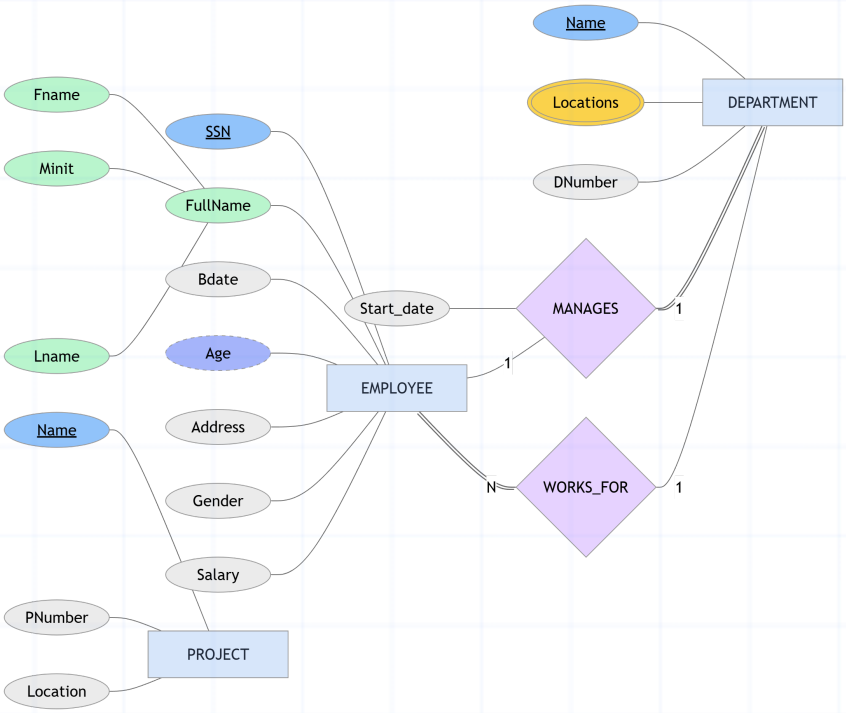


Diagram 21: Step 4 — PROJECT Entity

4.3.6 Step 5 — CONTROLS Relationship

> **From Requirements**
>
> Each project is controlled by exactly one department.

CONTROLS connects PROJECT and DEPARTMENT with cardinality N:1.

- **Project side:** *"exactly one"*→ **total** participation (double line), max = 1
- **Department side:** The requirement says nothing about whether a department must control any projects.

> **Assumption — CONTROLS (Department side)**
>
> **Stated:** Every project is controlled by exactly one department → Project side is **total**.
>
> **Assumption:** A department may control zero or many projects → Department side is **partial**, min = 0, max = N.
>
> *The requirements do not require every department to control a project.*

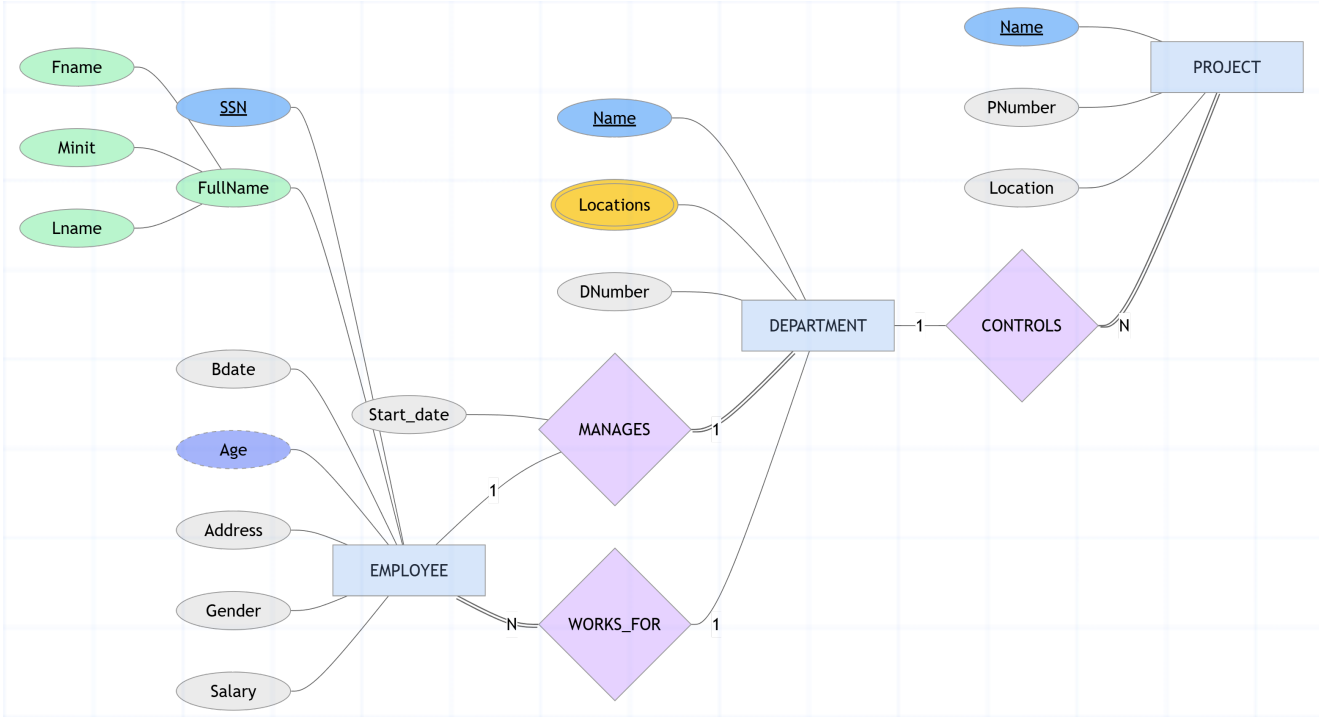


Diagram 22: Step 5 — CONTROLS Relationship

4.3.7 Step 6 — WORKS_ON Relationship

> **From Requirements**
>
> *Each employee **may** work on several **projects**. We track the **number of hours** per week each employee works on each project.*

WORKS_ON connects EMPLOYEE and PROJECT with cardinality M:N.

- `Hours` → **relationship attribute** on WORKS_ON — it belongs to the (Employee, Project) pairing, not to either entity alone
- **Employee side:** The word **"may"** explicitly makes this **partial**(min = 0), max = N
- **Project side:** Not stated — see assumption below.

> **Assumption — WORKS_ON (Project side)**
>
> **Stated:** *"May work on"*→ Employee participation is **partial**— directly from requirements.
>
> **Assumption:** A project that has no employees assigned is not yet operational → Project side is **total**, min = 1, max = N.
>
> *This assumption must be confirmed with stakeholders — it is not stated in the text.*

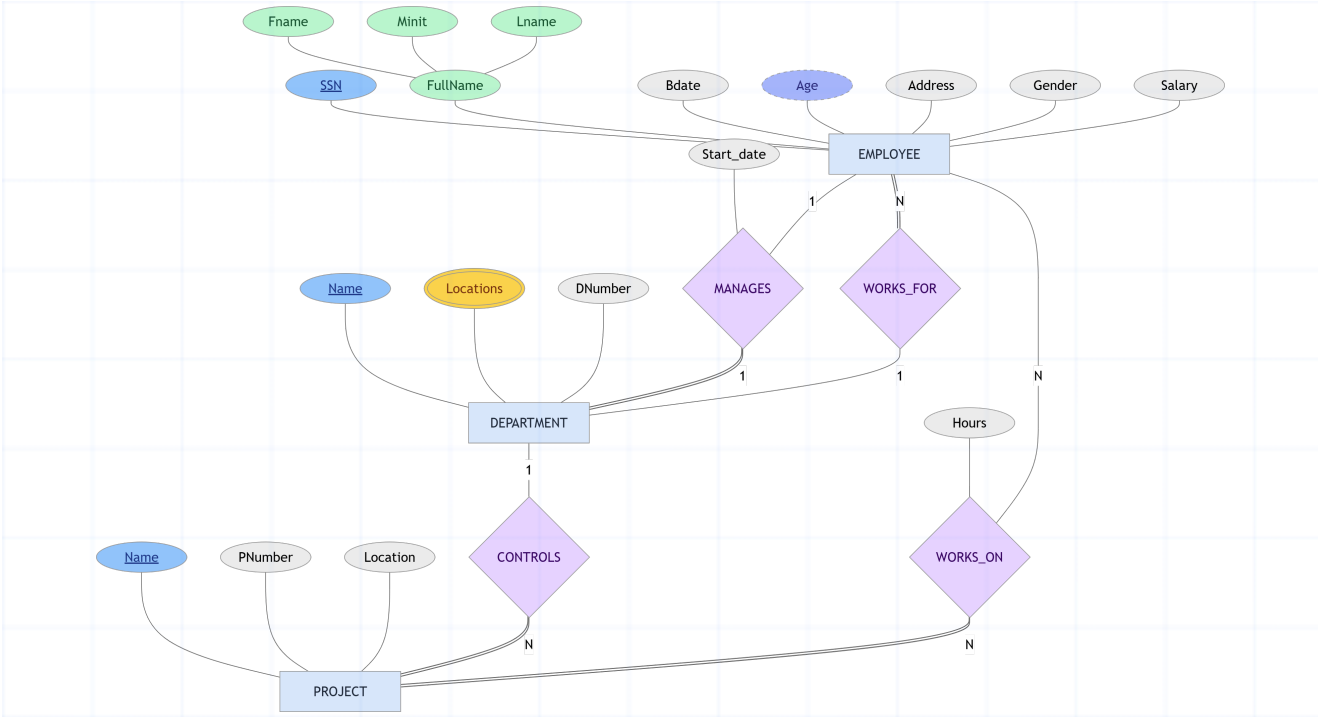


Diagram 23: Step 6 — WORKS_ON Relationship

4.3.8 Step 7 — SUPERVISION Relationship

> **From Requirements**
>
> *Each employee may be supervised by at most one other employee.*

SUPERVISION is a recursive (unary) relationship — EMPLOYEE related to itself, with two roles: Supervisor and Supervisee.

- **Supervisee side:** *"may be supervised"* → **partial**(min = 0);*"at most one"*→ max = 1
- **Supervisor side:** Not stated — see assumption below.
- **Project side:** Not stated — see assumption below.

> **Assumption — SUPERVISION (Supervisor side)**
>
> **Stated:** Each employee *may* be supervised by at most one → Supervisee side is **partial, max = 1**.
>
> **Assumption:** Not every employee supervises anyone (e.g., a regular worker has no one under them; the top-level manager also has no supervisor) → Supervisor side is **partial**, min = 0, max = N.

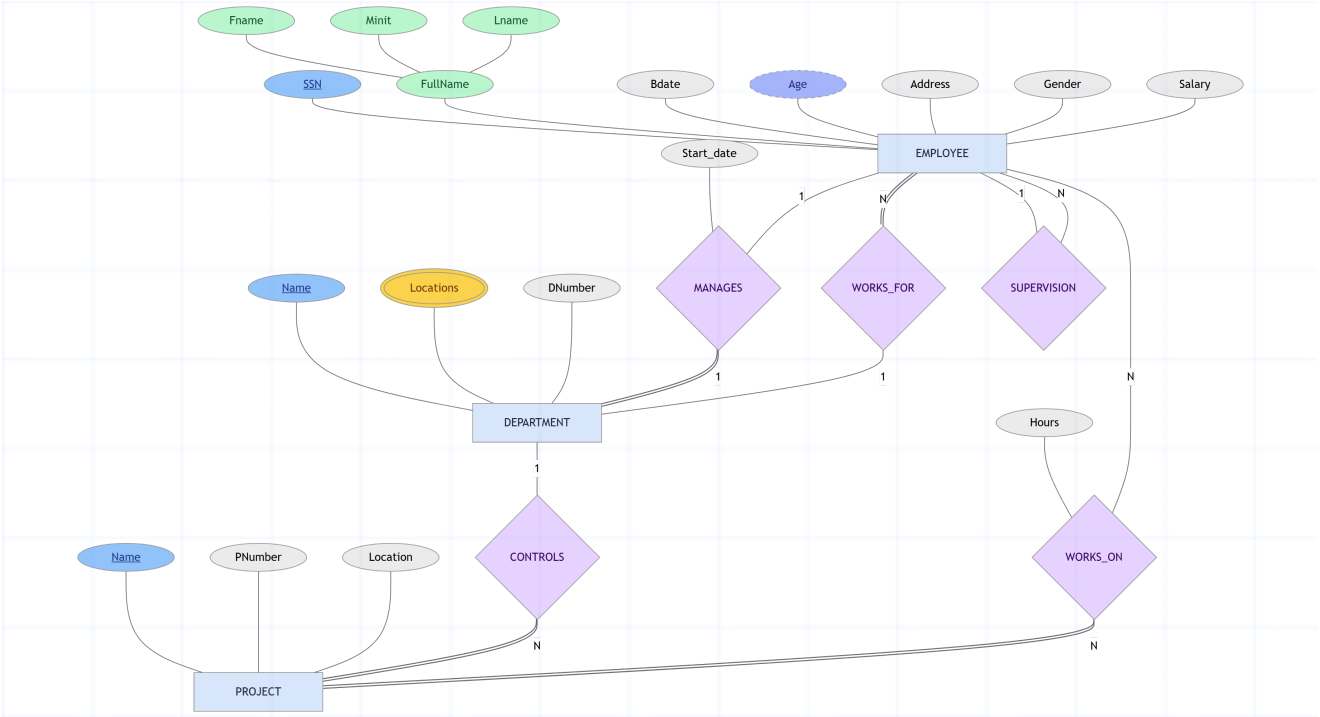


Diagram 24: Step 7 — SUPERVISION Relationship

4.3.9 Step 8 — DEPENDENT Weak Entity + HAS (Full Diagram)

> **From Requirements**
>
> *Employees may have dependents. Each dependent has a name, gender, date of birth, and relationship to the employee.*

DEPENDENT is a weak entity — it has no key of its own and cannot exist without its owning employee.

- `Name` → **partial key**(dashed-underline oval) — unique within one employee's dependents, not globally
- `Gender` , `Bdate` , `Relationship` → simple attributes

- `HAS` is the **identifying relationship**(double diamond) — the combination `(SSN, Name)` uniquely identifies each dependent
- **Employee side:** *"may have"* →**partial**(min = 0, max = N)
- **Dependent side:** A dependent cannot exist without an employee →**total**(min = 1) — this follows structurally from the weak entity definition, not from an assumption

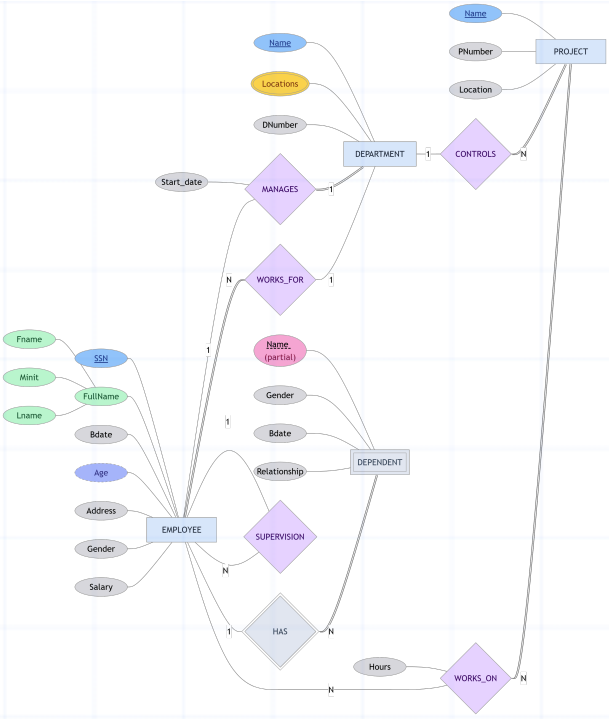


Diagram 25: Step 8 — DEPENDENT Weak Entity + HAS (Full Diagram)

5 Min-Max Notation

Chen notation tells us *whether* a participation is total or partial and *what* the cardinality ratio is (1:1, 1:N, M:N), but it stops short of answering a frequent business question: *exactly how many*? “At most three active orders,” “between two and five team members,” “exactly one department chair” — these rules all collapse into Chen’s coarse labels. **Min-Max notation** was introduced precisely to close that gap: it keeps the same diagram shape but replaces each line-end label with a numeric pair (min, max) that pins down both bounds explicitly.

5.1 What Do min and max Mean?

- **min**— the minimum number of relationship instances this entity *must* participate in:
  - min=0→ participation is **optional**(equivalent to a single line in Chen)
  - min≥1→ participation is **mandatory**(equivalent to a double line in Chen)
- **max**— the maximum number of relationship instances this entity *can* participate in:
  - max=1→ at most one
  - max=N(or∞) → no upper limit

(min,max) pair	English reading	Chen equivalent
(0,1)	“zero or one”	Partial, cardinality 1
(1,1)	“exactly one”	Total, cardinality 1
(0,N)	“zero or more”	Partial, cardinality N
(1,N)	“one or more”	Total, cardinality N
(4,N)	“at least four”	Total, cardinality N

Why use Min-Max instead of Chen labels?

Chen notation labels each relationship line with 1, N, or M — this captures the *maximum* cardinality but not the *minimum*. Min-Max adds the minimum, making it immediately clear whether participation is optional or mandatory without relying on single-vs-double lines. It is especially useful when writing precise business rules for a development team to implement.

5.2 Placement Rule

The (min, max) pair is written on the line **next to the entity it constrains**. The general form is:

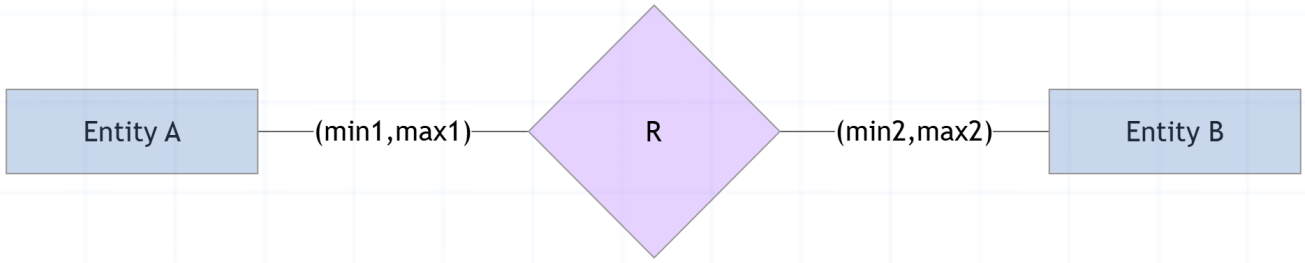


Diagram 26: General form of Min-Max placement on a binary relationship.

- An instance of Entity A may be related to at least min1 and at most max1 instances of Entity B.
- Likewise, min2 is the minimum number and max2 is the maximum number of instances of Entity A in to which Entity B is related.

EXAMPLE 17: **Min-Max notation on an EMPLOYEE–DEPARTMENT relationship (illustrative values).**

The diagram below shows Min-Max annotation on a generic EMPLOYEE–DEPARTMENT relationship. The values here are **chosen to illustrate specific constraint patterns** and are independent of the Company Database worked example.

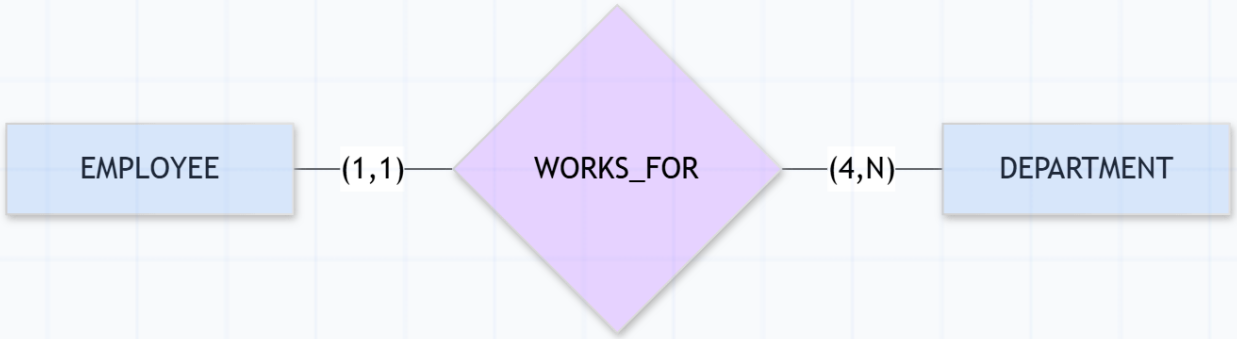


Diagram 27: Min-Max notation on an EMPLOYEE–DEPARTMENT relationship (illustrative values).

Pair	Placed next to	Meaning
(1,1)	EMPLOYEE	Every employee is associated with exactly one department — mandatory and singular.
(4,N)	DEPARTMENT	Every department has at least 4 employees — a specific minimum business rule.

5.3 Company Database in Min-Max Notation

The table below re-expresses every relationship from the Worked Example using (min,max) pairs, consistent with the Full Constraint Summary and the Chen diagram.

Relationship	Entity A	(min,max) on A	Entity B	(min,max) on B
WORKS_FOR	EMPLOYEE	(1,1)	DEPARTMENT	(0,N)
MANAGES	EMPLOYEE	(0,1)	DEPARTMENT	(1,1)
WORKS_ON	EMPLOYEE	(0,N)	PROJECT	(1,N)
CONTROLS	PROJECT	(1,1)	DEPARTMENT	(0,N)
SUPERVISION	EMPLOYEE (Supervisee)	(0,1)	EMPLOYEE (Supervisor)	(0,N)
HAS	EMPLOYEE	(0,N)	DEPENDENT	(1,1)

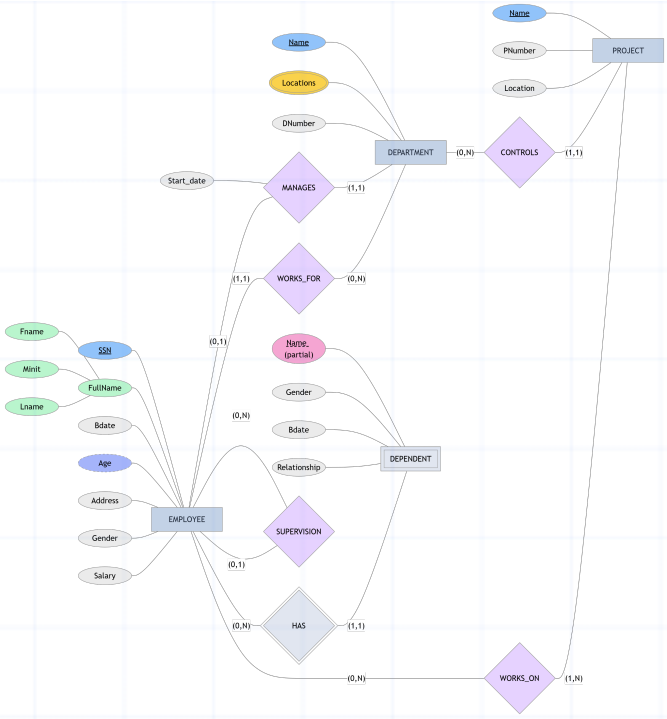


Diagram 28: Full Company ER diagram annotated with Min-Max notation.

5.4 Chen vs Min-Max: Side-by-Side Examples

EXAMPLE 18: Professor CHAIRS Department

The university has several departments. Each department must have exactly one chairperson who must be a professor. A professor can chair at most one department. Some professors may not chair any department — the chairperson role is optional for professors. If a professor is appointed as chair, they must chair exactly one department. No department can exist without a chairperson.

> **Note**
>
> A department chair is*a faculty member in a given academic department*, assigned by the dean to manage that department.

Show Solution

Relationship: CHAIRS

Property	Entity A — Professor	Entity B — Department
Participation	Partial (optional)	Total (mandatory)
(min,max)	(0,1)	(1,1)
Chen cardinality label	1	1

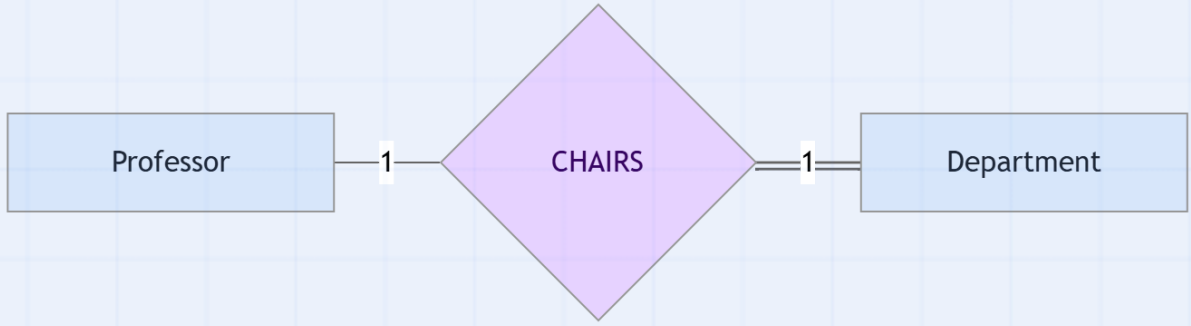


Diagram 29: Professor CHAIRS Department (Chen)

How to read — Chen:

- **Left → Right (LTR):** Each Professor chairs **0 or 1** Department.
- **Right → Left (RTL):** Each Department is chaired by **exactly 1** Professor.
- The **double line** on the Department side means Department participation is **total** (mandatory).
- The **single line** on the Professor side means Professor participation is **partial** (optional).

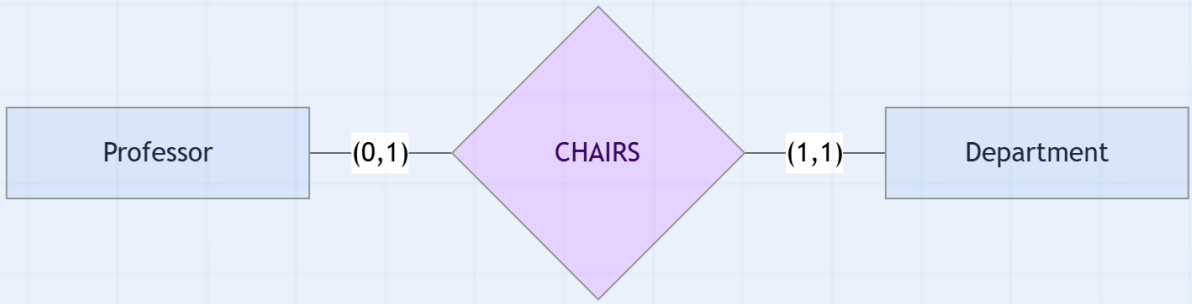


Diagram 30: Professor CHAIRS Department (Min-Max)

How to read — Min-Max:

- A professor chairs **at most one** department — but is not required to chair any.
- Every department has **exactly one** chair — mandatory, and only one.

> **Key insight**
>
> Chen told us the *ratio* (1:1) but left participation implicit in the line style. Min-Max spells out both the minimum and maximum in a single, self-documenting pair.

EXAMPLE 19: Student HOLDS_PERMIT Parking_Permit

Students may obtain a parking permit from the university. A student can have at most one parking permit. Some students don't have cars and therefore have no permit. Each parking permit is assigned to exactly one student, but permits can exist in the system before being assigned. If a student leaves, their permit becomes available for reassignment.

Show Solution

Relationship: HOLDS_PERMIT

Property	Entity A — Student	Entity B — Parking_Permit
Participation	Partial (optional)	Partial (optional)
(min,max)	(0,1)	(0,1)
Chen cardinality label	1	1

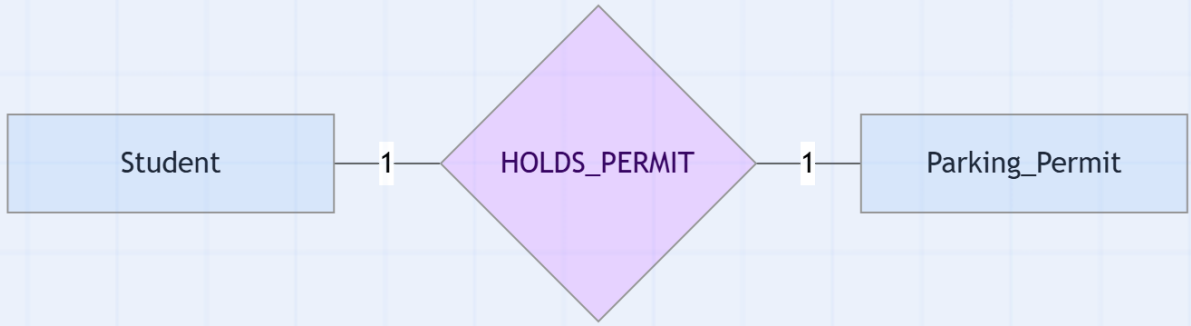


Diagram 31: Student HOLDS_PERMIT Parking_Permit (Chen)

How to read — Chen:

- **Left → Right (LTR):** Each Student holds **0 or 1** Parking Permit.
- **Right → Left (RTL):** Each Parking Permit is held by **0 or 1** Student.
- **Single line** on both sides → participation is **partial** on both ends (neither student nor permit is required to be matched).

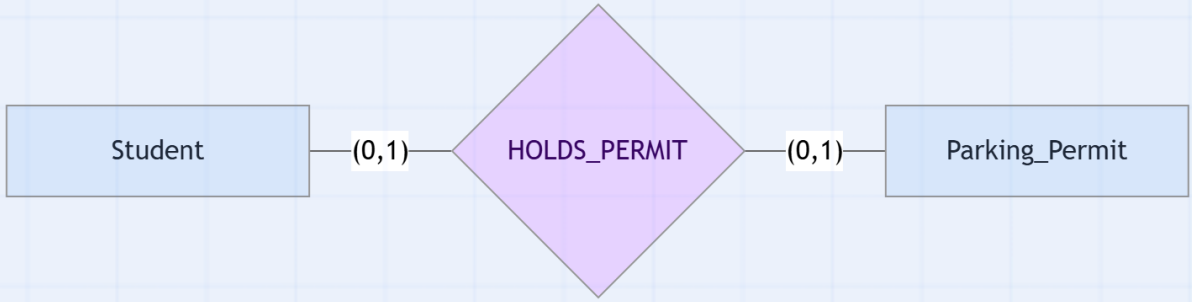


Diagram 32: Student HOLDS_PERMIT Parking_Permit (Min-Max)

How to read — Min-Max:

- A student holds **at most one** parking permit — but may hold none.
- A parking permit is held by **at most one** student — but may be unassigned.

> **Key insight**
>
> Both sides are optional. Min-Max makes this unambiguous without having to check whether lines are single or double.

> **Note**
>
> The full implementation of reassignment — `ON DELETE SET NULL`, status trigger, and the design discussion `SET NULL` vs. `CASCADE`

EXAMPLE 20: Department OFFERS Course

Every department in the university must offer at least one course. A course must be offered by exactly one department. Departments can offer multiple courses. Courses cannot exist without being associated with a department. New departments must create at least one course within their first semester.

✅ Show Solution

Relationship: OFFERS

Property	Entity A — Department	Entity B — Course
Participation	Total (mandatory)	Total (mandatory)
(min,max)	(1,N)	(1,1)
Chen cardinality label	N	1



Diagram 33: Department OFFERS Course (Chen)

How to read — Chen:

- **Left → Right (LTR):** Each Department offers **1 or more** Courses.
- **Right → Left (RTL):** Each Course is offered by **exactly 1** Department.
- **Double line** on both sides → participation is **total** on both ends.



Diagram 34: Department OFFERS Course (Min-Max)

How to read — Min-Max:

- Every department offers **at least one** course (no upper limit).
- Every course is offered by **exactly one** department.

> **Key insight**
>
> Chen's double lines on both sides simply said "mandatory on both ends." Min-Max adds the upper bound: a department can offer *many* courses, but a course belongs to *only one* department.

> **Warning**
>
> The static ER diagram cannot capture the *timing* rule "within the first semester." The enforcement trigger

EXAMPLE 21: Professor ADVISES Student

Professors may serve as academic advisors to students. A professor can advise multiple students. Some professors choose not to advise any students. A student may have at most one academic advisor. Some students, particularly freshmen, may not have an assigned advisor yet. Each advisor-student relationship is formalized through the advising system.

Show Solution

Relationship: ADVISES

Property	Entity A — Professor	Entity B — Student
Participation	Partial (optional)	Partial (optional)
(min,max)	(0,N)	(0,1)
Chen cardinality label	N	1

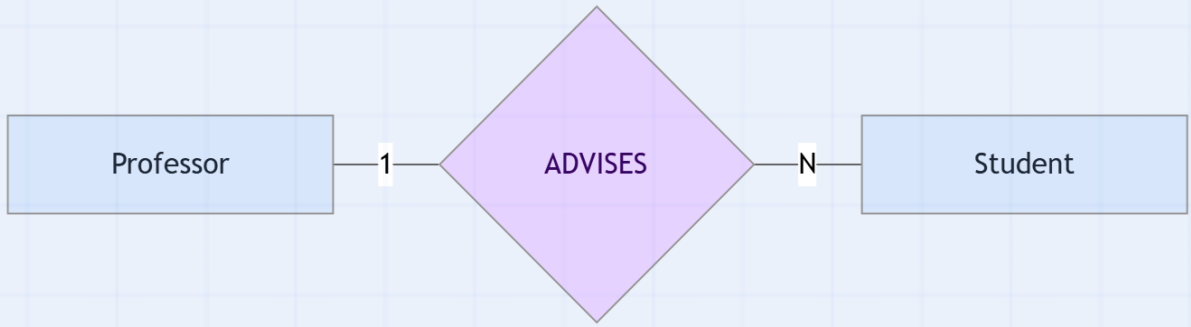


Diagram 35: Professor ADVISES Student (Chen)

How to read — Chen:

- **Left → Right (LTR):** Each Professor advises **0 or more** Students.
- **Right → Left (RTL):** Each Student is advised by **0 or 1** Professor.

- ◦ **Single line** on both sides → participation is **partial** on both ends.

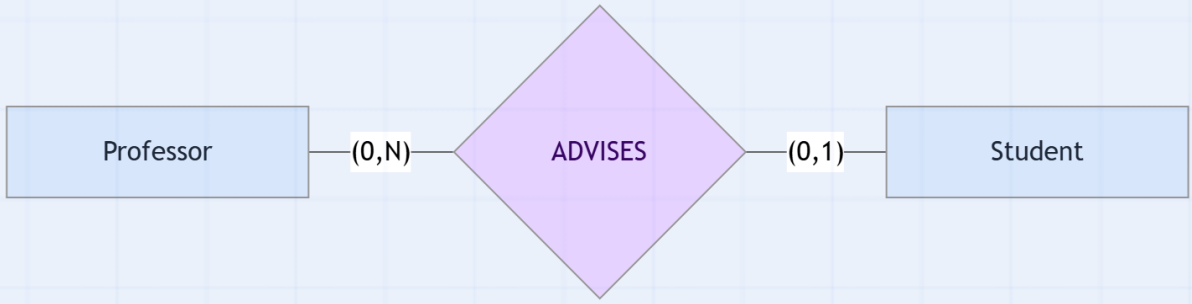


Diagram 36: Professor ADVISES Student (Min-Max)

How to read — Min-Max:

- A professor may advise **any number** of students — or none at all.
- A student may have **at most one** advisor — but is not required to have one.

> **Key insight**
>
> Chen indicated a 1:N ratio with optional participation, but gave no upper bound on the N side. Min-Max confirms there truly is no cap on how many students a professor can advise, while also confirming a student cannot have more than one.

The phrase *"Each advisor-student relationship is formalized through the advising system"* is a strong hint that **ADVISES is not a simple binary connection** — it carries descriptive information of its own. In ER modelling, when a relationship has data associated with it (not belonging to either entity alone), that data is modelled as **relationship attributes**.

Attribute	Type	Meaning
assignment_date	DATE	When the advising relationship was established
status	ENUM(ACTIVE, INACTIVE, PENDING)	Current state of the advisory relationship
end_date	DATE (nullable)	When the relationship was formally terminated
notes	TEXT	Free-text notes entered by advisor or registrar

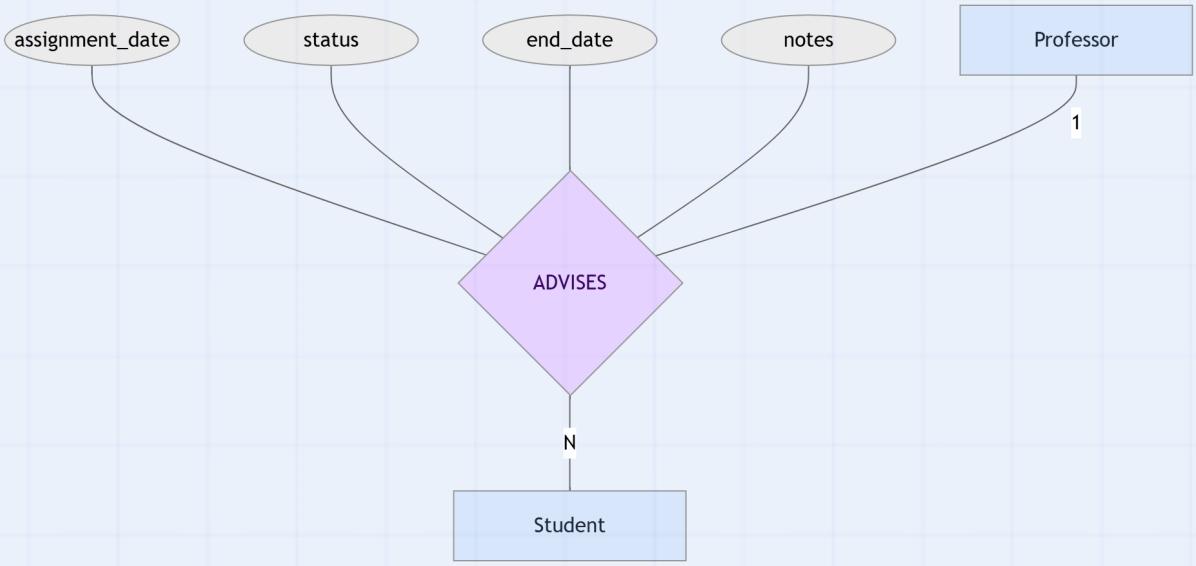


Diagram 37: ADVISES relationship with attributes (assignment_date, status, end_date).

5.5 Specific Value Constraints: (0 , 3) (0,3), (2 , 5) (2,5), and Beyond

> **Not every business rule fits neatly into the four standard patterns**
> **(0,1), (1,1), (0,N), (1,N)**
>
> Min-Max notation allows **any non-negative integers** as the min and max values.

Example	English meaning	Typical business rule
(0,3)	zero to three	A customer may place at most 3 active orders at a time
(2,5)	two to five	A project team must have between 2 and 5 members
(1,4)	one to four	An employee manages at least 1, at most 4 direct reports
(3,3)	exactly three	A committee seat is always filled by exactly 3 co-chairs

These constraints are **perfectly valid** at the conceptual level — they are precise expressions of real business rules that the common patterns simply cannot capture. Write them on your ER diagram exactly as you would any other (min,max) pair.

> ⚠ **Conceptual validity ≠ automatic SQL enforcement**
>
> Standard SQL foreign-key constraints only know how to enforce "must exist" (referential integrity). They have no mechanism to *count* how many rows a related entity participates in before accepting or rejecting a change. Specific value constraints such as (0,3) or (2,5) therefore **cannot be enforced by foreign keys alone**.
>
> Enforcing specific value constraints in the database requires either subquery-`CHECK`s (PostgreSQL), triggers, stored procedures, or application logic.

6 Enhanced Entity-Relationship (EER) Modeling

6.1 The Scenario: Hospital Staff Management (AAUP Medical Center)

At the **Arab American University Medical Center (AAUP Center)** in Jenin, database designers need to model various employee types (`DOCTOR` ,

`RESEARCHER` , `NURSE` , `SECRETARY` , `TECHNICIAN` , `ENGINEER`).

All staff members share foundational attributes: `SSN` , `Name` , `Address` , and `Salary` . However, specific roles require unique local attributes (e.g., `Specialty` for doctors, `EquipmentCert` for technicians). Furthermore, employees are categorized by payment method (`SALARIED_EMPLOYEE` vs. `HOURLY_EMPLOYEE`) and leadership roles (`MANAGER`).

This domain presents complex real-world business constraints:

- **Strict Disjointness:** An employee must be **either** Salaried **or** Hourly.
- **Overlapping Roles:** A single individual can hold multiple roles simultaneously. For example, **Dr. Tariq Zaid** serves as an attending physician in Cardiology (`DOCTOR`) while also directing cardiovascular research trials (`RESEARCHER`), or acting as the department head (`MANAGER`).

6.1.1 Case 1: Separate Entities (Duplicated Attributes)

In a standard ER model without superclass/subclass features, each functional role is modeled as a completely independent entity table (`DOCTOR` , `RESEARCHER` , `TECHNICIAN` , etc.). Base attributes (`SSN` , `Name` , `Address` , `Salary`) are repeated inside every single table.

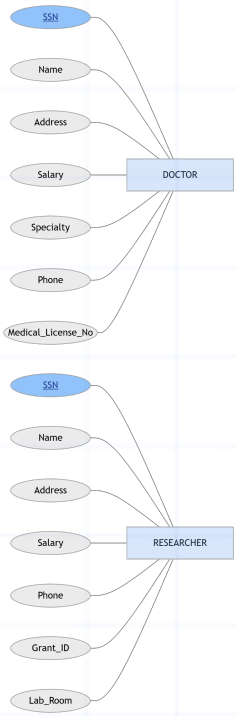


Diagram 38: Standard ER Diagram for Case 1: Independent Subclass Entities (Duplicated Base Attributes)

Issues & Vulnerabilities in Case 1:

- **Update Anomalies:** If Dr. Tariq moves from Jenin to Nablus, an `UPDATE` statement executed on the `DOCTOR` table leaves his record in the `RESEARCHER` table out of date, resulting in an inconsistent database state.
- **Complex Global Queries:** Retrieving a roster of all hospital personnel requires executing a costly `UNION` across 5+ tables (`SELECT ... FROM DOCTOR UNION SELECT ... FROM RESEARCHER ...`).
- **High Maintenance Overhead:** Modifying a shared property (e.g., increasing the character limit of `Address`) requires running schema migrations (`ALTER TABLE`) on every role table individually.
- **Relationship Clutter:** Connecting employees to another entity (e.g., `DEPARTMENT`) requires drawing separate, redundant relationship links for every entity type (`DOCTOR_WORKS_IN` , `NURSE_WORKS_IN` , etc.).

6.1.2 Case 2: Single Table with All Attributes

To avoid separate tables, another standard ER approach collapses every role into a single, massive `EMPLOYEE` table containing all shared attributes along with every role-specific attribute (`Specialty` , `EquipmentCert` , `TypingSpeed` , `Grant_ID` , `PayScale`).

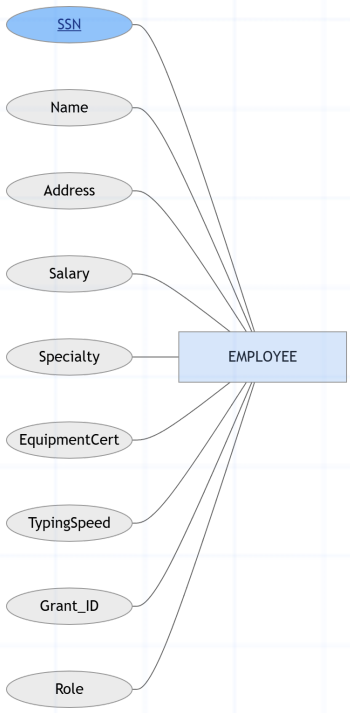


Diagram 39: Standard ER Diagram for Case 2: Single Table with All Attributes

SSN (PK)	Name	Address	Salary	Specialty	EquipmentCert	TypingSpeed	Grant_ID	Role
401928	Tariq Zaid	Jenin	2500	Cardiology	NULL	NULL	AAUP-202	Doc/Res
302194	Ahmad Omar	Nablus	1200	NULL	MRI Specialist	NULL	NULL	Tech

Issues & Vulnerabilities in Case 2:

- **Massive `NULL` Value Overhead:** Because most employees fill only one or two specific roles, most role-specific columns contain `NULL` values, wasting storage and diluting data density.
- **Inability to Enforce Mandatory Attributes:** If a doctor *must* have a `Specialty` , you cannot define `Specialty` as `NOT NULL` at the database level because non-doctor rows (nurses, secretaries) would fail validation.
- **Loss of Semantic Clarity:** Combining distinct conceptual entities into one flat table hides business logic and forces reliance on artificial type-discriminator flags (e.g., `Job_Type` strings).

6.1.3 The Need for Enhanced ER (EER) Modeling

Standard ER modeling lacks the constructs needed to represent complex organizational domains efficiently. Enhanced ER introduces specialized concepts that solve the defects of both Case 1 and Case 2 by incorporating semantic and object-oriented concepts like **inheritance, specialization, and generalization**.

EER Addresses Complex Domains:

1. **Attribute Inheritance:** Subtype entities automatically inherit all attributes and relationships of their supertype entity, eliminating redundancy.
2. **Specialization (Top-Down):** The process of maximizing differences by identifying lower-level subgroups from a higher-level entity.
3. **Generalization (Bottom-Up):** The process of minimizing differences by identifying a higher-level generalized entity from multiple lower-level subgroups.

6.2 Subclasses and Superclasses in EER Diagrams

In Enhanced Entity-Relationship (EER) modeling, **superclass/subclass relationships** allow you to represent specialization and generalization, similar to inheritance in object-oriented programming.

📖 **DEFINITION 20:** **Superclass**

An entity type that includes distinct subgroupings of its entities that require explicit representation (e.g., `EMPLOYEE`).

📖 **DEFINITION 21:** **Subclass**

A subgrouping of entities from a superclass that shares common attributes and relationships with the superclass, but may have additional specific attributes or relationships of its own (e.g., `SECRETARY`, `ENGINEER`, `MANAGER`).

📝 **EXAMPLE 22:**

`EMPLOYEE` may be further grouped into:

- `SECRETARY`, `ENGINEER`, `TECHNICIAN`, ... : Based on the **EMPLOYEE's Job**.
- `MANAGER`: EMPLOYEEs who are managers.
- `SALARIED_EMPLOYEE`, `HOURLY_EMPLOYEE`: Based on the **EMPLOYEE's method of pay**.

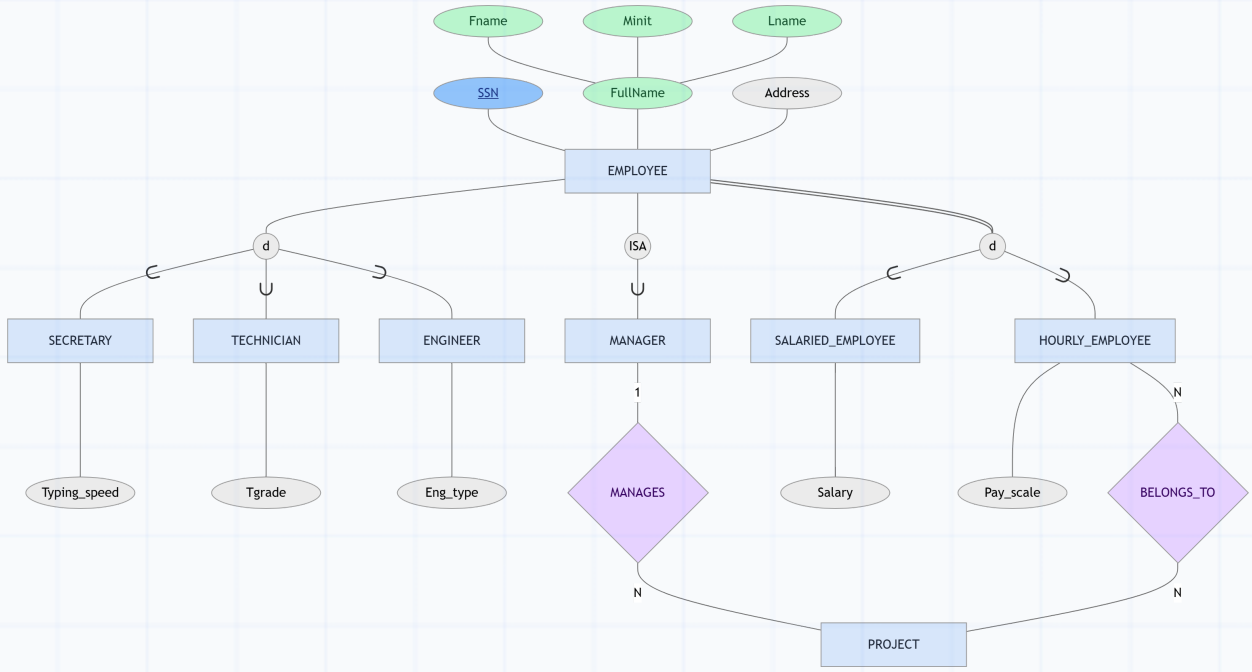


Diagram 40: EER diagram notation illustrating superclass/subclass specialization constraints (disjointness, overlapping, and single-subclass IS-A relationships) for an EMPLOYEE entity.

Note

- EMPLOYEE is the superclass for the three specializations: SECRETARY, ENGINEER, TECHNICIAN}, {MANAGER}, SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.
- Each of these subgroupings is a subset of EMPLOYEE entities called a subclass of EMPLOYEE.
- These are called IS-A relationships: SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE, ...
- An entity cannot exist in the database merely by being a member of a subclass; it must also be a member of the superclass.
- A member of the superclass can be optionally included as a ember of any number of its subclasses.
- A salaried employee who is also an engineer belongs to the two subclasses: ENGINEER, and SALARIED_EMPLOYEE.
- A salaried employee who is also an engineering manager belongs to the three subclasses: MANAGER, ENGINEER, and ALARIED_EMPLOYEE.
- It is not necessary that every entity in a superclass be a member of some subclass.
- An entity that is member of a subclass inherits All attributes of the entity as a member of the superclass and All relationships of the entity as a member of the superclass.

6.3 Generalization

DEFINITION 22: **Generalization**

Generalization is a bottom-up design process that serves as the inverse of specialization. It combines multiple lower-level entity types that share common characteristics into a single higher-level superclass.

EXAMPLE 23: CAR, TRUCK generalized into VEHICLE

Designing independently, you create `CAR` and `TRUCK` all with `LicensePlate`, `Make`, `Model`, `Year`. Noticing the overlap, you generalize them into `VEHICLE`, extracting the shared attributes:

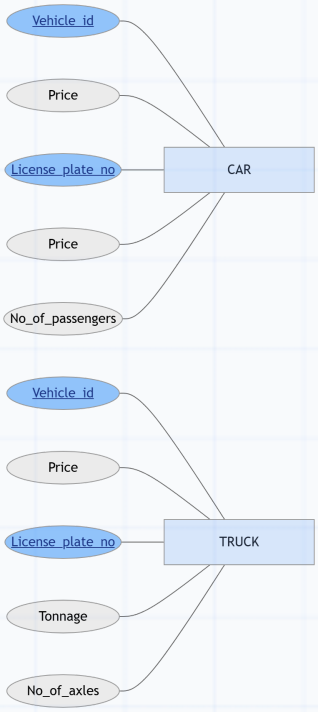


Diagram 41: CAR and TRUCK

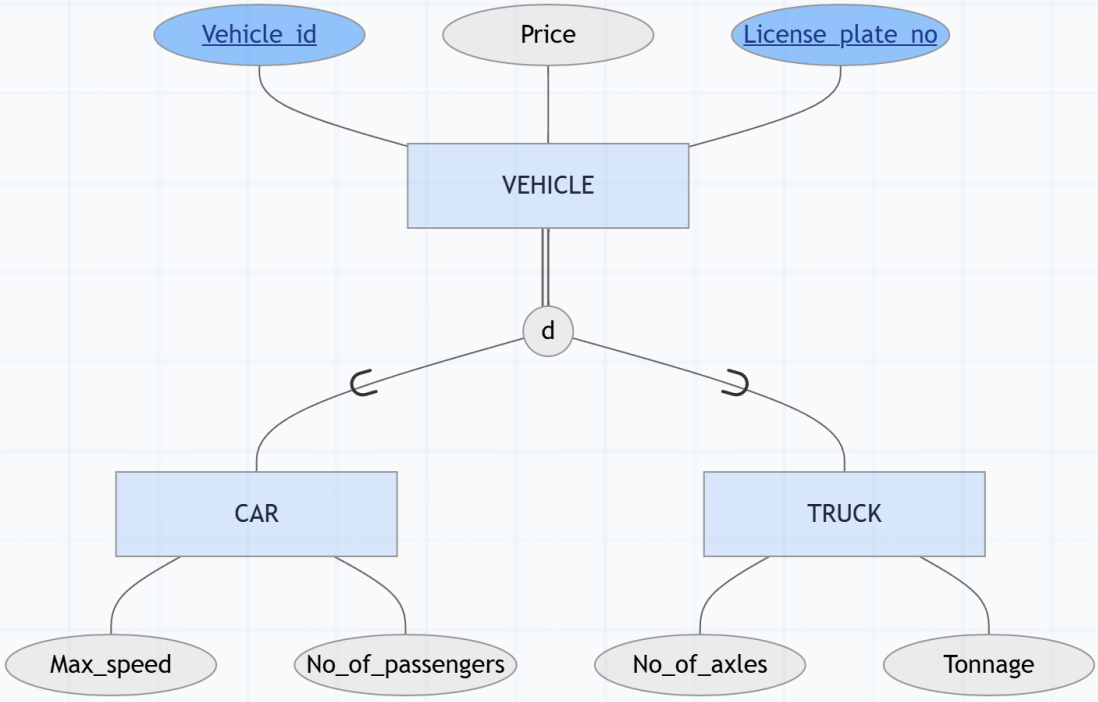


Diagram 42: Generalizing CAR and TRUCK into the superclass VEHICLE

Specialization vs. Generalization

Specialization vs. Generalization produce identical EER diagrams. The distinction is purely in the *decision process* (top-down design vs. bottom-up abstraction). In practice, real designs mix both: you may generalize two existing entities and then specialize the resulting superclass further.

6.4 Specialization

📖 DEFINITION 23: **Specialization**

Specialization is a top-down design process in Enhanced Entity-Relationship (EER) modeling where a higher-level entity type (superclass) is divided into lower-level entity types (subclasses) based on distinguishing characteristics or job roles.

📝 EXAMPLE 24: **University System — PERSON**

Superclass: `PERSON`

- *Common attributes:* `Person_ID`, `Name`, `Email`, `Phone_Number`

Subclasses (Specializations):

- `STUDENT`: Has specific attributes like `GPA`, `Major`, `Credits_Completed`.
- `PROFESSOR`: Has specific attributes like `Office_Hours`, `Department`, `Tenure_Status`, `Salary`.

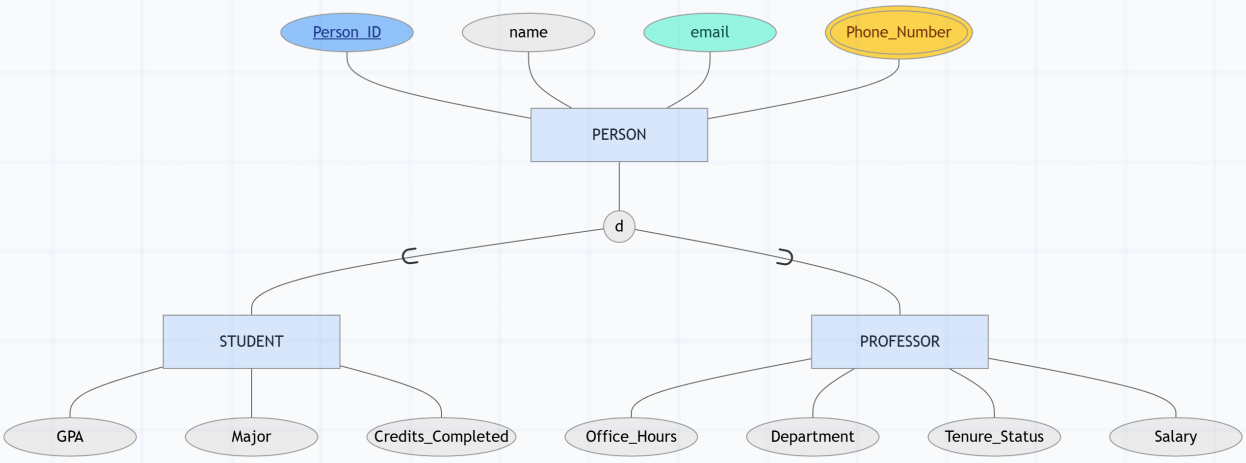


Diagram 43: University System — PERSON (Specializations)

Why specialize?

A standard `PERSON` does not have a `GPA` or `Tenure_Status`. Adding these specific fields to a single table would result in a lot of empty (NULL) values for people who aren't students or professors.

6.5 Constraints on Specialization

A specialization (or generalization) hierarchy is governed by two independent sets of constraints that dictate how instances flow from the superclass into the subclasses.

6.5.1 Disjointness Constraint

The disjointness constraint determines whether an entity instance can belong to **more than one** subclass simultaneously within the same specialization set.

- **Disjoint (d):** Each entity instance can belong to **at most one** of the subclasses. The subclasses are mutually exclusive.
  - **Example:** In an EMPLOYEE specialization, an employee can be either an `HOURLY_EMPLOYEE` or a `SALARIED_EMPLOYEE` —never both at the same time.
- **Overlapping (o):** An entity instance **may belong to multiple subclasses** simultaneously.
  - **Example:** At a university, a `PERSON` can be both a `STUDENT` and an `EMPLOYEE` (such as a graduate teaching assistant) at the same time.

📝 EXAMPLE 25: **The Faculty Ph.D. Candidate**

Imagine a university database for **Arab American University** that models people on campus.

1. **The Core Entities:**
   - `PERSON` (Superclass): Stores general demographic details like `Person_ID`, `First_Name`, `Last_Name`, and `Email`.
   - `INSTRUCTOR` (Subclass): Stores faculty-specific attributes like `Office_Number`, `Rank`, and `Salary`.
   - `STUDENT` (Subclass): Stores student-specific attributes like `Student_ID`, `Major`, and `Cumulative_GPA`.
2. **The Real-World Situation:**
   - Meet **Dr. Samer**, who is a full-time faculty member in the Computer Systems Engineering department teaching undergraduate courses.
   - At the same time, Dr. Samer is enrolled as a part-time doctoral researcher in a Ph.D. program, meaning he takes advanced coursework and registers for dissertation credits.

✅ Show Solution

Why Do We Need the Overlapping (o) Constraint Here?

If the specialization of `PERSON` into `INSTRUCTOR` and `STUDENT` were defined as **Disjoint (d)** (mutually exclusive):

- The database schema would enforce a strict rule: **An entity instance can belong to *at most one* subclass.**
- When Dr. Samer is added to the system, the database forces a choice: Is he *only* an `INSTRUCTOR` or *only* a `STUDENT`?
- **The Problem:** If you classify him strictly as an `INSTRUCTOR`, the system drops his `Student_ID`, `GPA`, and course enrollment records. If you classify him strictly as a `STUDENT`, the system strips away his `Office_Number`, teaching

assignments, and faculty salary profile. **You lose critical real-world data.**

By setting the constraint to **Overlapping (*o*)**, the EER model allows Dr. Samer's single `PERSON` instance to simultaneously link to **both** subclass tables (`INSTRUCTOR` and `STUDENT`):

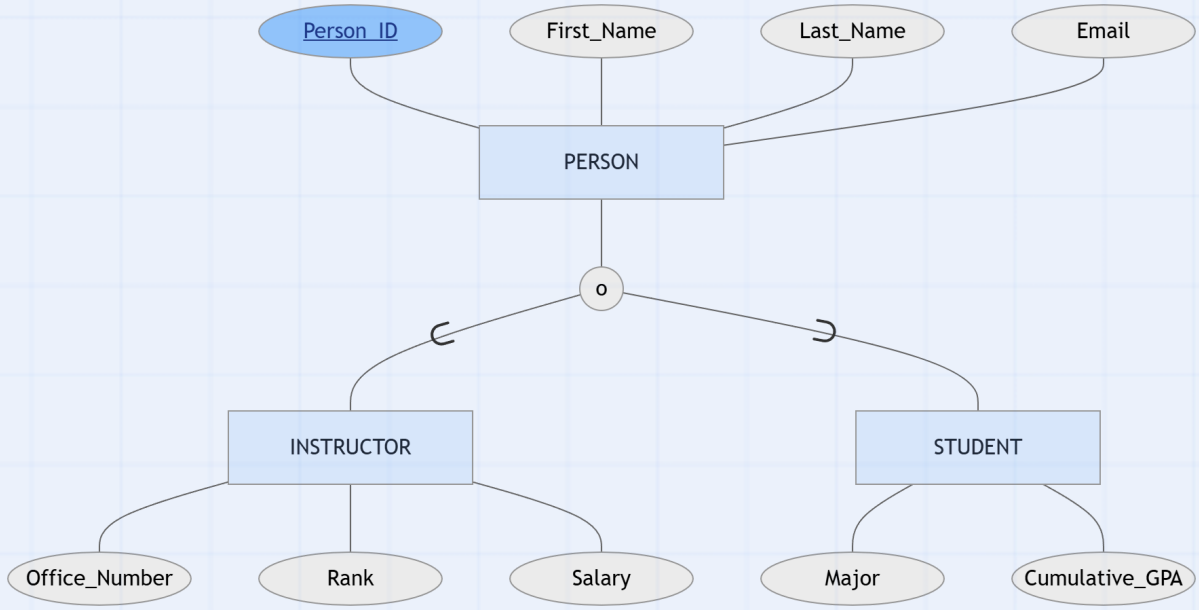


Diagram 44: The Faculty Ph.D. Candidate

6.5.2 Completeness Constraint

The completeness constraint determines whether every instance in the superclass *must* participate in at least one subclass, or if it can exist purely as a superclass instance.

- **Total:** Represented visually by a **double line** connecting the superclass to the circle. Every instance in the superclass **must** belong to at least one subclass.
  - **Example:** Every `SHAPE` must be classified as either a `CIRCLE` , `RECTANGLE` , or `TRIANGLE` —there are no generic shapes stored in the system.
- **Partial:** Represented visually by a **single line**. An instance in the superclass **may** belong to a subclass, but it is not required to.
  - **Example:** Not every `EMPLOYEE` is a `MANAGER` ; most employees exist simply as general employees without any subclass membership.

7 Solved Exercises

📝 EXERCISE 1:

Design a comprehensive Entity-Relationship (ER) Diagram for an academic task-tracking system based strictly on the following business rules and operational constraints:

1. **Teachers:** The system tracks **Teachers**. Each teacher is uniquely identified by a `Teacher_ID` and has a `Name` and a unique `Email` address.

2. **Tasks:** Teachers create **Tasks**. However, task numbering is not unique; multiple teachers can independently create their own "Task 1", "Task 2", etc. Therefore, a task cannot be identified without knowing which teacher created it. The system tracks a sequential `Task_Number` (which resets per teacher) and a `Title`. Every task in the system must belong to exactly one teacher.
3. **Groups:** Teachers form and manage student **Groups**. Group numbers are localized to the teacher, meaning Teacher 1 can have a "Group 1" and Teacher 2 can also have a completely separate "Group 1". A group requires a `Group_No` and a custom `Group_Name`. A group cannot exist without its managing teacher.
4. **Students: Students** are independent strong entities tracked globally in the system by a unique `Student_ID`, a `Name`, and an `Email`.
5. **Team Memberships:** A student can belong to **more than one group** simultaneously. Similarly, each group can contain **multiple students**.
6. **Task Submissions & Grading:** Groups collaborate to complete assignments. A group can solve **multiple tasks**, and a single task is distributed to be solved by **multiple groups**.
7. **Operational Constraints:**
   - When a group completes a task, a shared **Mark** is issued for that specific group's submission.
   - **Every task created must be distributed to at least one group** to be solved (Total/Mandatory participation for tasks).

✅ Show Solution

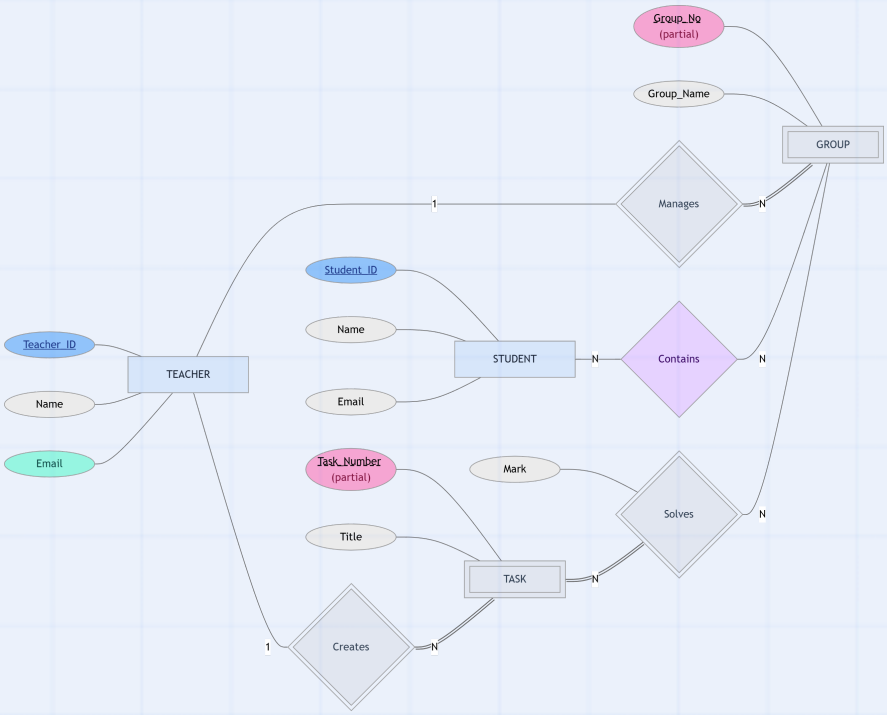


Diagram 45: academic task-tracking system

Chapter3 The Relational Data Model

1 A Brief History — Why the Relational Model?

The relational model was introduced in **1970** by the English computer scientist **Edgar F. Codd** in his landmark paper *"A Relational Model of Data for Large Shared Data Banks"*. To appreciate the model's design choices, it helps to understand the problems it was solving.

Before the relational model, databases were based on **hierarchical** (tree-shaped) and **network** (graph-shaped) models. These models forced application programmers to:

- **Know the physical layout of data** — any change to file structure or storage required rewriting every application that touched that data. Codd called this *physical data dependence*.
- **Navigate the "Connection Trap"** — to retrieve related data in network models, a programmer had to walk explicit pointer paths. Missing a pointer or traversing in the wrong direction produced wrong results silently.
- **Accept inconsistency** — there was no mathematical foundation to define what "consistent" even meant, so the same fact could be stored contradictorily in different parts of the database.

Codd's 1970 paper targeted three specific goals:

- **Data independence:** Programs should work unchanged even when physical storage changes.
- **Resolving the Connection Trap:** Data relationships should be expressed declaratively (by value), not by physical pointers.
- **Mathematical consistency:** A sound basis — set theory and predicate logic — to define, query, and reason about data.

> **Key Takeaways**
>
> ⭐ The key insight was to represent all data as **tables of values**
>
> ⭐ where relationships between tables are expressed by **matching values** (foreign keys) rather than by pointers. A query language can then declare *what* data is needed, not *how* to navigate to it — this is the foundation that SQL is built on.

1.1 The Connection Trap

To understand the **Connection Trap** in legacy network databases (like the CODASYL model), imagine a database for a company with **Employees**, **Projects**, and **Departments**:

- **Department** pointers link to **Employees** assigned to that department.
- **Employee** pointers link to **Projects** that the employee works on.
- **Project** pointers link back to the **Department** funding the project.

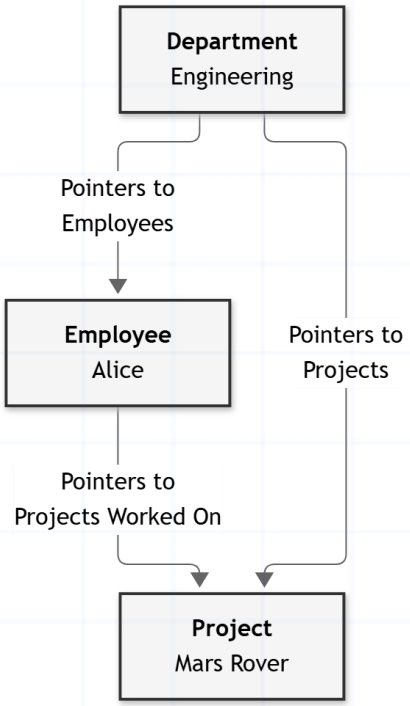


Diagram 46: Network Database Pointer Relationships — Illustrating the explicit pointer paths between Department, Employee, and Project entities that programmers had to manually navigate, creating the risk of the "Connection Trap."

🎬 SCENARIO 1: Missing a Pointer Loop / Navigation Error

suppose a manager asks a programmer:

> *"List all projects funded by the Engineering department along with the employees working on them."*

💡 Show Resolution

Where the "Connection Trap" Silent Errors Happen

Because there was no high-level query language like `SELECT ... JOIN`, the programmer had to write procedural code (in languages like COBOL) to manually move a database cursor through explicit pointer addresses step-by-step:

- The programmer starts at `Department ("Engineering")`.
- They follow the owner-to-member pointer path down to `Employee ("Alice")`.
- They follow Alice's project pointer to find `Project ("Mars Rover")`.
- **The Trap:** The programmer assumes that because they traversed `Engineering → Alice → Mars Rover`, the *Mars Rover* project must be funded by *Engineering*.

However, **Alice might work on a project funded by Marketing**.

By navigating through the *Employee-Project* pointer chain instead of checking the explicit *Department-Project* funding pointer chain, the programmer silently associates the *Mars Rover* project with *Engineering* budget reports.

Why Codd's Relational Model Solved This

Edgar F. Codd recognized that requiring programmers to act as human navigational paths meant business logic was tied directly to physical memory structures.

In a relational database, you eliminate pointer-walking entirely by joining on matching data values instead of memory pointers:

```
SELECT Department.Name, Employee.Name, Project.Name
FROM Department
JOIN Project ON Department.Dept_ID = Project.Funding_Dept_ID
JOIN Employee_Project ON Project.Project_ID = Employee_Project.Project_ID
JOIN Employee ON Employee_Project.Emp_ID = Employee.Emp_ID;
```

Here, the DBMS engine handles the physical navigation underneath, ensuring that missing or misdirected pointer paths are structurally impossible.

1.2 Physical Data Dependence

To understand **Physical Data Dependence**, imagine a company in the 1960s using a legacy mainframe file system or early hierarchical database (like IBM's IMS) to store employee records.

Imagine an enterprise payroll application where each employee record is stored on a magnetic tape or fixed-block disk as a raw, contiguous array of bytes (similar to a C `struct` or COBOL record definition).

Table 11: Original File Layout: Every employee record in the data file is laid out with fixed byte offsets

Bytes	Attribute	Example Data
00–08	Emp_ID	100249581
09–28	Name	John Doe
29–48	Address	123 Main Street
49–56	Salary	00075000

Total Record Length: 57 bytes.

The COBOL application reading this data reads the file directly by byte offset:

```
READ EMPLOYEE-FILE.
MOVE RECORD(09:20) TO EMPLOYEE-NAME.
MOVE RECORD(49:8) TO EMPLOYEE-SALARY.
```

SCENARIO 2: **The Business Change Request**

The company expands nationally and needs to add a **Phone** field right after the `Address` field. **New Record Length:** 62 bytes.

Table 12: *New File Layout*

Bytes	Attribute	Change
00–08	Emp_ID	Unchanged
09–28	Name	Unchanged
29–48	Address	Unchanged
49–53	Phone	NEW FIELD ADDED
54–61	Salary	Shifted right by 5 bytes

💡 Show Resolution

The Pain Point: What Breaks?

Because the applications had **physical data dependence**:

1. **Broken Offsets:** Every program that reads salary now reads bytes 49–56 (which now contains part of the Phone and partial salary digits), producing corrupted data or crash errors.
2. **Cascading Code Changes:** It wasn't just the payroll program that needed updating. **Every application that ever touched the file**—such as the performance review system, tax reporting software, and benefits portal—had to be modified, recompiled, and re-tested, even if those programs didn't use the Phone field at all.
3. **Storage Hardware Upgrades:** Changing from magnetic tapes to disks, or changing disk block sizes, required modifying the low-level record-seeking routines written directly into application code.

> **How Codd's Relational Model Solved It: Physical Data Independence**
>
> Edgar F. Codd introduced an abstraction layer that decoupled application logic from physical disk layout.
>
> **In a relational database:**
>
> 1. You execute `ALTER TABLE Employee ADD COLUMN Phone VARCHAR(5);` .
> 2. Applications query data by attribute name, not by physical byte address.
> 3. The DBMS handles where bytes live on disk behind the scenes. If a new column is added, or if the database administrator changes indexes or moves the file to a different storage disk, the existing `SELECT` queries continue to work **without a single line of application code needing to be changed**.

2 Where Do Tables Come From?

Before we dive into the mathematical definition of tables, domains, and keys, we must answer a fundamental question: **Where do the tables come from?**

We don't just invent tables randomly. We design them based on how we perceive the real world. This process starts with two core concepts: **Entities** and **Relationships**.

Database design is usually a three-step process:

- **Conceptual Design (The "What"):** We create an abstract model of the real world. We identify the things that matter (entities) and how they are connected (relationships). This is often done using an Entity-Relationship (ER) diagram. This model is independent of any specific database software.
- **Logical Design (The "How"):** We take our conceptual model and map it to a specific data model, such as the Relational Model. We define tables (relations), columns (attributes), keys, and constraints. This is the main subject of this chapter.
- **Physical Design (The "Where"):** We implement our logical design on a specific Database Management System (DBMS) like MySQL, PostgreSQL, or Oracle, deciding on file organizations and indexes.

3 Mathematical Backgound

3.1 Set theory

> 📖 DEFINITION 1: **Set theory**
>
> is a fundamental branch of mathematics that studies **sets**, which are well-defined collections of distinct objects called *elements* or *members*.

> **Tip**
>
> In computer science and database theory, set theory provides the mathematical foundation for how data is structured and queried.

Key Concepts in Set Theory:

- 1. Elements and Membership: An object x is an element of a set A, written as $x \in A$. If x is not in A, it is written as $x \notin A$.
  - Example: $A = \{1, 2, 3, 4\}$. Here, $3 \in A$, but $5 \notin A$.
- 2. Core Set Operations
  - **Union ($A \cup B$):** Combines all unique elements from both sets.
    - $\{1, 2\} \cup \{2, 3\} = \{1, 2, 3\}$
  - **Intersection ($A \cap B$):** Finds only elements shared by both sets.

    - $\{1, 2\} \cap \{2, 3\} = \{2\}$
  - **Difference ($A \setminus B$ or $A - B$):** Finds elements in set A that are not in set B.
    - $\{1, 2, 3\} - \{2, 3\} = \{1\}$
  - **Cartesian Product ($A \times B$):** The set of all ordered pairs (a, b) where $a \in A$ and $b \in B$.
    - $\{1, 2\} \times \{X, Y\} = \{(1, X), (1, Y), (2, X), (2, Y)\}$
- 3. Essential Properties
  - **Uniqueness:** A set contains no duplicate elements (e.g., $\{1, 1, 2\}$ is identical to $\{1, 2\}$).
  - **Unordered:** The sequence of elements does not matter (e.g., $\{A, B\} = \{B, A\}$).

3.2 First-Order Predicate Logic (FOPL)

> 📖 DEFINITION 2: **First-Order Predicate Logic (FOPL)**
>
> is a formal mathematical language used to express statements, reason logically, and determine truth values based on rules.

> **Tip**
>
> While **propositional logic** can only deal with simple declarative statements (e.g., "It is raining" = True/False), **first-order logic** breaks statements down into objects, properties, relationships, and quantifiers.

Key Components of First-Order Logic:

- **Objects (Constants):** Specific entities in the domain of discourse (e.g., `John`, `CS101`, `5`).
- **Variables:** Symbols representing unspecified objects (e.g., x, y, z).
- **Predicates:** Functions that express properties of objects or relationships between them and evaluate to True or False:
  - Property: $\text{IsStudent}(x)$
  - Relationship: $\text{EnrolledIn}(x, y)$
- **Functions:** Expressions that map objects to another object (e.g., $\text{FatherOf}(x)$).
- **Logical Connectives:** Standard operators to combine expressions:
  - AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$), IF AND ONLY IF ($\leftrightarrow$).
- **Quantifiers:**
  - **Universal Quantifier ($\forall$):** "For all" or "For every".
  - **Existential Quantifier ($\exists$):** "There exists at least one".

📝 EXAMPLE 1:

- **Natural English:** "Every student enrolled in CS101 has passed a prerequisite."
- **First-Order Logic:**

$$\forall x \left(\text{Student}(x) \land \text{EnrolledIn}(x, \text{CS101}) \rightarrow \exists y \left(\text{Prerequisite}(y) \land \text{Passed}(x, y)\right)\right)$$

4 The Relational Model

Proposed by E. F. Codd in 1970, the relational model provides a formal framework for database systems grounded in **set theory** and **first-order predicate logic**. Data is represented declaratively as collections of relations (tables) rather than physical pointers.

4.1 Core Concepts & Structural Architecture

Before diving into formal notation, it is essential to establish the structural hierarchy of a relational database, moving from global definitions down to the stored data state.

📖 DEFINITION 3: **Database Schema (S)**

Database Schema (S): Defines the **entire database structure** as a set containing all relation schemas $\{R_1, R_2, \ldots, R_m\}$ along with the set of integrity constraints (IC) that govern them (such as Primary Keys, Foreign Keys, and Domain Constraints).

$$S = (\{R_1, R_2, \ldots, R_m\}, IC)$$

📖 DEFINITION 4: **Relation Schema (R)**

Relation Schema (R): The static logical definition of a single table, specifying its name, attributes, and domains:

$$R(A_1 : D_1, A_2 : D_2, \ldots, A_n : D_n)$$

📖 DEFINITION 5: **Relation Instance**

Relation Instance ($r(R)$): The dynamic snapshot of actual data stored in a relation at a given point in time:

$$r(R) = \{t_1, t_2, \ldots, t_m\} \quad \text{where each } t_i \in D_1 \times D_2 \times \cdots \times D_n$$

4.2 Mathematical Definition

A **relation** R of degree (arity) n is defined mathematically as a subset of the Cartesian product of n domains:

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n$$

Where:

- **Domain (D_i):** A set of atomic (indivisible) values of the same type.
  - $D_{\text{Salary}} = \{x \in \mathbb{R} \mid x \geq 0\}$
  - $D_{\text{Gender}} = \{\text{'M'}, \text{'F'}\}$
- **Degree (n):** The number of attributes (columns) defining the relation schema.
- **Cardinality ($|R|$):** The total number of tuples (rows) present in the relation instance.

4.3 Formal Tuple Notation

Each element of a relation R is an n-tuple t:

$$t = (v_1, v_2, \ldots, v_n) \quad \text{where } v_i \in D_i$$

Referencing Tuple Components

- **Index-Based Notation:**

$$t[i] = v_i$$

- **Attribute-Name Notation:**

$$t[A_i] \quad \text{or} \quad t.A_i$$

- **Sub-tuple (Projection) Notation:**

  For a subset of attributes $X \subseteq \{A_1, A_2, \ldots, A_n\}$:

$$t[X]$$

📝 EXAMPLE 2: **Concrete Example Table: EMPLOYEE (الموظفون)**

Table 13: *Relation EMPLOYEE (الموظفون)*

Emp_ID(PK)	Fname	Lname	Salary	Dept_Code(FK)
101	Ahmed	Al-Mansour	75000.00	ENG
102	Bilal	Hassan	62000.00	MKT
103	Fatima	Al-Zahra	80000.00	ENG

Formal Schema Notation

$$\text{EMPLOYEE}(\text{Emp_ID}, \text{Fname}, \text{Lname}, \text{Salary}, \text{Dept_Code})$$

Cartesian Product Set Mapping

$$\text{EMPLOYEE} \subseteq D_{\text{Emp_ID}} \times D_{\text{Fname}} \times D_{\text{Lname}} \times D_{\text{Salary}} \times D_{\text{Dept_Code}}$$

For tuple $t_1 = (101, \text{'Ahmed'}, \text{'Al-Mansour'}, 75000.00, \text{'ENG'})$:

$$t_1[\text{Emp_ID}] = 101$$

$$t_1[\text{Fname}] = \text{'Ahmed'}$$

$$t_1[\{\text{Fname}, \text{Lname}\}] = (\text{'Ahmed'}, \text{'Al-Mansour'})$$

Table 14: *Mathematical Mapping Summary*

Relational Term	Formal Mathematical Term	Database Term	Example Value
Relation	Set R	Table	`EMPLOYEE`
Attribute	Variable / Set Dimension A_i	Column / Field	`Fname`, `Salary`
Tuple	Element $t \in R$	Row / Record	$t_1 = (101, \text{'Ahmed'}, \text{'Al-Mansour'}, 75000.00, \text{'ENG'})$
Domain	Set D_i	Data Type / Domain	$D_{\text{Salary}} = \{x \in \mathbb{R} \mid x \geq 0\}$
Degree	Arity (n)	Column Count	**5**
Cardinality	Set Size ($\lvert R \rvert$)	Row Count	**3**

4.4 Properties of Relations

Table 15: *Properties of Relations*

Property	Formal Relation	SQL Table
No duplicate tuples	Required by set theory	Enforced only with PK or UNIQUE
Tuple ordering	Irrelevant	Undefined without ORDER BY
Attribute ordering	Irrelevant	Fixed by column position
Atomic attribute values	Required (**1NF**)	Required in normalized tables
NULL values	Not allowed (formal model)	Allowed in ANSI SQL

Note

SQL tables are **multisets** (bags), not sets. Without a primary key, SQL allows duplicate rows. The formal relational model does not. Always define a primary key.

4.5 Domain Specification Components

> 📖 **DEFINITION 6: Domain**
>
> A domain D is a set of atomic (indivisible) values. Each domain has a logical name, data type, and format.

Table 16: *Domain Specification Components*

Component	What it Specifies	SQL Mechanism	Example
1. Data Type	The kind of values (integer, text, date ...)	`INT`, `VARCHAR`, `DATE`, `DECIMAL`, `BOOLEAN`	`Salary DECIMAL(10,2)`
2. Length / Precision	Maximum characters, total digits, decimal places	`VARCHAR(n)`, `CHAR(n)`, `DECIMAL(p,s)`	`Name VARCHAR(50)`, `GPA DECIMAL(3,2)`
3. Date Format	How date/time values are interpreted	`DATE` (ISO 8601: `YYYY-MM-DD` internally)	`Bdate DATE` stores `1995-08-20`
4. Range	Lower and upper bounds on numeric or date values	`CHECK(col BETWEEN a AND b)`	`CHECK(GPA BETWEEN 0.0 AND 4.0)`
5. Constraints	Rules that restrict allowed values beyond type	`NOT NULL`, `UNIQUE`, `CHECK(expr)`, `PRIMARY KEY`, `FOREIGN KEY`	`CHECK(Gender IN ('M','F'))`
6. NULL Support	Whether the attribute may be absent	`NULL` (default) or `NOT NULL`	`Middle_Name VARCHAR(30)` — nullable
7. Default Value	Value used when none is supplied on INSERT	`DEFAULT expr`	`Status VARCHAR(10) DEFAULT 'Active'`

4.5.1 Data Type

Databases provide support for different types of data and their variants:

- INTEGER:- Whole numbers (e.g., 1, 42, -17)
- FLOAT/ DECIMAL - Real numbers with decimal points.
- VARCHAR/ TEXT - Character strings.
- DATE- Calendar dates.
- BOOLEAN- True/False values.
- ENUM- Enumerated list of values

4.5.2 Length / Precision

Length means the number of characters or digits that an attribute value has.

- For VARCHAR(50): Maximum 50 characters.
- For DECIMAL(10,2): Total 10 digits, 2 after decimal point.
- For CHAR(6): Exactly 6 characters (e.g., PIN code).

4.5.3 Date Format

A date contains day, month, and year in combination:

- DD/MM/YYYY - 07/02/2026 (European format)
- MM/DD/YYYY - 02/07/2026 (American format)
- YYYY-MM-DD - 2026-02-07 (ISO 8601 standard)

4.5.4 Range

A range is specified by lower and upper bounds of data values:

- GPA: [0.0, 4.0]
- Age: [0, 150]
- Percentage: [0, 100]
- Temperature: [-273.15, ∞) (Celsius, above absolute zero)

4.5.5 Constraints

Particular conditions that put restrictions on allowed values:

- NOT NULL - Value must be provided
- UNIQUE - No duplicates allowed
- CHECK - Custom validation rule
- PRIMARY KEY - Uniquely identifies each row
- FOREIGN KEY - References another table

4.5.6 NULL Support

NULL represents *unknown*, *missing*, or *not applicable*. Different NULLs express different real-world meanings:

Table 17: *NULL Values meanings*

Reason for NULL	Example
Unknown	`Bdate` of an employee not on record
Not applicable	`Super_ssn` for the CEO (no supervisor)
Not yet known	`Mgr_start_date` during a transition period

4.5.7 Default Value

If nothing is entered, database assigns a default value:

- Status: DEFAULT 'Active'
- Created_Date: DEFAULT CURRENT_DATE
- Country: DEFAULT 'USA'
- Is_Active: DEFAULT TRUE

4.6 Keys and Integrity Constraints

While the mathematical definition of a relation (Section 5.2) tells us how data is structured, it does not tell us how to ensure the data is **accurate, unique, and meaningfully connected**. To enforce business rules and prevent data anomalies, the relational model relies on **Keys** (to identify records) and **Integrity Constraints** (to enforce rules).

4.6.1 Keys: Identifying Tuples

In the real world, we identify entities by unique characteristics (e.g., a person by their National ID, a book by its ISBN). In the relational model, we use **Keys** to uniquely identify tuples (rows) within a relation.

> ⓘ **Note**
>
> **Keys** are used to uniquely identify tuples (rows) within a relation.

4.6.1.1 Super Key

> 📖 DEFINITION 7: **Super Key**
>
> A set of one or more attributes (columns) that uniquely identifies a row (tuple) within a table. Any combination of columns that prevents duplicate rows is a super key.

Mathematical Definition

Let RR be a relation schema. A set of attributes SK⊆R is a super key if, for any valid state r(R), no two distinct tuples t_1 and t_2 can have the same values for SKSK.

- *Equation:*

$$\forall t_1, t_2 \in r(R), \quad (t_1 \neq t_2) \implies t_1[SK] \neq t_2[SK]$$

where:

- $r(R)$: A valid state (instance) of the relation schema R.
- t_1, t_2: Two distinct tuples (rows) within the relation state $r(R)$.
- SK: The set of attributes that form the super key ($SK \subseteq R$).
- $t[SK]$: The projection (values) of tuple t on the set of attributes SK

EXAMPLE 3: Understanding the Super Key Rule

Table 18: Relation State: STUDENT (r(R)

Student_ID	National_ID	Email	Name	Department
001	1029384756	ahmed.k@univ.edu.ps	Ahmed Khalil	Computer Science
002	1029384757	tariq.s@univ.edu.ps	Tariq Mansour	Computer Science
003	1029384758	sara.a@univ.edu.ps	Sara Al-Ahmad	Information Technology
004	1029384759	ahmed.k2@univ.edu.ps	Ahmed Khalil	Software Engineering

A **Super Key (SK)** is any combination of attributes (columns) that uniquely identifies a row in a table. This means if you know the value of SK, no two students in the entire table can share that exact value.

Examples from the Table Above

1. Single-Attribute Super Keys

These columns are naturally unique by themselves:

- $\{\text{Student_ID}\}$: Every student has a unique university ID number.
- $\{\text{National_ID}\}$: Every student has a unique national ID number (رقم الهوية).
- $\{\text{Email}\}$: Every university email address is unique.

2. Composite Super Keys (Supersets)

A Super Key remains a Super Key even if you add extra, unnecessary attributes to a unique key:

- $\{\text{Student_ID}, \text{Name}\}$: Unique because `Student_ID` is already unique.

- $\{\text{National_ID}, \text{Department}\}$: Unique because `National_ID` is already unique.
- $\{\text{Email}, \text{Name}, \text{Department}\}$: Unique because `Email` is already unique.
- $\{\text{Student_ID}, \text{National_ID}, \text{Email}, \text{Name}, \text{Department}\}$: The entire relation schema is also a valid Super Key.

3. NOT Super Keys

These attributes cannot uniquely identify a row:

- $\{\text{Name}\}$: Fails because two different students share the same name ("Ahmed Khalil").
- $\{\text{Department}\}$: Fails because multiple students belong to "Computer Science".
- $\{\text{Name}, \text{Department}\}$: Fails if two students named "Ahmed Khalil" ever enroll in the same department.

4.6.1.2 Candidate Key

📖 DEFINITION 8: **Candidate Key**

A **Candidate Key** is a *minimal* Super Key. It has the exact same unique identification property as a Super Key, but it contains **no redundant attributes**. If you remove any attribute from a Candidate Key, it ceases to be a key.

ⓘ **Mathematical Definition**

A set of attributes $CK \subseteq R$ is a **Candidate Key** for a relation schema R if and only if it satisfies the following two conditions:

1. CK is a super key.
2. Removing any attribute A from CK results in a set $X = CK \setminus \{A\}$ that is no longer capable of uniquely identifying every tuple (it ceases to be a Super Key).

📝 EXAMPLE 4:

Table 19: *Relation State: STUDENT (r(R))*

Student_ID	National_ID	Email	Name	Department
001	1029384756	ahmed.k@univ.edu.ps	Ahmed Khalil	Computer Science
002	1029384757	tariq.s@univ.edu.ps	Tariq Mansour	Computer Science
003	1029384758	sara.a@univ.edu.ps	Sara Al-Ahmad	Information Technology
004	1029384759	ahmed.k2@univ.edu.ps	Ahmed Khalil	Software Engineering

A **Candidate Key (CK)** is a **minimal Super Key**. To be a Candidate Key, a set of attributes must satisfy two rules:

1. **Uniqueness:** It can uniquely identify every row (tuple) in the table.
2. **Minimality (Irreducibility):** If you remove *any* attribute from it, it loses its uniqueness property.

$$\forall A \in CK, \quad CK \setminus \{A\} \quad \text{is no longer a Super Key}$$

Analysis of the Table Attributes

1. Valid Candidate Keys

These single attributes are Candidate Keys because each one uniquely identifies every student, and they cannot be reduced further:

- $CK_1 = \{\text{Student_ID}\}$: Every student has a unique university ID.
- $CK_2 = \{\text{National_ID}\}$: Every student has a unique national ID (رقم الهوية).
- $CK_3 = \{\text{Email}\}$: Every university email address is assigned to exactly one student.

Each of these is a valid candidate to be chosen as the table's **Primary Key**.

2. Super Keys that are NOT Candidate Keys

Consider these combinations:

- $SK_1 = \{\text{Student_ID}, \text{Name}\}$
  - *Is it unique?* Yes, because `Student_ID` is unique.
  - *Is it minimal?* **No.** If you remove `Name`, you are left with $\{\text{Student_ID}\}$, which is still unique. Therefore, `Name` is redundant.
- $SK_2 = \{\text{National_ID}, \text{Email}\}$
  - *Is it unique?* Yes.
  - *Is it minimal?* **No.** Removing `National_ID` leaves $\{\text{Email}\}$, which is already unique on its own.

3. Attributes that are NOT Candidate Keys

- $\{\text{Name}\}$: Fails uniqueness (two students are named "Ahmed Khalil").
- $\{\text{Department}\}$: Fails uniqueness (multiple students belong to "Computer Science").
- $\{\text{Name}, \text{Department}\}$: Fails uniqueness because two students with the same name could potentially enroll in the same department.

4.6.1.3 Primary Key

📖 DEFINITION 9: **Primary Key**

The specific candidate key chosen by the database designer to uniquely identify each row in the table. It must contain unique values and cannot contain `NULL` values.

Mathematical Definition

A set of attributes $PK \subseteq R$ is a **Primary Key** for a relation schema R if and only if for every valid relation state $r(R)$, it satisfies:

1. Uniqueness (Super Key Property)

$$\forall t_1, t_2 \in r(R), \quad (t_1 \neq t_2) \implies t_1[PK] \neq t_2[PK]$$

2. Entity Integrity Constraint (Non-Null Property)

$$\forall t \in r(R), \quad \forall A \in PK, \quad t[A] \neq \text{NULL}$$

(If PK consists of a single attribute $PK = \{A\}$, this simplifies to $\forall t \in r(R),\ t[PK] \neq \text{NULL}$.)

EXAMPLE 5:

Table 20: Relation State: FACULTY_DIRECTORY (r(R))

Staff_ID	National_ID	Name	Email	Department
101	901234567	Dr. Tariq Al-Hassan	tariq.h@univ.edu.ps	Computer Systems Eng.
102	901234568	Eng. Reem Mansour	reem.m@univ.edu.ps	Information Technology
103	901234569	Dr. Omar Khalid	NULL	Computer Systems Eng.
104	901234570	Layla Suleiman	NULL	Computer Science

Why $\underline{\text{Staff_ID}}$ is a Valid Primary Key:

1. **Uniqueness (Super Key Property):** Every employee has a unique `Staff_ID` (`101`, `102`, `103`, `104`). No two distinct rows share the same ID.
2. **Entity Integrity (Non-Null Property):** Every employee is guaranteed to have a `Staff_ID`. No row has a `NULL` value for this attribute.

Because both conditions are strictly satisfied across the relation, $\underline{\text{Staff_ID}}$ safely serves as the **Primary Key**.

Illustrating the Email Case (Why NULL Invalidates a Primary Key)

Even if all non-null email addresses are unique, `Email` fails as a Primary Key due to the following mathematical and logical violations:

1. Violation of the Entity Integrity Constraint

By definition, a Primary Key PK must satisfy:

$$\forall t \in r(R), \quad t[PK] \neq \text{NULL}$$

In this table, for tuple t_3 (Dr. Omar Khalid) and tuple t_4 (Layla Suleiman):

$$t_3[\text{Email}] = \text{NULL}$$

$$t_4[\text{Email}] = \text{NULL}$$

Because $t_3[\text{Email}] = \text{NULL}$, the **Entity Integrity Constraint is broken**.

2. Loss of Row Identification

In relational theory, `NULL` represents an **unknown** or **missing** value. Because `Email` is missing for both t_3 and t_4, the database cannot use `Email` to distinguish Dr. Omar Khalid from Layla Suleiman.

3. Failure of Mathematical Equality

In standard relational calculus and SQL three-valued logic, `NULL = NULL` evaluates to `UNKNOWN` (not `TRUE`). Therefore, testing uniqueness on `NULL` attributes fails the formal condition:

$$\forall t_1, t_2 \in r(R), \quad (t_1 \neq t_2) \implies t_1[PK] \neq t_2[PK]$$

Practical Takeaway

- `Staff_ID` remains the valid **Primary Key** ($\underline{\text{Staff_ID}}$) because it is both unique and strictly non-null ($\neq \text{NULL}$) for every tuple.
- `Email` can only be classified as an optional attribute or a candidate attribute with a sparse/filtered unique index, but **it can never be a Primary Key** whenever `NULL` entries exist.

Extending the Concept: Alternative (Alternate) Keys

In our `FACULTY_DIRECTORY` table, `National_ID` is also a completely valid Candidate Key because it is both **unique** for every single faculty member and **guaranteed to never be NULL** ($\forall t \in r(R),\ t[\text{National_ID}] \neq \text{NULL}$).

However, standard relational theory dictates that a relation schema R can have **exactly one Primary Key** (PK).

Definition: Alternate (or Alternative) Key

Any **Candidate Key** that is **not** chosen by the database designer to be the Primary Key is called an **Alternate Key** (or **Secondary Unique Key**).

Mathematical Relation

$$\text{Alternate Keys} = \text{Candidate Keys} \setminus \{\text{Primary Key}\}$$

Applied Example

From our pool of valid Candidate Keys for the faculty directory:

$$\text{Candidate Keys} = \{\{\text{Staff_ID}\}, \{\text{National_ID}\}\}$$

1. **Primary Key (PK):** We select $\underline{\text{Staff_ID}}$ to serve as the main entity identifier.
2. **Alternate Key (AK):** National_ID becomes the Alternate Key.

4.6.1.4 Foreign Key

📖 DEFINITION 10: **Foreign Key**

An attribute or set of attributes in one table that references the primary key (or candidate key) of another table, establishing a logical link and maintaining referential integrity between them.

Mathematical Definition

Let R_1 and R_2 be two relation schemas (where R_1 and R_2 can be the same relation in a self-reference), and let PK be the **Primary Key** of R_2.

A set of attributes $FK \subseteq R_1$ is a **Foreign Key** in R_1 referencing R_2 if it satisfies two conditions for every valid relational database state $db = \{r(R_1), r(R_2)\}$:

1. Domain Compatibility

$$\text{Dom}(FK) = \text{Dom}(PK)$$

(The attributes in FK have the same domain/data type as the primary key PK of R_2. If FK and PK are composite, there must be a one-to-one mapping between their corresponding attributes).

2. Referential Integrity Constraint

$$\forall t_1 \in r(R_1), \quad (\exists t_2 \in r(R_2) \quad \text{such that} \quad t_1[FK] = t_2[PK]) \quad \vee \quad (t_1[FK] = \text{NULL})$$

Key Properties:

- **Referencing vs. Referenced:** R_1 is called the **referencing relation**, and R_2 is the **referenced relation**.
- **Nullability:** Unlike Primary Keys, Foreign Key attributes **can contain NULL values** (unless explicitly constrained by a `NOT NULL` constraint), representing an unassigned or missing relationship.
- **Self-Referencing Foreign Key:** When $R_1 = R_2$, the foreign key references its own table's primary key (e.g., `Super_ssn` referencing `Ssn` within an `EMPLOYEE` relation).

EXAMPLE 6: University Course Enrollment System to illustrate how a Foreign Key (FK) works

To store information about university departments and their students without repeating data, we split the information into two relations:

1. `DEPARTMENT` (The **Referenced Relation** R_2)
2. `STUDENT` (The **Referencing Relation** R_1)

Table 21: The Referencing Relation: STUDENT

Table 22: The Referenced Relation: DEPARTMENT

Dept_ID (PK)	Dept_Name	Building
10	Computer Systems Engineering	Engineering Building
20	Information Technology	IT Complex
30	Computer Science	Main Campus

Student_ID (PK)	Name	Major	Dept_ID (FK)
2024001	Ahmed Khalil	Hardware Engineering	10
2024002	Tariq Mansour	Network Security	20
2024003	Sara Al-Ahmad	Data Science	30
2024004	Reem Suleiman	Undecided	NULL

For a Foreign Key $FK \subseteq \text{STUDENT}$ referencing DEPARTMENT:

$$\pi_{\text{Dept_ID}}(\text{STUDENT}) \setminus \{\text{NULL}\} \subseteq \pi_{\text{Dept_ID}}(\text{DEPARTMENT})$$

Let's check the values in `STUDENT.Dept_ID`:

$$\{10, 20, 30, \text{NULL}\} \setminus \{\text{NULL}\} = \{10, 20, 30\}$$

Since $\{10, 20, 30\} \subseteq \{10, 20, 30\}$ (the Primary Keys present in `DEPARTMENT`), the referential integrity constraint holds true.

Key Observations from This Example

1. Valid Linkage

Student `2024001` (Ahmed Khalil) has `Dept_ID = 10`. The database looks at `DEPARTMENT` with `Dept_ID = 10` and knows Ahmed belongs to **Computer Systems Engineering**.

2. Nullability (Allowed)

Student `2024004` (Reem Suleiman) has `Dept_ID = NULL`.

- **Why it's valid:** Unlike Primary Keys, Foreign Keys **can be NULL** if a relationship is optional (e.g., a newly enrolled student who hasn't declared a department yet).

3. Referential Integrity Violation (Prevented by the DBMS)

If you try to insert a new student with an invalid department:

```
INSERT INTO STUDENT VALUES ('2024005', 'Omar Khalid', 'AI', 99);
```

- **Result: REJECTED BY DBMS.**
- **Reason:** `Dept_ID = 99` does not exist in `DEPARTMENT`. The foreign key constraint prevents "orphan" references to non-existent departments.

4.6.2 Integrity Constraints (ICs)

> 📖 DEFINITION 11: **Integrity Constraints**
>
> Integrity Constraints are business rules applied to the relational schema to ensure the accuracy and consistency of the data. If a database state violates an integrity constraint, it is considered an **invalid state**.

4.6.2.1 Domain Constraints

These are the most basic constraints. They ensure that every attribute value must belong to its defined domain (data type, format, and range).

- **Rule:** Values must match the specified data type and fall within allowed ranges.
- **Example:** If the domain of `Age` is defined as integers between 18 and 100, inserting a value of `150` or `'Twenty'` violates the domain constraint.

4.6.2.2 Key Constraints & Entity Integrity

This constraint governs the Primary Key. Because the Primary Key's entire purpose is to uniquely identify a row, it must always have a value.

- **Rule (Entity Integrity):** No part of a Primary Key can be `NULL`.
- **Example:** You cannot insert an employee into the `EMPLOYEE` table without an `Emp_ID`. If `Emp_ID` is `NULL`, the database cannot uniquely identify or reference that specific employee.
- *Note:* Uniqueness is also enforced (no two rows can have the same PK), which is inherently guaranteed by the definition of a Primary Key.

4.6.2.3 Referential Integrity

This constraint governs Foreign Keys. It ensures that relationships between tables remain consistent. You cannot have an "orphan" record pointing to a non-existent parent.

- **Rule:** A Foreign Key value must either be completely `NULL` (meaning the relationship doesn't exist yet) OR it must exactly match an existing Primary Key value in the referenced table.
- **Example:** If the `DEPARTMENT` table only has departments `ENG`, `MKT`, and `HR`, you **cannot** insert an employee with `Dept_Code = 'FIN'`. The database will reject the insertion to maintain referential integrity.

4.7 Formal Schema Notation

In database literature and practice, there are two primary ways to express a relational schema:

1. **The Strict Formal Notation** (The Elmasri & Navathe academic standard).
2. **The Shorthand Relational Notation** (The practical, inline design standard).

4.7.1 Style 1 — The Strict Formal Notation (Elmasri & Navathe Style)

This notation is used in pure database theory. It strictly separates the **structural definition** of the tables from the **Integrity Constraints (ICs)**. It focuses on the mathematical *state* of the database, not the procedural *actions* of the DBMS.

The 5 Rules of Strict Formal Notation:

- **Domains are Mandatory:** Every attribute must be followed by a colon and its domain (data type).
  - *Format:* `R(A1:D1, A2:D2, ...)`
- **Primary Keys are Underlined:** The primary key attribute(s) are denoted by a solid underline. If it is a composite key, *every* attribute in the key is underlined.
  - *Format:* `R(` A1 `, A2, ...)`
- **Foreign Keys are NOT Marked Inline:** Foreign keys are written as normal attributes inside the parentheses. Their status as foreign keys is declared **only** in the Integrity Constraint (IC) list below the schema.
- **Integrity Constraints (ICs) are Listed Separately:** All rules regarding Foreign Keys (Referential Integrity), NOT NULL, and CHECK constraints are written sequentially below the relation as `IC1, IC2, IC3 ...`
- **Referential Integrity uses Subset Notation:** Foreign keys are expressed using the subset symbol (⊆).
  - *Format:* `R1.FK_Attribute` ⊆ `R2.PK_Attribute`

> 📝 **EXAMPLE 7: The Core Company Hierarchy (Simple Keys & Basic Constraints)**
>
> This schema models the foundational relationship between a company's departments and its employees. Every department must have a unique number and a non-null name, and is managed by a specific employee. Every employee has a unique SSN, works in one department, and reports to a supervisor. We must enforce a minimum salary threshold and ensure referential integrity between employees and their departments/supervisors.
>
> $$\begin{aligned}&\text{DEPARTMENT}(\underline{\text{Dnumber}} : \text{Integer}, \text{Dname} : \text{String},\\ &\quad \text{Mgr_ssn} : \text{String}, \text{Mgr_start_date} : \text{Date})\\ &\text{IC1: Dname} \neq \text{NULL}\\ &\text{IC2: Mgr_ssn} \subseteq \text{EMPLOYEE.Ssn}\end{aligned}$$
>
> $$\begin{aligned}&\text{EMPLOYEE}(\underline{\text{Ssn}} : \text{String}, \text{Fname} : \text{String},\\ &\quad \text{Lname} : \text{String}, \text{Salary} : \text{DECIMAL}(10,2),\\ &\quad \text{Dno} : \text{Integer}, \text{Super_ssn} : \text{String})\\ &\text{IC3: Fname} \neq \text{NULL}\\ &\text{IC4: Lname} \neq \text{NULL}\\ &\text{IC5: } Salary \geq 30000\\ &\text{IC6: Dno} \subseteq \text{DEPARTMENT.Dnumber}\\ &\text{IC7: Super_ssn} \subseteq \text{EMPLOYEE.Ssn}\end{aligned}$$
>
> $$S = \{\text{DEPARTMENT}, \text{EMPLOYEE}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}, \text{IC4}, \text{IC5}, \text{IC6}, \text{IC7}\}$$

📝 EXAMPLE 8: **Project Management and Assignments (Composite Keys)**

This schema models how employees are assigned to various projects. A project is uniquely identified by its number and is controlled by a specific department. The actual assignment of an employee to a project (and the hours they work) is tracked in a separate relation. The `WORKS_ON` relation uses a **Composite Primary Key** made entirely of two Foreign Keys. We also enforce a valid mathematical range for the hours worked.

$$\begin{aligned}
&\text{PROJECT}(\underline{\text{Pnumber}} : \text{Integer}, \text{Pname} : \text{String},\\
&\quad \text{Plocation} : \text{String}, \text{Dnum} : \text{Integer})\\
&\text{IC1: Pname} \neq \text{NULL}\\
&\text{IC2: Dnum} \subseteq \text{DEPARTMENT.Dnumber}
\end{aligned}$$

$$\begin{aligned}
&\text{WORKS_ON}(\underline{\text{Essn}} : \text{String}, \underline{\text{Pno}} : \text{Integer},\\
&\quad \text{Hours} : \text{DECIMAL}(10,2))\\
&\text{IC3: } 0.5 \leq Hours \leq 50.0\\
&\text{IC4: Essn} \subseteq \text{EMPLOYEE.Ssn}\\
&\text{IC5: Pno} \subseteq \text{PROJECT.Pnumber}
\end{aligned}$$

$$S = \{\text{PROJECT, WORKS_ON}\} \cup \{\text{IC1, IC2, IC3, IC4, IC5}\}$$

📝 EXAMPLE 9: **Employee Dependents (Set Constraints & Weak Entities)**

This schema models the dependents (spouse, children) of employees for insurance purposes. Because a dependent's name alone isn't unique across the whole company, they are identified by the combination of the Employee's SSN and the Dependent's name. This creates a **Composite Primary Key**. We also restrict the "Relationship" attribute to a specific set of allowed string values.

$$\begin{aligned}
&\text{DEPENDENT}(\underline{\text{Essn}} : \text{String},\\
&\quad \underline{\text{Dependent_name}} : \text{String},\\
&\quad \text{Gender} : \text{String}, \text{Bdate} : \text{Date},\\
&\quad \text{Relationship} : \text{String})\\
&\text{IC1: } Gender \in \{'M', 'F'\}\\
&\text{IC2: } Relationship \in \{'Spouse', 'Son', 'Daughter'\}\\
&\text{IC3: Essn} \subseteq \text{EMPLOYEE.Ssn}
\end{aligned}$$

$$S = \{\text{DEPENDENT}\} \cup \{\text{IC1, IC2, IC3}\}$$

4.7.2 The Shorthand Relational Schema Notation

This notation is used in practical database design, logical modeling, and university exams where writing out domains and massive IC lists is too time-consuming. It brings the structural relationships **inline** with the attributes.

The 4 Rules of Shorthand Notation:

- **Drop the Domains:** Data types (`:String` , `:Integer`) are omitted to save space.
- **Keep Primary Keys Underlined:** The solid underline for Primary Keys (simple and composite) remains exactly the same as the formal style.
- **Bring Foreign Keys Inline:** Foreign keys are denoted using an inline arrow (→) or inline subset (⊆) directly after the attribute name, pointing to the referenced table and primary key.
  - *Format:* `FK_Attribute` → `REFERENCED_TABLE(PK_Attribute)`
- **Composite Foreign Keys use Parentheses:** If multiple attributes form a composite foreign key, wrap them in parentheses before the arrow.
  - *Format:* `(FK1, FK2)` → `REFERENCED_TABLE(PK1, PK2)`

📝 EXAMPLE 10: **University Course Registration System (Comprehensive Shorthand)**

This schema models students enrolling in university courses. We use the shorthand notation to map the relationships directly inline to keep the design compact. A student has a unique ID and a GPA restricted between 0.0 and 4.0. Courses are offered by specific departments. Enrollments link students and courses, creating a composite primary key. *(Note:* `NN` *denotes NOT NULL,* `U` *denotes UNIQUE, and* `CK` *denotes CHECK constraint, which are common additions to shorthand notation).*

$$\begin{aligned}&\text{DEPARTMENT}(\underline{\text{DeptCode}}, \text{DeptName NN U},\\&\quad \text{Building})\end{aligned}$$

$$\begin{aligned}&\text{STUDENT}(\underline{\text{StudentID}}, \text{Fname NN},\\&\quad \text{Lname NN}, \text{Email NN U},\\&\quad \text{GPA CK } (\text{GPA BETWEEN 0.0 AND 4.0}),\\&\quad \text{SDepartment} \subseteq \text{DEPARTMENT.DeptCode})\end{aligned}$$

$$\begin{aligned}&\text{COURSE}(\underline{\text{CourseID}}, \text{Title NN},\\&\quad \text{Credits CK } (\text{Credits} > 0),\\&\quad \text{OfferedBy} \subseteq \text{DEPARTMENT.DeptCode})\end{aligned}$$

$$\begin{aligned}&\text{ENROLLMENT}(\underline{\text{StudentID}} \subseteq \text{STUDENT.StudentID},\\&\quad \underline{\text{CourseID}} \subseteq \text{COURSE.CourseID},\\&\quad \text{Semester}, \text{Grade})\end{aligned}$$

5 The ER-to-Relational Mapping Algorithm

The Entity-Relationship (ER) model is a **conceptual** tool used to design a database from a business perspective. However, a DBMS cannot execute an ER diagram. To implement the database, we must translate (map) the conceptual ER model into a **logical** Relational Schema (tables).

This translation follows a strict, step-by-step algorithm. If followed correctly, the resulting relational schema will inherently satisfy **Third Normal Form (3NF)**, preventing data redundancy and update anomalies.

5.1 Regular (Strong) Entities and Simple Attributes

The Rule: Map each strong entity type to a relation (table). Include all simple attributes. Choose one of the candidate keys as the Primary Key (PK).

- **Composite Attributes:** Flatten them. Include only the primitive sub-attributes as separate columns.
- **Derived Attributes: Discard them.** They are computed on the fly in SQL and never stored as columns.

EXAMPLE 11:

`EMPLOYEE` entity with simple attributes `Emp_ID` (Key), `Fname`, `Lname`. Composite attribute `Address` (Street, City, Zip). Derived attribute `Age` (from Bdate).

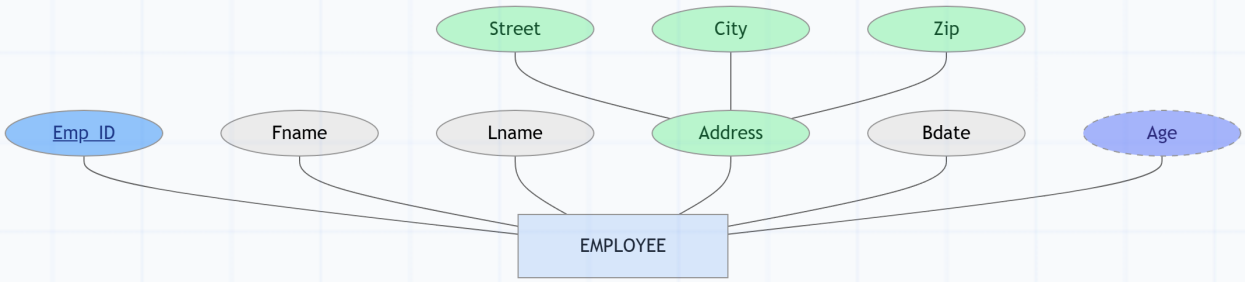


Diagram 47: EMPLOYEE entity with simple attributes Emp_ID (Key), Fname, Lname. Composite attribute Address (Street, City, Zip). Derived attribute Age (from Bdate).

Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}}, \text{Fname}, \text{Lname}, \text{Street}, \text{City}, \text{Zip}, \text{Bdate})$$

Note

`Address` *is flattened into Street, City, Zip.* `Age` *is dropped.*

5.2 Weak Entities

The Rule: Map the weak entity to a new relation. The Primary Key of this new relation is the **Composite Key** consisting of the owner entity's PK + the weak entity's partial key (discriminator). Include the owner's PK as a Foreign Key (FK).

EXAMPLE 12:

`DEPENDENT` is a weak entity of `EMPLOYEE`. Partial key is `Dependent_Name`.

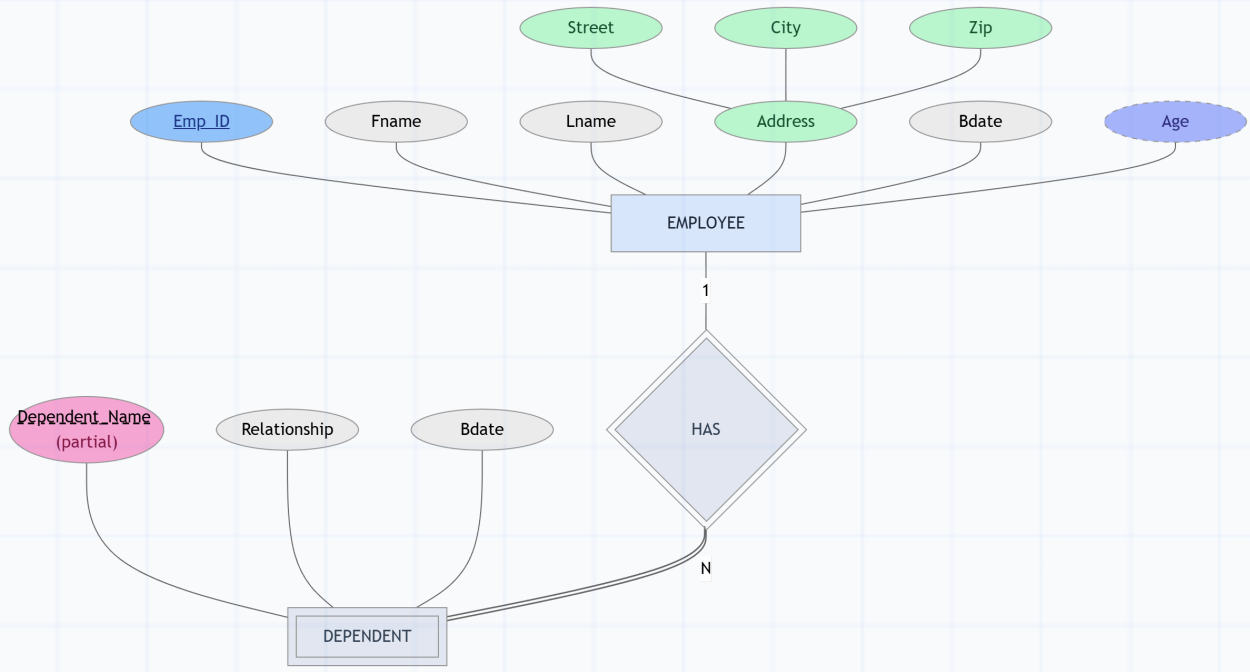


Diagram 48: DEPENDENT is a weak entity of EMPLOYEE. Partial key is Dependent_Name.

Show Solution

$$\text{DEPENDENT}(\underline{\text{Emp_ID}} : \text{Integer},$$
$$\underline{\text{Dependent_Name}} : \text{String},$$
$$\text{Relationship} : \text{String}, \text{Bdate} : \text{Date})$$
$$\text{IC1: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$$

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer})$$

$$S = \{\text{DEPENDENT}, \text{EMPLOYEE}\} \cup \{\text{IC1}\}$$

Note

`Emp_ID` *is underlined because it is part of the composite PK, and it also acts as the FK to EMPLOYEE.*

5.3 Multivalued Attributes

The Rule: Multivalued attributes cannot be mapped as a single column (violating 1NF). Create a **new relation** to represent it. The PK of this new relation is the combination of the owning entity's PK and the multivalued attribute itself.

EXAMPLE 13:

`EMPLOYEE` has a multivalued attribute `Degree_Certifications`.

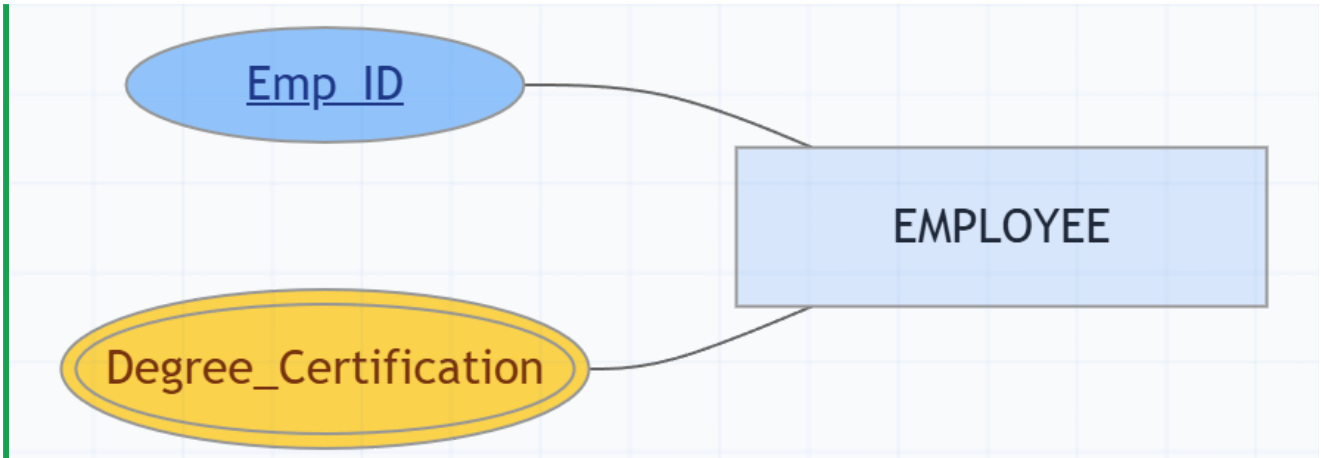


Diagram 49: EMPLOYEE has a multivalued attribute Degree_Certifications.

Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer})$$

$$\text{EMPLOYEE_DEGREES}(\underline{\text{Emp_ID}} : \text{Integer}, \underline{\text{Degree_Certification}})$$

$$\text{IC1: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$$

$$S = \{\text{EMPLOYEE}, \text{EMPLOYEE_DEGREES}\} \cup \{\text{IC1}\}$$

> **Note**
>
> *This creates a 1:N relationship implicitly between EMPLOYEE and EMPLOYEE_DEGREES.*

5.4 Mapping Relationships, Relationship Attributes, and Participation

When translating an Entity-Relationship (ER) diagram into a Relational Schema, the **cardinality** (1:N, 1:1, M:N) dictates the *structural mapping* (where tables and foreign keys go). However, the **participation constraint** (Total vs. Partial) dictates the *integrity constraints* (specifically, whether a Foreign Key can be `NULL`).

- **Total Participation (Double line in ER):** Every instance of the entity **must exist** in the relationship. (e.g., "Every order *must* have a customer").
- **Partial Participation (Single line in ER):** An instance of the entity **may or may not exist** in the relationship. (e.g., "A customer *may or may not* have placed an order").

Below are the comprehensive rules and cases for mapping all binary relationship scenarios, including how to handle relationship attributes.

5.4.1 Case 1: 1:N Relationship (Partial Participation on N-side) + Rel Attribute

Rules to Apply:

- **Structural Rule:** Add the 1-side PK as a Foreign Key in the N-side table.
- **Attribute Rule:** Place the relationship attribute in the N-side table.
- **Constraint Rule:** Because the N-side is Partial, the FK **may or may not** have a value (it is nullable).

The Structural Rule: Use the **Foreign Key approach**. Take the Primary Key of the "1-side" (parent) entity and add it as a Foreign Key attribute in the "N-side" (child) entity's relation. Relationship attributes are also placed on the N-side.

The Participation Impact:

- **Partial Participation on the N-side:** The Foreign Key in the N-side table is allowed to be `NULL`.
  - *Example:* An `EMPLOYEE` may or may not be assigned to a `DEPARTMENT`. The `Dept_ID` FK in `EMPLOYEE` is nullable.
- **Total Participation on the N-side:** The Foreign Key in the N-side table **cannot** be `NULL`. You must add a `NOT NULL` constraint to the FK.
  - *Example:* Every `ORDER` *must* belong to a `CUSTOMER`. The `Cust_ID` FK in `ORDER` is `NOT NULL`.
- **Total Participation on the 1-side:** This does not change the FK placement, but it implies a business rule: every record in the 1-side table *must* be referenced by at least one record in the N-side table. (This is typically enforced by application logic, as standard SQL lacks a direct "must be referenced by at least one child" constraint).

EXAMPLE 14:

A `DEPARTMENT` (1) manages `EMPLOYEE`s (N). The relationship `WORKS_FOR` has a relationship attribute `Start_Date`. An employee **may or may not** be assigned to a department yet (e.g., a new hire in orientation). A department **may or may not** have employees.

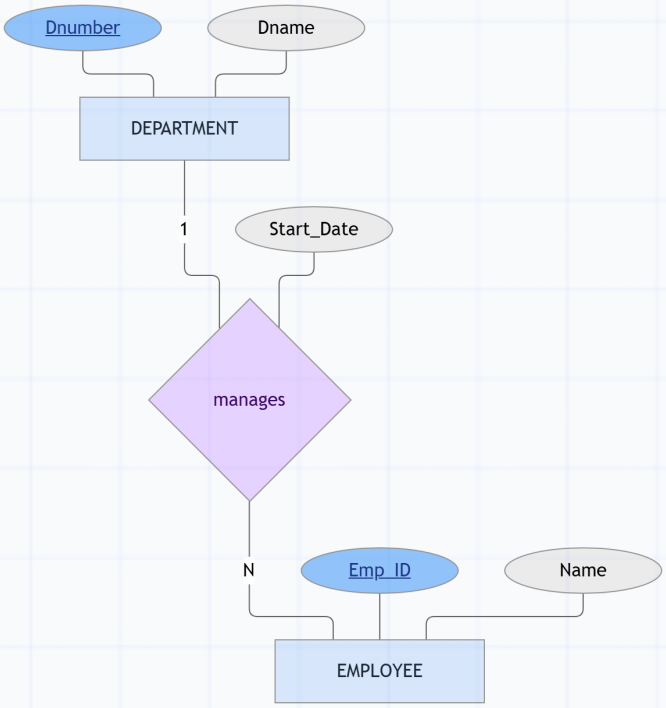


Diagram 50: 1:N Relationship (Partial Participation on N-side) + Rel Attribute

Show Solution

$$\text{DEPARTMENT}(\underline{\text{Dnumber}} : \text{Integer}, \text{Dname} : \text{String})$$
$$\text{IC1: Dname} \neq \text{NULL}$$

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Name} : \text{String},$$
$$\text{Dno} : \text{Integer}, \text{Start_Date} : \text{Date})$$
$$\text{IC2: Dno} \subseteq \text{DEPARTMENT.Dnumber}$$

$$S = \{\text{DEPARTMENT}, \text{EMPLOYEE}\} \cup \{\text{IC1}, \text{IC2}\}$$

> **Note**
>
> `Dnumber` is added to the N-side (`EMPLOYEE`) as `Dno` . The relationship attribute `Start_Date` is placed alongside it. Because participation is Partial, `Dno` is allowed to be `NULL` (no $\neq$ `NULL` constraint).

5.4.2 Case 2: 1:N Relationship (Total Participation on N-side) + Rel Attribute

Rules to Apply:

- **Structural Rule:** Add the 1-side PK as a Foreign Key in the N-side table.
- **Attribute Rule:** Place the relationship attribute in the N-side table.
- **Constraint Rule:** Because the N-side is Total, the FK **must exist**. Add a `NOT NULL` constraint to the FK.

EXAMPLE 15:

A `CUSTOMER` (1) places `ORDER` s (N). The relationship `PLACES` has an attribute `Order_Status` . Every single order **must exist** for a valid customer. A customer **may or may not** have placed orders.

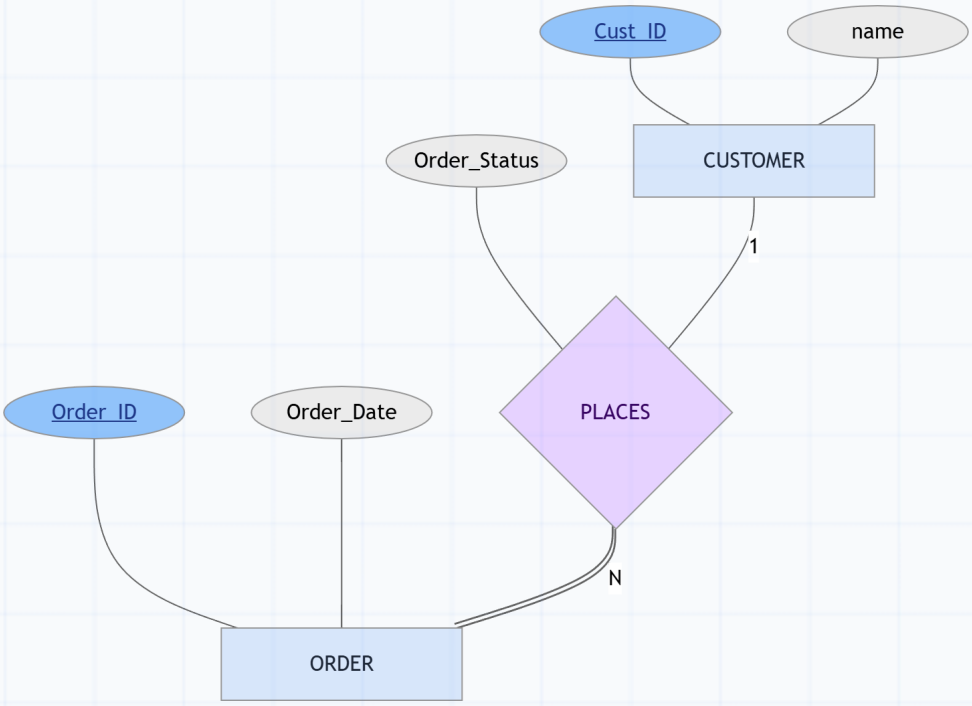


Diagram 51: A CUSTOMER (1) places ORDERs (N). The relationship PLACES has an attribute Order_Status. Every single order must exist for a valid customer. A customer may or may not have placed orders.

✅ Show Solution

$$\text{CUSTOMER}(\underline{\text{Cust_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{ORDER}(\underline{\text{Order_ID}} : \text{Integer}, \text{Order_Date} : \text{Date},$$
$$\text{Cust_ID} : \text{Integer}, \text{Order_Status} : \text{String})$$
$$\text{IC2: Cust_ID} \neq \text{NULL}$$
$$\text{IC3: Cust_ID} \subseteq \text{CUSTOMER.Cust_ID}$$

$$S = \{\text{CUSTOMER}, \text{ORDER}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}\}$$

> **Note**
>
> Because an order **must exist** for a customer (Total Participation), we explicitly enforce **IC3:** `Cust_ID ≠ NULL`. The relationship attribute `Order_Status` is placed in the N-side table.

5.4.3 Case 3: 1:1 Relationship (Total on One Side, Partial on Other) + Rel Attribute

Rules to Apply:

- **The Golden Rule:** Always place the Foreign Key on the side with **Total Participation** to avoid unnecessary `NULL`s.
- **Attribute Rule:** Place the relationship attribute in the table that receives the Foreign Key.
- **Constraint Rule:** The FK on the Total side **must exist** (`NOT NULL`).

📝 EXAMPLE 16:

A `PERSON` (1) owns a `PASSPORT` (1). The relationship `OWNS` has an attribute `Issue_Date`. A passport **must exist** for a valid person (Total). A person **may or may not** own a passport (Partial).

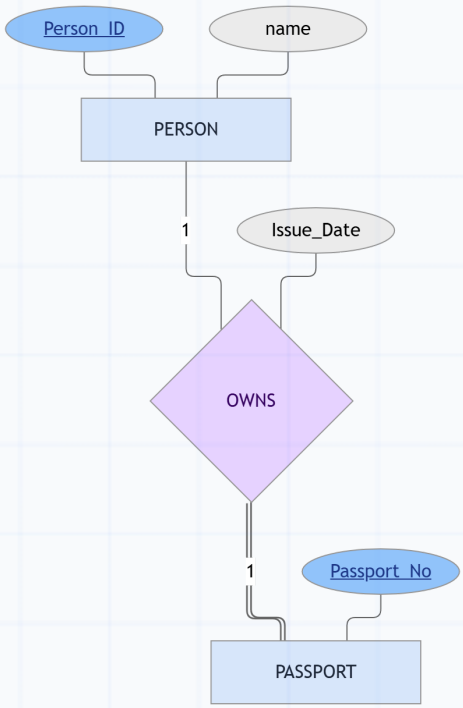


Diagram 52: A PERSON (1) owns a PASSPORT (1). The relationship OWNS has an attribute Issue_Date. A passport must exist for a valid person (Total). A person may or may not own a passport (Partial).

Show Solution

$$\text{PERSON}(\underline{\text{Person_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{PASSPORT}(\underline{\text{Passport_No}} : \text{String},$$
$$\text{Issue_Date} : \text{Date}, \text{Person_ID} : \text{Integer})$$
$$\text{IC2: Person_ID} \neq \text{NULL}$$
$$\text{IC3: Person_ID} \subseteq \text{PERSON.Person_ID}$$

$$S = \{\text{PERSON}, \text{PASSPORT}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}\}$$

Note

Following the Golden Rule, `Person_ID` is placed in `PASSPORT` (the Total side). The relationship attribute `Issue_Date` is also placed there. Because a passport **must exist** with an owner, **IC2** enforces `Person_ID ≠ NULL`.

5.4.4 Case 4: 1:1 Relationship (Partial on BOTH Sides) + Rel Attribute

Rules to Apply:

- **Structural Rule:** Place the FK on either side (structurally valid either way).
- **Attribute Rule:** Place the relationship attribute in the table that receives the FK.
- **Constraint Rule:** Because both sides are Partial, the FK **may or may not** have a value (it must allow `NULL`).

EXAMPLE 17:

An `EMPLOYEE` (1) is assigned a `PARKING_SPOT` (1). The relationship `ASSIGNED` has an attribute `Zone`. An employee **may or may not** have a spot. A spot **may or may not** be assigned to an employee.

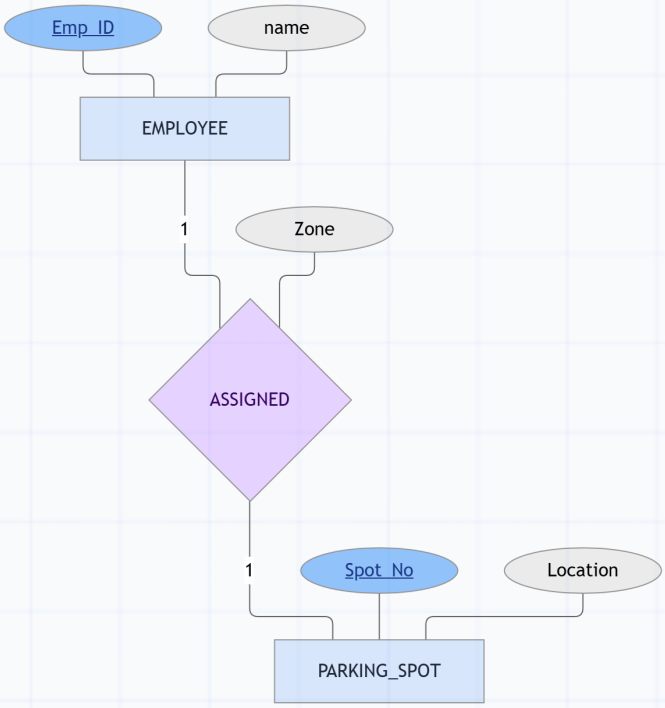


Diagram 53: An EMPLOYEE (1) is assigned a PARKING_SPOT (1). The relationship ASSIGNED has an attribute Zone. An employee may or may not have a spot. A spot may or may not be assigned to an employee.

Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{PARKING_SPOT}(\underline{\text{Spot_No}} : \text{String}, \text{Location} : \text{String},$$
$$\text{Zone} : \text{String}, \text{Emp_ID} : \text{Integer})$$
$$\text{IC2: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$$

$$S = \{\text{EMPLOYEE}, \text{PARKING_SPOT}\} \cup \{\text{IC1}, \text{IC2}\}$$

Note

We placed the FK and the relationship attribute `Zone` in `PARKING_SPOT`. Because both sides *may or may not exist* in the relationship, `Emp_ID` is allowed to be `NULL` (no $\neq$ `NULL` constraint).

5.4.5 Case 5: M:N Relationship + Relationship Attribute

Rules to Apply:

- **Structural Rule:** Create a brand new **Junction (Associative) Table**.

- **Key Rule:** The PK of the junction table is the composite of both entities' PKs.
- **Attribute Rule:** The relationship attribute **must** be placed inside the new junction table.

EXAMPLE 18:

A `STUDENT` (M) enrolls in a `COURSE` (N). The relationship `ENROLLS_IN` has an attribute `Grade`. A student **may or may not** be enrolled. A course **may or may not** have students.

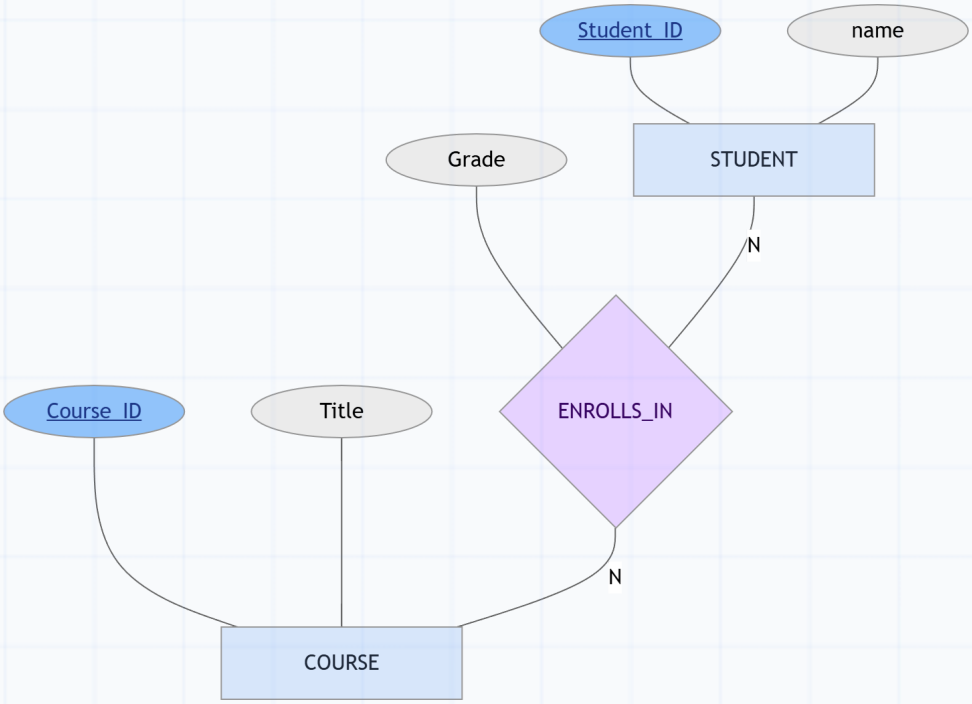


Diagram 54: A STUDENT (M) enrolls in a COURSE (N). The relationship ENROLLS_IN has an attribute Grade. A student may or may not be enrolled. A course may or may not have students.

Show Solution

$$\text{STUDENT}(\underline{\text{Student_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{COURSE}(\underline{\text{Course_ID}} : \text{String}, \text{Title} : \text{String})$$
$$\text{IC2: Title} \neq \text{NULL}$$

$$\text{ENROLLMENT}(\underline{\text{Student_ID}} : \text{Integer},$$
$$\underline{\text{Course_ID}} : \text{String}, \text{Grade} : \text{String})$$
$$\text{IC3: } Grade \in \{'A', 'B', 'C', 'D', 'F'\}$$
$$\text{IC4: Student_ID} \subseteq \text{STUDENT.Student_ID}$$
$$\text{IC5: Course_ID} \subseteq \text{COURSE.Course_ID}$$

$$S = \{\text{STUDENT}, \text{COURSE}, \text{ENROLLMENT}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}, \text{IC4}, \text{IC5}\}$$

> **Note**
>
> The M:N relationship is resolved via the `ENROLLMENT` junction table. The relationship attribute `Grade` lives exclusively in this new table. The composite PK is (`Student_ID` , `Course_ID`).

5.4.6 Case 6: Unary (Recursive) 1:N Relationship + Rel Attribute

Rules to Apply:

- **Structural Rule:** Add a Foreign Key to the **same table**, referencing its own Primary Key.
- **Attribute Rule:** Place the relationship attribute in the same table alongside the recursive Foreign Key.

EXAMPLE 19:

The `EMPLOYEE` entity participates in a unary 1:N relationship `SUPERVISION` . An employee (as a manager) supervises other employees. The relationship has an attribute `Start_Date` . An employee **may or may not** have a supervisor (e.g., the CEO has none).

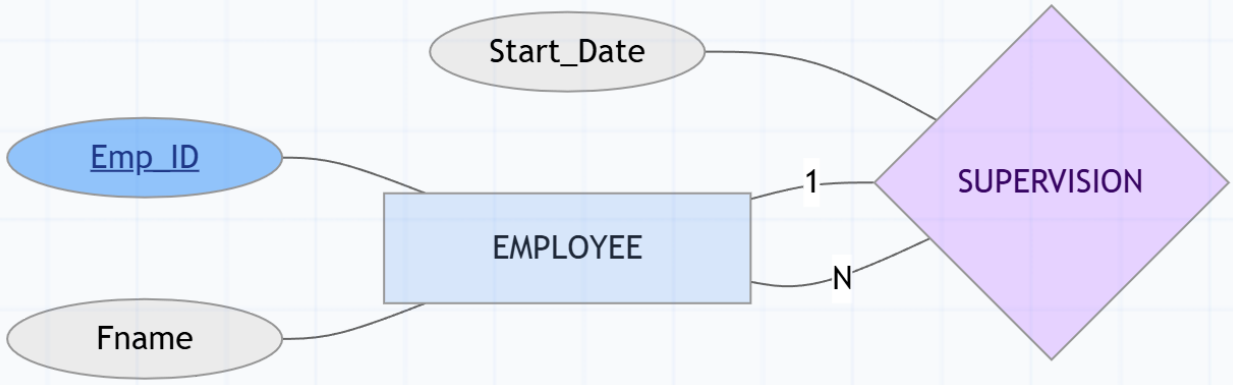


Diagram 55: The EMPLOYEE entity participates in a unary 1:N relationship SUPERVISION. An employee (as a manager) supervises other employees. The relationship has an attribute Start_Date. An employee may or may not have a supervisor (e.g., the CEO has none).

> **Show Solution**

$$\begin{aligned}&\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Fname} : \text{String},\\ &\qquad \text{Super_Emp_ID} : \text{Integer}, \text{Start_Date} : \text{Date})\\ &\text{IC1: Fname} \neq \text{NULL}\\ &\text{IC2: Super_Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}\\ &S = \{\text{EMPLOYEE}\} \cup \{\text{IC1}, \text{IC2}\}\end{aligned}$$

Note

`Super_Emp_ID` is added to the `EMPLOYEE` table to point to `Emp_ID`. The relationship attribute `Supervision_Start_Date` is placed right next to it. Because the CEO **may or may not** have a supervisor, `Super_Emp_ID` is allowed to be `NULL`.

5.4.7 Case 7: Weak Entity (Identifying Relationship) + Partial Key

Rules to Apply:

- **Structural Rule:** Map the weak entity to a new table.
- **Key Rule:** The Primary Key is **Composite**, formed by the owner entity's PK + the weak entity's partial key (discriminator). Both are underlined.
- **Constraint Rule:** The owner's PK acts as a Foreign Key and **must exist** (`NOT NULL`) because a weak entity **must exist** with its owner (Total participation in the identifying relationship).

EXAMPLE 20:

`DEPENDENT` is a weak entity of `EMPLOYEE`. It has a partial key `Dep_Name`. A dependent **must exist** for an employee (Total participation). An employee **may or may not** have dependents (Partial participation).

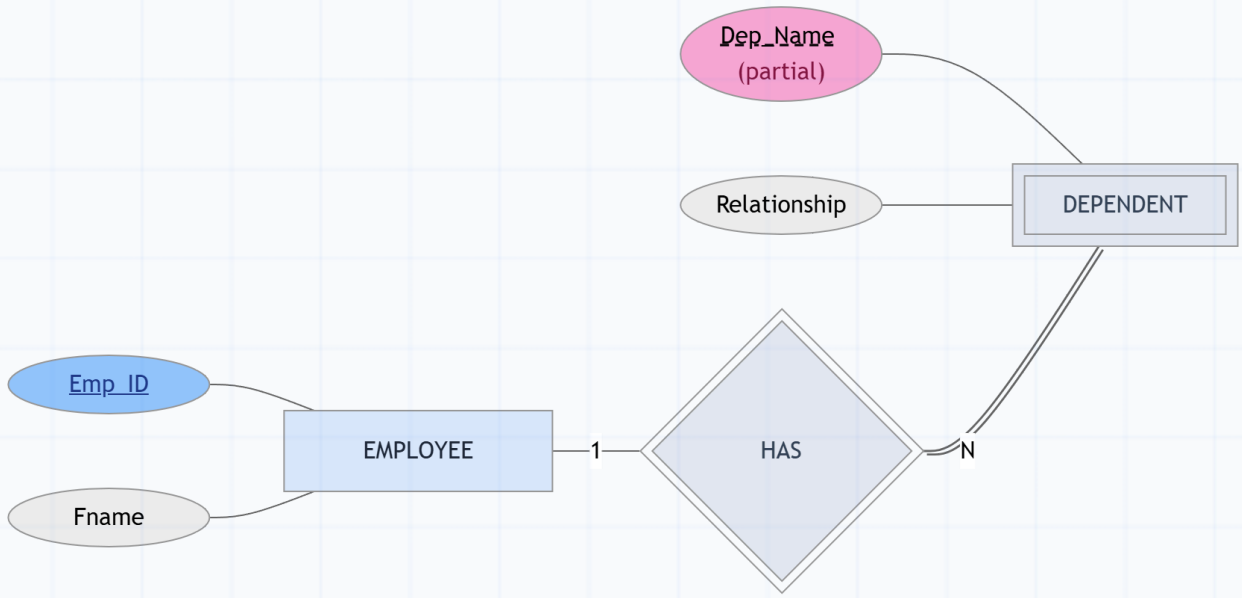


Diagram 56: DEPENDENT is a weak entity of EMPLOYEE. It has a partial key Dep_Name. A dependent must exist for an employee (Total participation). An employee may or may not have dependents (Partial participation).

Show Solution

$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Fname} : \text{String})$
$\text{IC1: Fname} \neq \text{NULL}$

$\text{DEPENDENT}(\underline{\text{Emp_ID}} : \text{Integer},$
$\quad \underline{\text{Dep_Name}} : \text{String},$
$\quad \text{Relationship} : \text{String})$
$\text{IC2: } Relationship \in \{'Spouse','Son','Daughter'\}$
$\text{IC3: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$

$S = \{\text{EMPLOYEE, DEPENDENT}\} \cup \{\text{IC1, IC2, IC3}\}$

> **Note**
>
> The PK of `DEPENDENT` is composite (`Emp_ID` , `Dep_Name`). Both are underlined. `Emp_ID` is underlined because it is part of the PK, and it simultaneously acts as the Foreign Key. Since `Emp_ID` is already **underlined** as part of the Composite Primary Key of `DEPENDENT` , the Entity Integrity Constraint **automatically** guarantees that `Emp_ID` $\neq$ `NULL` .

Chapter4 Relational Algebra

CHAPTER OVERVIEW

Understanding relational algebra is essential for grasping how query optimizers execute, transform, and evaluate database queries under the hood. The chapter covers the fundamental definitions, five primary vocabulary terms, and the core operations:

- **Unary Operations:** Selection (σ), Projection (π), and Rename (ρ).
- **Set-Theoretic Operations:** Union ($\cup$), Intersection ($\cap$), Set Difference ($-$), and Cartesian Product ($\times$).
- **Joins & Advanced Concepts:** How foreign keys and conditions transform raw Cartesian products into meaningful joins across normalized relations.

> 🎯 **Learning Outcomes**
>
> At the end of this chapter, you will be able to:
>
> 1. Explain the difference between declarative SQL statements and procedural relational algebra expressions.
> 2. Define and correctly apply core terms: *Relation*, *Tuple*, *Attribute*, *Degree* (number of columns), and *Cardinality* (number of rows).
> 3. Filter relation rows using the **Selection (σ)** operator with logical and comparison conditions.
> 4. Filter relation columns using the **Projection (π)** operator and understand its automatic duplicate elimination property.
> 5. Rename relations and attributes using the **Rename (ρ)** operator to resolve column name conflicts during complex operations.
> 6. Evaluate **Union ($\cup$)**, **Intersection ($\cap$)**, and **Set Difference ($-$)** while enforcing union-compatibility rules (matching degrees and compatible attribute domains).
> 7. Compute the **Cartesian Product ($\times$)** of two relations and calculate the resulting degree and cardinality.
> 8. Explain why unconditioned Cartesian products create invalid data pairs and how primary/foreign key relationships filter these into meaningful join operations.
> 9. Convert standard SQL queries (`SELECT` , `WHERE` , `JOIN` , `UNION` , `EXCEPT` , `INTERSECT`) into equivalent relational algebra expressions.

1 What Is Relational Algebra?

SQL is a *declarative* language: you say *what* you want; the DBMS decides *how* to compute it. The "how" is determined by translating your SQL into **relational algebra** — a *procedural* formal language where every operation is an explicit step. Understanding relational algebra means understanding how the query optimizer thinks, why certain queries are fast, and what is happening beneath every `JOIN` you write.

> 📖 DEFINITION 1: **Relational Algebra**

Relational algebra is a **procedural formal query language** for relations. Each *operation* takes one or two relations as input and returns a new relation as output.

Closure property

The result of every relational algebra operation is itself a relation. This means operations can be freely composed — the output of one becomes the input of the next.

$$Expression = Opk(\ldots Op2(Op1(R1, R2))\ldots)$$

There are **6 fundamental operations** (from which all others can be derived) and several convenient **additional (derived) operations**.

Group	Sub-Category / Type	Operations
Fundamental (Basic)	Unary	Selection, Projection, Rename
	Set-Theoretic	Union, Set Difference, Cartesian Product
Derived (Additional)	Set-Theoretic	Set Intersection
	Joins	Natural Join, Theta Join, Outer Joins, Semijoins
	Division	Division
Extended	Aggregations	Aggregate Functions (Group By)

2 Fundamental Concepts in Relational Algebra

Before you can work with relational algebra operators, you need to know the vocabulary. The good news is that there are only five terms to remember.

- Relation is just a table. It has rows and columns, and it represents a set of data about a specific entity - like employees or departments. In relational algebra, every operation takes one or more relations as input and produces a relation as output.
- Tuple is a single row in that table. If your `Employees` table has 10 rows, it has 10 tuples. Each tuple represents one record - one employee, in this case.
- Attribute is a column. The `Employees` table might have attributes like `employee_id`, `name`, and `department_id`. Attributes define what kind of data each tuple holds.
- Degree is the number of attributes in a relation. If `Employees` has five columns, its degree is 5. This matters when you're combining relations - some operations require matching degrees.
- Cardinality is the number of tuples in a relation. If your `Employees` table has 200 rows, its cardinality is 200. Cardinality changes as you add or remove rows; degree changes only when you add or remove columns.

3 Fundamental Operations: Unary Operations (Operate on a single relation)

3.1 The SELECT Operation — σ (Filter Rows)

The **SELECT** operation is used to select a subset of the tuples from a relation that satisfy a **selection condition**. One can consider the SELECT operation to be a filter that keeps only those tuples that satisfy a qualifying condition. The SELECT operation can also be visualized as a horizontal partition of the relation into two sets of tuples-those tuples that satisfy the condition and are selected, and those tuples that do not satisfy the condition and are discarded.

$$\sigma_{\text{condition}}(\text{R})$$

RELATIONAL ALGEBRA (RELAX)

```
σ condition (R)
```

SQL EQUIVALENT

```
SELECT * FROM R WHERE condition
```

Property	Value
Input	Relation R
Output columns	Same as R (degree unchanged)
Output rows	All tuples where condition is true (cardinality ≤ |R|)
Condition operators	=,≠,<,>,≤,≥,combined with ∧ (AND), ∨ (OR), ¬ (NOT)

📝 EXAMPLE 1: **Selection by Salary Threshold**

List all employees whose salary is greater than 65,000.

✅ Show Solution

$$\sigma_{\text{Salary}>65000}(\text{Employee})$$

RELATIONAL ALGEBRA (RELAX)

```
σ Salary > 65000 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Salary > 65000
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 65000 (Employee)
```

EXAMPLE 2: Selection by Department ID

List all employees who belong to department 1.

Show Solution

$$\sigma_{Dno=1} Employee$$

RELATIONAL ALGEBRA (RELAX)

```
σ Dno = 1 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Dno = 1
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Dno = 1 (Employee)
```

EXAMPLE 3: Selection with Logical NOT

List all employees who do not work in department 2.

Show Solution

$$\sigma_{\neg(Dno=2)}Employee$$

RELATIONAL ALGEBRA (RELAX)

```
σ ¬ Dno = 2 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE NOT Dno = 2
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ ¬ Dno = 2 (Employee)
```

EXAMPLE 4: Compound Selection with Logical AND

List all employees who work in department 1 AND earn more than 60,000.

Show Solution

$$\sigma_{\text{Dno}=1 \wedge \text{Salary}>60000}(\text{Employee})$$

RELATIONAL ALGEBRA (RELAX)

```
σ Dno = 1 ∧ Salary > 60000 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Dno = 1 AND Salary > 60000
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Dno = 1 ∧ Salary > 60000 (Employee)
```

EXAMPLE 5: Pattern Matching (Starts With)

Select all employees whose first name starts with the letter 'H'.

Show Solution

$$\sigma_{Fname\ LIKE\ 'H\%'}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
σ Fname LIKE 'H%' (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Fname LIKE 'H%'
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Fname LIKE 'H%' (Employee)
```

EXAMPLE 6: Pattern Matching (Contains)

Select all employees whose first name contains the letter 'm'.

Show Solution

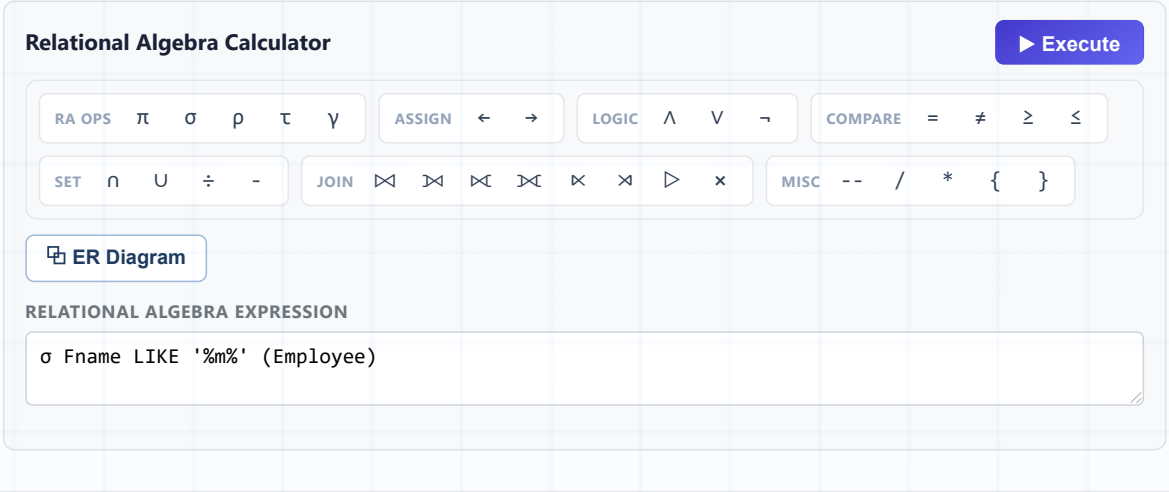
$$\sigma_{Fname\ LIKE\ '\%m\%'}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
σ Fname LIKE '%m%' (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Fname LIKE '%m%'
```



3.2 The PROJECT Operation— π (Filter Columns)

While Selection operation works on rows, similarly projection operation of relational algebra works on columns. It basically allows us to pick specific columns from a given relational table based on the given condition and ignoring all the other remaining columns.

> **Note**
>
> By Default, projection operation removes **duplicate values**.

$$\pi_{A_1, A_2, \ldots, A_k}(R)$$

RELATIONAL ALGEBRA (RELAX)

```
π A_1, A_2, A_k (R)
```

SQL EQUIVALENT

```
SELECT DISTINCT A_1, A_2, A_k FROM R
```

EXAMPLE 7: Simple Projection

List the first name, last name, and salary of every employee.

✅ Show Solution

$$\pi_{\text{Fname,Lname,Salary}}(\text{Employee})$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Lname, Salary (Employee)
```

SQL EQUIVALENT

```
SELECT DISTINCT Fname, Lname, Salary FROM Employee
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Lname, Salary (Employee)
```

EXAMPLE 8: Combined Select + Project

List the first name and salary of all female employees earning above 60,000.

Show Solution

$$\pi_{\text{Fname,Salary}}(\sigma_{\text{Gender}='F' \wedge \text{Salary}>60000}(\text{Employee}))$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Salary (σ Gender = 'F' ∧ Salary > 60000 (Employee))
```

SQL EQUIVALENT

```
SELECT DISTINCT Fname, Salary FROM (SELECT * FROM Employee WHERE Gender = 'F' AND Salary > 60000)
AS t1
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Salary (σ Gender = 'F' ∧ Salary > 60000 (Employee))
```

EXAMPLE 9: Combined Select + Project

Retrieve Fname as FIRST_NAME of all employees

Show Solution

$$\pi_{FIRST_NAME \leftarrow Fname}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
π FIRST_NAME (ρ FIRST_NAME←Fname (Employee))
```

SQL EQUIVALENT

```
SELECT DISTINCT FIRST_NAME FROM (SELECT * FROM (SELECT Fname AS FIRST_NAME FROM Employee) AS src)
AS t1
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π FIRST_NAME (ρ FIRST_NAME←Fname (Employee))
```

3.3 The Rename Operation— ρ (Renames a relation or its attribute names)

Rename operator basically allows you to give a temporary name to a specific relational table or to its columns. It is very useful when we want to avoid ambiguity, especially in complex Queries. Rename is a unary operation used for renaming attributes of a relation.

Rename Relation

$$\rho_S(R)$$

RELATIONAL ALGEBRA (RELAX)

```
ρ S (R)
```

SQL EQUIVALENT

```
SELECT * FROM R AS S
```

Rename Attributes

$$\rho_{att1_new \leftarrow att1, att2_new \leftarrow att2}(R)$$

RELATIONAL ALGEBRA (RELAX)

```
ρ att1_new←att1, att2_new←att2 (R)
```

Rename Relation and Attributes

$$\rho_{R_new}(\rho_{att1_new \leftarrow att1}(\pi_{att1,att2}(R)))$$

RELATIONAL ALGEBRA (RELAX)

```
ρ R_new (ρ att1_new←att1 (π att1, att2 (R)))
```

EXAMPLE 10:

Create a named subexpression `Supervisor` containing SSN, first name, and last name of all employees, using shorter column names.

Show Solution

$$\rho_{Supervisor}(\pi_{Ssn,Fname,Lname}(Employee))$$

RELATIONAL ALGEBRA (RELAX)

```
ρ Supervisor (π Ssn, Fname, Lname (Employee))
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT DISTINCT Ssn, Fname, Lname FROM Employee) AS Supervisor
```

Relational Algebra Calculator

Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
ρ Supervisor (π Ssn, Fname, Lname (Employee))
```

4 Set-Theoretic Operations (Operate on two relations)

Set-theoretic operations in relational algebra are binary operators that combine or compare tuples from two relations, treating each relation as a formal set of tuples.

let's $R(A_1, A_2, ..., A_n)$and $S(B_1, B_2, ..., B_n)$are relations. The Set Compatibility Rules - ∪,∩, and − all require union-compatible operands:

1. Degree(R) = Degree(S): have the same number of attributes.
2. Domain(A_i) = Domain(B_i) , for 1<= i <=n: Corresponding attributes must have **compatible domains**(same or convertible types).

4.1 Union — ∪ (Combine Two Relations)

📖 DEFINITION 2: **Union (R ∪ S)**

Returns all tuples that belong to relation R, relation S, or both. The **result takes the attribute names from the left operand**(R)

$$R \cup S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ∪ S)
```

SQL EQUIVALENT

```
SELECT * FROM R
UNION
SELECT * FROM S
```

📝 EXAMPLE 11: — Union

Find all SSNs that appear in either the Employee table or the Works_On table.

✅ Show Solution

$$\pi_{Ssn}(Employee) \cup \pi_{Essn}(Works_On)$$

RELATIONAL ALGEBRA (RELAX)

```
(π Ssn (Employee) ∪ π Essn (Works_On))
```

SQL EQUIVALENT

```
SELECT DISTINCT Ssn FROM Employee
UNION
SELECT DISTINCT Essn FROM Works_On
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(π Ssn (Employee) ∪ π Essn (Works_On))
```

EXAMPLE 12: — Union

Retrieve the Social Security Numbers (Ssn) for all employees who work in **Department 3** or directly supervise an employee in **Department 3**

Show Solution

To solve this using **Relational Algebra**, we break the problem into two parts and combine them using the **Union** operator:

1. Find employees working in Department 3.
2. Find supervisors of employees working in Department 3
3. Combine results using Union.

$$R_1 = \pi_{\text{Ssn}}(\sigma_{\text{Dno}=3}(\text{Employee}))$$
$$R_2 = \pi_{\text{Super_ssn}}(\sigma_{\text{Dno}=3}(\text{Employee}))$$
$$\text{Result} = R_1 \cup R_2$$

RELATIONAL ALGEBRA (RELAX)

```
R_1 = π Ssn (σ Dno = 3 (Employee))
R_2 = π Super_ssn (σ Dno = 3 (Employee))
(R_1 ∪ R_2)
```

SQL EQUIVALENT

```
WITH R_1 AS (
  SELECT DISTINCT Ssn FROM (SELECT * FROM Employee WHERE Dno = 3) AS t1
),
R_2 AS (
  SELECT DISTINCT Super_ssn FROM (SELECT * FROM Employee WHERE Dno = 3) AS t2
),
Result AS (
  SELECT * FROM R_1
UNION
SELECT * FROM R_2
)
SELECT * FROM Result
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
R_1 = π Ssn (σ Dno = 3 (Employee))
R_2 = π Super_ssn (σ Dno = 3 (Employee))
```

4.2 Intersection — ∩

📖 DEFINITION 3: **Intersection (R ∩ S)**

Returns only the tuples that are present in **both** relation R and relation S.

$$R \cap S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ∩ S)
```

SQL EQUIVALENT

```
SELECT * FROM R
INTERSECT
SELECT * FROM S
```

Note

Intersection is a **derived** operation — it can always be rewritten using set difference.

$$R \cap S = R - (R - S)$$

EXAMPLE 13: **Intersection**

Retrieve the Male Employees that work in department 1

Show Solution

Steps:

- Retrieve all employees in department 1 as R1.
- Retrieve all Male employees in Employee as R2.
- Result = R1 INTERSECTION R2.

$$R_1 = \sigma_{\text{Dno}=1}(\text{Employee})$$
$$R_2 = \sigma_{\text{Gender}='M'}(\text{Employee})$$
$$\text{Result} = R_1 \cap R_2$$

RELATIONAL ALGEBRA (RELAX)

```
R_1 = σ Dno = 1 (Employee)
R_2 = σ Gender = 'M' (Employee)
(R_1 ∩ R_2)
```

SQL EQUIVALENT

```
WITH R_1 AS (
  SELECT * FROM Employee WHERE Dno = 1
),
R_2 AS (
  SELECT * FROM Employee WHERE Gender = 'M'
),
Result AS (
  SELECT * FROM R_1
INTERSECT
SELECT * FROM R_2
)
SELECT * FROM Result
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
R_1 = σ Dno = 1 (Employee)
R_2 = σ Gender = 'M' (Employee)
```

4.3 Set Difference — − (In One But Not the Other)

📖 DEFINITION 4: **Set Difference (R - S)**

Returns all tuples that are present in relation R but **not** present in relation S. Requires union compatibility.

$$R - S$$

RELATIONAL ALGEBRA (RELAX)

```
(R - S)
```

SQL EQUIVALENT

```
SELECT * FROM R
EXCEPT
SELECT * FROM S
```

📝 EXAMPLE 14: **Set Difference**

Find the SSNs of employees who are NOT currently assigned to any project.

✅ Show Solution

$$\pi_{Ssn}(Employee) - \pi_{Essn}(Works_On)$$

RELATIONAL ALGEBRA (RELAX)

```
(π Ssn (Employee) - π Essn (Works_On))
```

SQL EQUIVALENT

```
SELECT DISTINCT Ssn FROM Employee
EXCEPT
SELECT DISTINCT Essn FROM Works_On
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(π Ssn (Employee) - π Essn (Works_On))
```

5 Cartesian Product — × (All Combinations)

Unary relational algebra operations like **Select** and **Project** operate on a **single relation** at a time. However, in real-world relational databases, data is organized according to **normalization principles** to eliminate redundancy, avoid data anomalies, and maintain data integrity. As a result, related information is split across multiple tables (for example, storing employee details in an `Employee` relation and department details in a `Department` relation).

To answer queries that require information spanning multiple tables, we need a mechanism to correlate tuples across relations:

- **Combining Schemas:** The **Cartesian Product ($R \times S$)** serves as the fundamental relational algebra operation that pairs every tuple in R with every tuple in S. This generates a new relation containing the combined attributes of both relations (Degree: degree(R) + degree(S)).
- **The Foundation for Joins:** While a raw Cartesian Product generates all possible combinations—including many non-matching or irrelevant pairs—it forms the mathematical foundation for **Join operations**. When followed by a **Selection** on a matching key condition (such as R.Dno = S.Dnumber), it narrows down the combined dataset to extract meaningful, related records across normalized tables.

Summary

Without the Cartesian Product (and the join operations built upon it), a relational system could only query individual, isolated tables, defeating the core purpose of a normalized relational database design.

$$R \times S$$

RELATIONAL ALGEBRA (RELAX)

```
(R × S)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM R) AS t1 CROSS JOIN (SELECT * FROM S) AS t2
```

Table 23: *Relation R*

Table 24: *Relation S*

Table 25: *R X S*

r1	r2
1	2
3	4
5	6

s1
7
8

r1	r2	s1
1	2	7
1	2	8
3	4	7
3	4	8
5	6	7
5	6	8

Property	Value
Output degree	degree(R) + degree(S) — shared names qualified as R.col and S.col
Output cardinality	| R | × | S |

EXAMPLE 15: Cartesian Product

Produce all combinations of Employee and Department rows. Notice the resulting degree (columns) and cardinality (rows).

✅ Show Solution

$$\text{Employee} \times \text{Department}$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee × Department)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Employee) AS t1 CROSS JOIN (SELECT * FROM Department) AS t2
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee × Department)
```

Important

The raw Cartesian product generates **30 × 8 = 240 rows** but most are meaningless combinations. It is almost always followed by a σσ to keep only matching rows — that combination is precisely a **join**.

EXAMPLE 16: Column Name Conflict

Try to pair every employee with every other employee using a Cartesian product.

Show Solution

$$Employee \times Employee$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee × Employee)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee × Employee)
```

Why Employee × Employee Fails (Column Name Conflict)

When you write Employee × Employee, both copies of the table bring the **same 10 column names** (`Fname` , `Lname` , `Ssn` , ...).
A relation must have **unique column names** — there is no way to tell `Fname` (left copy) from `Fname` (right copy).

The correct approach: use ρ to **rename** one (or both) copies before forming the product:

$$\rho_{E1}(Employee) \times \rho_{E2}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
(ρ E1 (Employee) × ρ E2 (Employee))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(ρ E1 (Employee) × ρ E2 (Employee))
```

The general rule

every column in a relation must be uniquely identifiable. Whenever a Cartesian product would produce duplicate column names, use ρ to rename one operand before the product.

5.1 Joining Multiple Tables using Foreign Keys

When information is spread across multiple tables, we must combine (merge) them to retrieve the data we need.

- **Why Unconditioned Merges Fail:** Performing a Cartesian product (merging tables without a condition) pairs every row of the first table with every row of the second table, creating meaningless and illogical combinations.
- **Using Foreign Keys as Join Conditions:** To ensure we extract accurate and logical data, we must take advantage of foreign key relationships. Using the foreign key as a join condition (PK = FK) filters out invalid pairs, guaranteeing that only truly related records are merged.

EXAMPLE 17:

Retrieve all Female dependent names for each employee.

Show Solution

To retrieve the names of all dependents associated with female employees, you must combine the **Employee** relation and the **Dependent** relation using their foreign key relationship.

1. Perform the Cartesian Product:

$$\text{All Pairs} \leftarrow \text{Employee} \times \text{Dependent}$$

2. Apply the Selection (Filter for matching Foreign Key AND Female Gender):

$$\text{FemaleDependents} \leftarrow \sigma_{\text{Ssn}=\text{Essn} \wedge \text{Gender}=\text{'F'}}(\text{AllPairs})$$

3. Project the Desired Attribute:

$$\text{Result} \leftarrow \pi_{\text{Dependent_name}}(\text{FemaleDependents})$$

$$Emp = \pi_{Ssn,Fname,Lname}(Employee)$$
$$\text{AllPairs} \leftarrow \text{Emp} \times \text{Dependent}$$
$$\text{FemaleDependents} \leftarrow \sigma_{\text{Ssn=Essn} \wedge \text{Gender='F'}}(\text{AllPairs})$$
$$\text{Result} \leftarrow \pi_{\text{Dependent_name},Fname,Lname}(\text{FemaleDependents})$$

RELATIONAL ALGEBRA (RELAX)

```
Emp = π Ssn, Fname, Lname (Employee)
AllPairs = (Emp × Dependent)
FemaleDependents = σ Ssn = Essn ∧Gender = 'F' (AllPairs)
π Dependent_name, Fname, Lname (FemaleDependents)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤
SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
Emp = π Ssn, Fname, Lname (Employee)
AllPairs = (Emp × Dependent)
```

6 Joins

📖 DEFINITION 5: **Join Operation**

A join operation in relational algebra combines tuples (rows) from two separate relations (tables) based on a specific condition, acting fundamentally as a Cartesian product followed by a selection.

6.1 Theta Join (Cartesian Product + Filter)

📖 DEFINITION 6: **A theta-join (⋈θ)**

A theta-join ($\bowtie_\theta$) is a relational algebra operation that combines two relations based on any general comparison or conditional operator, rather than strictly equality. It is formally defined as performing a Cartesian product first and then immediately applying a selection condition (σ) to filter out unrelated combinations.

$$R \bowtie_\theta S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋈ [standard condition] S)
```

where:

- **General Comparison Operators:** Unlike an equi-join (which strictly uses equality `$=$`), a theta-join allows any standard condition operator ($\theta \in \{=, \neq, <, \leq, >, \geq\}$).
- **Formal Algebraic Equivalence:** Formally, a theta-join between two relations R and S is defined as performing a **Cartesian Product** first and then applying a **Selection** (σ) condition:

$$R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$$

Where θ represents the join condition (e.g., $R.A > S.B$ or $R.Ssn = S.Essn$).

Tip

A **Theta Join** ($\bowtie_{\theta}$) is simply a general form of an **inner join**.

- **Mechanism:** It combines two relations by pairing rows that satisfy a specified condition θ (e.g., $=, \neq, <, >, \leq, \geq$).
- **Inner Join Behavior:** Only tuples that evaluate to **true** under condition θ are included in the result set; unmatched tuples are discarded.
- **Equivalence:** A Theta Join is formally defined as a Cartesian product followed by a Selection:

$$\text{Relation}_1 \bowtie_{\theta} \text{Relation}_2 \equiv \sigma_{\theta}(\text{Relation}_1 \times \text{Relation}_2)$$

- An **equijoin** is simply a specific type of **theta join** where the condition relies strictly on the equality operator ($=$). While all equijoins are theta joins, not all theta joins are equijoins (as theta joins can use any comparison operator, such as $>$, $<$, or $\neq$).

EXAMPLE 18:

List each employee with their project number and hours, joining Employee and Works_On on Ssn = Essn.

Show Solution

$$\text{Employee} \bowtie_{\text{Ssn}=\text{Essn}} \text{Works_On}$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee ⋈(Ssn = Essn) Works_On)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Employee) AS t1 JOIN (SELECT * FROM Works_On) AS t2 ON Ssn = Essn
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee ⋈(Ssn = Essn) Works_On)
```

Why it is called an Equijoin here?

Because the specific condition chosen for θ is an equality comparison ($\mathrm{Ssn} = \mathrm{Essn}$), this theta join qualifies specifically as an **equijoin**.

6.2 Natural Join — ⋈

📖 DEFINITION 7: **Natural Join**

a **Natural Join (⋈)** is a derived relational algebra operation that automatically performs a join based on all attributes that share the same name in both relations.

Key Properties:

- **Implicit Equality Condition:** It matches rows where all shared attribute names have equal values (an implicit equijoin).
- **Duplicate Column Elimination:** Unlike a Cartesian product or theta join, duplicate common attributes are automatically removed—appearing only once in the resulting relation's schema.

Result Schema Rule

If relation R has attributes (A, B, C) and relation S has attributes (B, C, D), then the natural join $R \bowtie S$ results in a relation with attributes (A, B, C, D).

$$R \bowtie S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋈ S)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM R) AS t1 NATURAL JOIN (SELECT * FROM S) AS t2
```

Natural Join is Name-Driven

Natural join joins on **every** column that shares the same name in both tables.

- If PK and FK have **different names** → no common columns → result is a **Cartesian product** (all combinations, probably wrong)
- If you accidentally share **too many** column names → the join condition becomes overly restrictive
- Always verify which columns will be matched before using natural join

EXAMPLE 19: **Natural Join: When It Works Directly**

List each department with all its office locations.

Show Solution

- The `Department` table has `Dnumber` as its primary key.
- The `Dept_Locations` table has `Dnumber` as a foreign key.
- **Both columns share the exact same name** → natural join will automatically join on `Dnumber`.

$$Department \bowtie Dept_Locations$$

RELATIONAL ALGEBRA (RELAX)

```
(Department ⋈ Dept_Locations)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Department) AS t1 NATURAL JOIN (SELECT * FROM Dept_Locations) AS t2
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Department ⋈ Dept_Locations)
```

EXAMPLE 20: Natural Join: When Column Names Differ (Danger!)

Try to list each employee with the name of their department using natural join.

Show Solution

`Employee.Dno` is the FK referencing `Department.Dnumber` — but the column names are **different**.

Since there are **no columns with identical names** between Employee and Department, a natural join produces a **Cartesian product** — every employee paired with every department, regardless of which department they belong to.

$$Employee \bowtie Department$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee ⋈ Department)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee ⋈ Department)
```

Fix with Rename ρ: Force Natural Join

Strategy: Use ρ to rename `Dnumber` to `Dno` in Department, making the column name match.

$$Deps = \rho_{Dno \leftarrow Dnumber} Department$$
$$Employee \bowtie Deps$$

RELATIONAL ALGEBRA (RELAX)

```
Deps = ρ Dno←Dnumber (Department)
(Employee ⋈ Deps)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ | SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
Deps = ρ Dno←Dnumber (Department)
(Employee ⋈ Deps)
```

6.3 Outer Joins

Standard (inner) joins **discard unmatched rows**. Outer joins **preserve** them by filling missing values with NULL.

Operator	Symbol	Keeps unmatched from
Left outer join	R⟕S	R (left operand)
Right outer join	R⟖S	S (right operand)
Full outer join	R ⟗ S	Both R R and S S

6.3.1 Left Outer Join(R ⟕ S)

📖 DEFINITION 8: **Left Outer Join**

A **Left Outer Join** is an extended relational algebra operation that returns all matching tuples from both relations, plus all unmatched tuples from the left relation.

$$R ⟕ S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⟕ S)
```

Key Properties:

- **Preserves Left Relation Tuples:** Guarantees that every tuple from the left relation appears in the result at least once.
- `NULL` **Padding for Unmatched Rows:** If a tuple in R has no matching tuple in S based on the join condition, it is included in the output schema with `NULL` values for all attributes originating from S.
- **Schema / Output Columns:** The output schema contains all attributes from both R and S.

📝 EXAMPLE 21:

Retrieve all employees along with their corresponding department information, even if some employees do not belong to any department.

Show Solution

$$Employee \bowtie_{Dno=Dnumber} Department$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee ⋈(Dno = Dnumber) Department)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Employee) AS t1 LEFT JOIN (SELECT * FROM Department) AS t2 ON Dno = Dnumber
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee ⋈(Dno = Dnumber) Department)
```

6.3.2 Right Outer Join(R ⟕ S)

📖 DEFINITION 9: **Right Outer Join**

A **Right Outer Join** is an extended relational algebra operation that returns all matching tuples from both relations, plus all unmatched tuples from the right relation.

$$R \bowtie S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋈ S)
```

Key Properties:

- **Preserves Right Relation Tuples:** Guarantees that every tuple from the right relation appears in the result at least once.
- `NULL` **Padding for Unmatched Rows:** If a tuple in S has no matching tuple in R based on the join condition, it is included in the output schema with `NULL` values for all attributes originating from R.

- **Schema / Output Columns:** The output schema contains all attributes from both R and S.

EXAMPLE 22:

Retrieve all departments along with their corresponding employee information, even if some departments do not have any employees assigned to them.

Show Solution

$$Department \bowtie_{Dno=Dnumber} Employee$$

RELATIONAL ALGEBRA (RELAX)

```
(Department ⋈(Dno = Dnumber) Employee)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Department) AS t1 RIGHT JOIN (SELECT * FROM Employee) AS t2 ON Dno = Dnumber
```

Relational Algebra Calculator

▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Department ⋈(Dno = Dnumber) Employee)
```

6.3.3 Full Outer Join(R ⟗ S)

DEFINITION 10: **Full Outer Join**

A Full Outer Join is an extended relational algebra operation that combines the results of both a **Left Outer Join** and a **Right Outer Join**. It returns all matching tuples from both relations, along with all unmatched tuples from both the left and right relations.

$$R \bowtie S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⟗ S)
```

Key Properties:

- **Preserves All Tuples:** Guarantees that every tuple from both relations (R and S) appears in the result at least once
- Unmatched tuples from the left relation (R) are padded with `NULL` values for all attributes of the right relation (S).
- Unmatched tuples from the right relation (S) are padded with `NULL` values for all attributes of the left relation (R)
- **Schema / Output Columns:** The output schema contains all attributes from both relations R and S.

EXAMPLE 23:

List all employees, all project assignments, and all projects—including employees who are not assigned to any project, project assignment records without a matching employee, and projects that have zero employees assigned to them.

Show Solution

$$\text{Employee} \bowtie_{\text{Ssn}=\text{Essn}} \text{Works_On} \bowtie_{\text{Pno}=\text{Pnumber}} \text{Project}$$

RELATIONAL ALGEBRA (RELAX)

```
((Employee ⋈(Ssn = Essn) Works_On) ⋈(Pno = Pnumber) Project)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM (SELECT * FROM Employee) AS t1 FULL OUTER JOIN (SELECT * FROM
Works_On) AS t2 ON Ssn = Essn) AS t3 FULL OUTER JOIN (SELECT * FROM Project) AS t4 ON Pno = Pnumber
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
((Employee ⋈(Ssn = Essn) Works_On) ⋈(Pno = Pnumber) Project)
```

7 Extended Relational Algebra — Aggregation (γ)

Standard relational algebra operates on **sets** of tuples with no grouping or counting. Real queries often need summaries: "count employees per department", "average salary by gender", "maximum project hours". The **extended relational algebra** adds the **aggregate operator** γ (gamma).

$$\gamma_G; F(A) \rightarrow N\ (R)$$

RELATIONAL ALGEBRA (RELAX)

where:

- γ **(Gamma):** Represents the extended relational algebra aggregation operator.
- **G is grouping attribute** (equivalent to SQL `GROUP BY`).
- **F(A):** The aggregate function **F** applied to attribute **A**.
- **N**: alias/name to the computed result.
- R**:** The input relation/table or subquery.

Table 26: *Aggregate functions*

Function	Meaning
COUNT()	Count rows in each group
SUM(col)	Sum of values
AVG(col)	Average of values
MIN(col)	Minimum value
MAX(col)	Maximum value

EXAMPLE 24: Count Employees per Department

How many employees work in each department?

Show Solution

$$\gamma Dno; COUNT() \rightarrow NumEmp(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Dno;COUNT(*)→NumEmp (Employee)
```

SQL EQUIVALENT

```
SELECT Dno, COUNT(*) AS NumEmp FROM Employee GROUP BY Dno
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Dno;COUNT(*)→NumEmp (Employee)
```

EXAMPLE 25: Average Salary per Department

What is the average salary in each department?

Show Solution

$$\gamma Dno; AVG(Salary) \rightarrow AvgSalary(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Dno;AVG(Salary)→AvgSalary (Employee)
```

SQL EQUIVALENT

```
SELECT Dno, AVG(Salary) AS AvgSalary FROM Employee GROUP BY Dno
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Dno;AVG(Salary)→AvgSalary (Employee)
```

EXAMPLE 26: Multiple Aggregates: Max and Min Salary per Department

In each department, what is the highest and lowest salary?

Show Solution

$$\gamma Dno; MAX(Salary) \rightarrow MaxSal, MIN(Salary) \rightarrow MinSal(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Dno;MAX(Salary)→MaxSal,MIN(Salary)→MinSal (Employee)
```

SQL EQUIVALENT

```
SELECT Dno, MAX(Salary) AS MaxSal, MIN(Salary) AS MinSal FROM Employee GROUP BY Dno
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Dno;MAX(Salary)→MaxSal,MIN(Salary)→MinSal (Employee)
```

EXAMPLE 27: Aggregation with Filter: Departments Averaging Over $33,000

List department numbers where the average salary exceeds $33,000.

Show Solution

$$\sigma_{AvgSalary>33000}(\gamma Dno; AVG(Salary) \rightarrow AvgSalary(Employee))$$

RELATIONAL ALGEBRA (RELAX)

```
σ AvgSalary > 33000 (γ Dno;AVG(Salary)→AvgSalary (Employee))
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT Dno, AVG(Salary) AS AvgSalary FROM Employee GROUP BY Dno) AS t1 WHERE
AvgSalary > 33000
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ AvgSalary > 33000 (γ Dno;AVG(Salary)→AvgSalary (Employee))
```

EXAMPLE 28: Total Hours per Employee Across All Projects

For each employee, calculate the total hours worked across all their project assignments.

Show Solution

$$\gamma Essn; SUM(Hours) \rightarrow TotalHours(Works_On)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Essn;SUM(Hours)→TotalHours (Works_On)
```

SQL EQUIVALENT

```
SELECT Essn, SUM(Hours) AS TotalHours FROM Works_On GROUP BY Essn
```

Relational Algebra Calculator

▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Essn;SUM(Hours)→TotalHours (Works_On)
```

8 Exercises

8.1 University Database

8.1.1 University Database Schema Overview

The **University Database** schema is designed to model core academic and administrative operations within a higher education institution. It manages information across six key entities: students, faculty, departments, courses, course enrollments, and teaching assignments.

8.1.1.1 Entity Descriptions & Key Relationships

- `Department`
  - **Attributes:** `Did` (Primary Key), `Dname`, `Chair_id` (Foreign Key → `Faculty.Fid`), `Building`.
  - **Description:** Represents academic departments within the university. Each department operates in a specific building and is chaired by a faculty member.
- `Faculty`
  - **Attributes:** `Fid` (Primary Key), `Fname`, `Lname`, `Rank`, `Salary`, `Dept_id` (Foreign Key → `Department.Did`).
  - **Description:** Represents academic staff members. Each faculty member belongs to a specific department and holds an academic rank (e.g., Professor, AssocProf, AssistProf).
- `Student`
  - **Attributes:** `Sid` (Primary Key), `Fname`, `Lname`, `Email`, `Major`, `GPA`, `Year`, `Advisor_id` (Foreign Key → `Faculty.Fid`).
  - **Description:** Stores student demographic and academic performance data. Each student is assigned a faculty member as their academic advisor.
- `Course`
  - **Attributes:** `Cid` (Primary Key), `Cname`, `Credits`, `Dept_id` (Foreign Key → `Department.Did`).
  - **Description:** Defines subjects offered by the university. Every course belongs to an offering department and carries a specified number of academic credit units.
- `Enrollment` *(Junction Table for Student–Course $M:N$ Relationship)*
  - **Attributes:** `Sid` (Foreign Key → `Student.Sid`), `Cid` (Foreign Key → `Course.Cid`), `Semester`, `Grade`.
  - **Composite Primary Key:** (`Sid`, `Cid`, `Semester`)
  - **Description:** Tracks student course registrations and academic performance (letter grades) across different academic terms.
- `Teaches` *(Junction Table for Faculty–Course $M:N$ Relationship)*
  - **Attributes:** `Fid` (Foreign Key → `Faculty.Fid`), `Cid` (Foreign Key → `Course.Cid`), `Semester`, `Room`.
  - **Composite Primary Key:** (`Fid`, `Cid`, `Semester`)
  - **Description:** Maps teaching responsibilities, indicating which faculty member teaches a specific course section, semester, and assigned classroom location.

8.1.1.2 Schema Reference

- **Student** $(\text{Sid}, \text{Fname}, \text{Lname}, \text{Email}, \text{Major}, \text{GPA}, \text{Year}, \text{Advisor_id})$
- **Faculty** $(\text{Fid}, \text{Fname}, \text{Lname}, \text{Rank}, \text{Salary}, \text{Dept_id})$

- **Department** (Did, Dname, Chair_id, Building)
- **Course** (Cid, Cname, Credits, Dept_id)
- **Enrollment** (Sid, Cid, Semester, Grade)
- **Teaches** (Fid, Cid, Semester, Room)

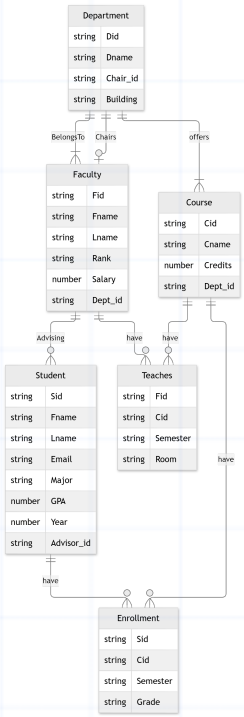


Diagram 57: University Database

8.1.2 Exercises

🖍️ EXERCISE 1: Selection & Projection (Basic)

Find the first name, last name, and GPA of all **Computer Science ('CS')** students who have a GPA higher than **3.5**.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

✅ Show Solution

$$\pi_{\text{Fname,Lname,GPA}}\left(\sigma_{\text{Major='CS'} \wedge \text{GPA}>3.5}(\text{Student})\right)$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Lname, GPA (σ Major = 'CS' ∧GPA > 3.5 (Student))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Lname, GPA (σ Major = 'CS' ∧GPA > 3.5 (Student))
```

🏏 EXERCISE 2: Theta Join / Equijoin

Find the full names ($\text{Fname}, \text{Lname}$) of all students along with the last name (Lname) of their academic advisor.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

✅ Show Solution

$$\pi_{\text{Student.Fname,Student.Lname,Faculty.Lname}}\left(\text{Student} \bowtie_{\text{Advisor_id=Fid}} \text{Faculty}\right)$$

RELATIONAL ALGEBRA (RELAX)

```
π Student.Fname, Student.Lname, Faculty.Lname ((Student ⋈(Advisor_id = Fid) Faculty))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Student.Fname, Student.Lname, Faculty.Lname ((Student ⋈(Advisor_id = Fid) Faculty))
```

EXERCISE 3: Natural Join & Multi-Table Join

List the names of all courses (Cname) offered by the 'Computer Science' department, including the department name (Dname).

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

Show Solution

$$\pi_{\text{Cname,Dname}} (\text{Course} \bowtie \sigma_{\text{Dname='Computer Science'}} (\text{Department}))$$

RELATIONAL ALGEBRA (RELAX)

```
π Cname, Dname ((Course ⋈ σ Dname = 'Computer Science' (Department)))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Cname, Dname ((Course ⋈ σ Dname = 'Computer Science' (Department)))
```

🖍️ EXERCISE 4: Outer Joins

List all faculty members ($\mathrm{Fname}, \mathrm{Lname}$) and the course IDs ($\mathrm{Cid}$) they teach, including faculty members who are not currently teaching any courses.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

✅ Show Solution

$$\pi_{\mathrm{Fname},\mathrm{Lname},\mathrm{Cid}}\left(\mathrm{Faculty} \bowtie_{\mathrm{Faculty.Fid}=\mathrm{Teaches.Fid}} \mathrm{Teaches}\right)$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Lname, Cid ((Faculty ⋈(Faculty.Fid = Teaches.Fid) Teaches))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Lname, Cid ((Faculty ⋈(Faculty.Fid = Teaches.Fid) Teaches))
```

EXERCISE 5: Aggregation

Find the average GPA and the total number of students for each Major.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

Show Solution

$$\gamma_{\text{Major};\text{AVG(GPA)}\rightarrow\text{AvgGPA},\ \text{COUNT(Sid)}\rightarrow\text{TotalStudents}}(\text{Student})$$

RELATIONAL ALGEBRA (RELAX)

```
γ Major;AVG(GPA)→AvgGPA, COUNT(Sid)→TotalStudents (Student)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - | JOIN ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Major;AVG(GPA)→AvgGPA, COUNT(Sid)→TotalStudents (Student)
```

Chapter5 Normalization and Functional Dependencies

1 A Single Table Approach

Imagine a university keeps all student enrollment data in a **single table** named `Enrollment`:

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Dr. Sameer	A
101	Ahmed	MATH201	Calculus II	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C

> **The Problem**
>
> This looks simple — but it contains serious problems known as **anomalies**.

2 Anomalies in Unnormalized Tables

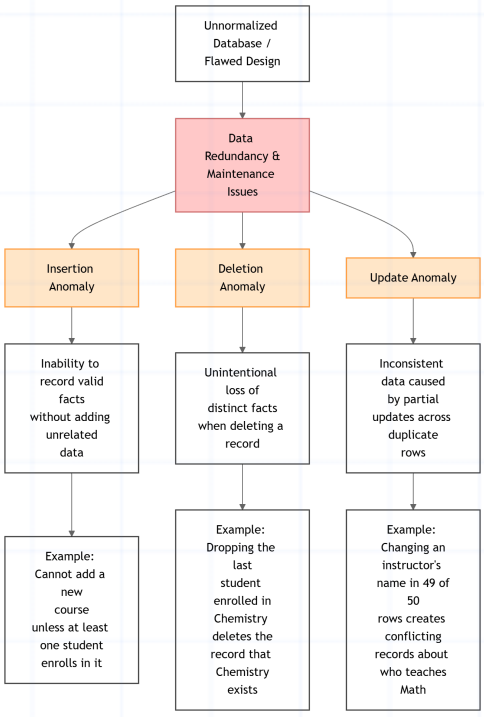


Diagram 58: Breaking down database anomalies caused by unnormalized design and data redundancy.

2.1 Insertion Anomaly

Problem: Cannot insert certain data without other unrelated data.

Suppose a new course `CS202 – Data Structures` is added, but no student has enrolled yet. You cannot insert `CourseID = CS202` and `CourseTitle = Data Structures` unless you also have a `StudentID`. The course information depends on a student being enrolled.

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Dr. Layla	A
101	Ahmed	MATH201	Calculus II	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C
Null	**Null**	**CS202**	**Data Structures**	**Null**	**Null**

2.2 Deletion Anomaly

Problem: Deleting data unintentionally removes other important data.

If Ahmed drops `MATH201`, and he was the only student in that course, deleting his record also deletes the course info (`MATH201`, `Calculus II`, `Dr. Ibrahem`).

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Dr. Layla	A
102	Sami	CS101	Intro to CS	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C

2.3 Update Anomaly

Problem: Redundant data leads to inconsistencies during updates.

Dr. Layla teaches `CS101`. Her name appears in **every row** where `CS101` is offered. If Dr. Layla changes her name (e.g., to *Prof. Layla*), you must update every row with `CS101`. If you miss one row, data becomes inconsistent.

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	**Prof. Layla**	A
101	Ahmed	MATH201	Calculus II	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	**Dr. Layla** ← inconsistent!	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C

2.4 Solution: Normalization

Normalization is the process of decomposing a table into smaller, well-structured tables to:

- **Eliminate redundancy** — the same data isn't stored multiple times unnecessarily
- **Avoid anomalies** — relationships between tables are clear and logical
- **Ensure data integrity** — queries run faster and are easier to maintain

We split the `Enrollment` table into three tables:

Table 27: *Students*

StudentID	StudentName
101	Ahmed
102	Sami

Table 28: *Courses*

CourseID	CourseTitle	Instruct
CS101	Intro to CS	Dr. Layla
MATH201	Calculus II	Dr. Ibrahem
PHY301	Quantum Physics	Dr. Amani

Table 29: *Enrollments*

StudentID	CourseID	Grade
101	CS101	A
101	MATH201	B+
102	CS101	A-
102	PHY301	C

Now each anomaly disappears:

- **Adding a new course?** Just insert into `Courses` — no student needed.
- **A student drops a course?** Delete only from `Enrollments` — course data stays intact.
- **Instructor name changes?** Update one row in `Courses` — done.

> **Note**
>
> When you perform **ER-to-Relational Mapping**, following standard mapping rules automatically structures your tables to be in **at least 3NF** in the vast majority of cases:
>
> - **Entities → Tables:** Each entity type becomes a separate relation, giving entity attributes a clear home around a chosen entity key.
> - **Relationships → Foreign Keys or Junction Tables:** Binary 1:N relationships use foreign keys, and M:N relationships become separate junction tables (e.g., `Enrollments`), which naturally isolates composite key attributes like `Grade`.
> - **Attributes → Atomic Fields:** Multi-valued attributes are broken out into separate tables, satisfying 1NF.

3 Functional Dependencies

Before we can normalize, we need a precise language for describing what determines what in a relation. That language is **Functional Dependencies (FDs)**.

In database design, FDs provide the formal mathematical foundation used to capture real-world business rules about how attributes relate to one another.

Think of an FD like a deterministic mathematical function:

$$y = f(x) \implies \text{knowing } x \text{ always gives you exactly one } y$$

The database equivalent is written $X \to Y$ and reads:

> **"For every valid instance of X, there is a unique value of Y in the table."**

In other words, for any two tuples (rows) t_1 and t_2 in a relation state $r(R)$:

$$\text{if } t_1[X] = t_2[X], \text{ then } t_1[Y] = t_2[Y]$$

If two rows share the same X value, they **must** share the same Y value—no exceptions.

> 📖 DEFINITION 1: **Functional Dependency**
>
> A Functional Dependency X→Y (read as "X functionally determines Y") holds in a relation R if, for **any** valid state of R, whenever two tuples agree on the attributes of X, they **must** also agree on the attributes of Y.
>
> - **Determinant:** The left-hand side (X).
> - **Dependent:** The right-hand side (Y).

3.1 Why FDs Are Essential

Without functional dependencies, you cannot formally define keys or prove that a design is free of data anomalies. FDs drive the entire theoretical foundation of relational database design:

1. **Functional Dependencies ($X \to Y$):** Formally define attribute-level constraints based on domain semantics (e.g., $\text{StudentID} \to \text{StudentName}$).
2. **Keys (Super & Candidate Keys):** Derived directly from FDs by computing attribute set closures (X^+). A set X is a superkey if $X^+ = R$, and a candidate key if X is also minimal.
3. **Normal Forms ($1\text{NF} \to 2\text{NF} \to 3\text{NF} \to \text{BCNF}$):** Evaluated strictly by testing conditions on FDs and keys:
   - **2NF:** Eliminates **partial dependencies** ($\text{Non-Prime} \subset \text{Candidate Key}$).
   - **3NF:** Eliminates **transitive dependencies** ($\text{Non-Prime} \to \text{Non-Prime}$).
   - **BCNF:** Ensures that for every non-trivial FD $X \to Y$, X must be a **superkey**.

> 📝 EXAMPLE 1: **Attribute-Level Determinism**
>
> A company assigns each employee a unique employee number (EmpNo) or Social Security Number (SSN).
>
> $$\text{SSN} \to \text{Name}$$

- **Meaning:** If you know the value of SSN, you can uniquely identify the Name of that employee.
- **Reading the Notation:** The expression $\mathrm{SSN} \rightarrow \mathrm{Name}$ is read as:

> "SSN *determines* Name" or "SSN *functionally implies* Name."

Does the Reverse Hold ($\mathrm{Name} \rightarrow \mathrm{SSN}$)?

$$\mathrm{Name} \rightarrow \mathrm{SSN} \quad \times (\mathrm{False})$$

No. It would violate database consistency if two distinct employees happened to share the same name (e.g., two employees named "Ahmed"). Because their SSN values would be different, Name cannot uniquely determine SSN.

FDs Come from Domain Knowledge, Not Data Snapshots

Functional dependencies are constraints on the **relation schema** itself, derived from real-world business rules—**not** from a temporary snapshot of data:

- × **Do not infer an FD** just because it happens to hold in the current rows of a table.
- ✓ **An FD is valid** only if it holds for *every possible valid state* the database could ever enter.

3.2 Discovering FDs from Stakeholders

Functional dependencies are not invented by database designers; they are discovered by interviewing domain experts (stakeholders) who own and understand the real-world business rules.

EXAMPLE 2: Stakeholder Interview & FD Extraction

By asking targeted questions to the **University Registrar**, we extract the core business constraints and translate them directly into mathematical Functional Dependencies:

Stakeholder Question	Registrar's Business Rule	Extracted FD Notation
"Can a student ID belong to multiple students?"	*"No, each student has a unique ID and a single official name."*	$\text{StudentID} \rightarrow \text{StudentName}$
"Can a course ID have different course titles?"	*"No, a course code like CS101 always has a single fixed title."*	$\text{CourseID} \rightarrow \text{CourseTitle}$
"Can multiple instructors teach the same course?"	*"Yes, but in any given term, each course is taught by only one instructor."*	$(\text{CourseID}, \text{Term}) \rightarrow \text{Instructor}$
"What determines the final grade assigned to a student?"	*"A grade is assigned to a specific student for a specific course."*	$(\text{StudentID}, \text{CourseID}) \rightarrow \text{Grade}$

Consider the following sample table containing student enrollment records:

StudentID	StudentName	CourseID	CourseTitle	Term	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Fall 2026	Dr. Layla	A
101	Ahmed	MATH201	Calculus II	Fall 2026	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	Fall 2026	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Spring 2026	Dr. Amani	C
103	Tariq	CS101	Intro to CS	Spring 2026	Prof. Omar	B

- **Data is Just an Instance:** The table rows above illustrate the rules, but the rules (FDs) are defined by domain logic, not the data snapshot.
- **Formal Validation:**
  - $\text{StudentID} \rightarrow \text{StudentName}$ holds because whenever two rows share the same StudentID (e.g., 101), they share the exact same StudentName ("Ahmed").
  - $(\text{CourseID}, \text{Term}) \rightarrow \text{Instructor}$ holds because $(\text{CS101}, \text{Fall 2026})$ uniquely determines "Dr. Layla", while $(\text{CS101}, \text{Spring 2026})$ uniquely determines "Prof. Omar".

3.2.1 FD Validation

An Functional Dependency (FD) $X \rightarrow Y$ is **valid** (holds) in a relation state $r(R)$ if and only if for every pair of tuples (rows) t_1 and t_2 in $r(R)$ that agree on the attribute set X, they must also agree on the attribute set Y.

Formally:

$$X \rightarrow Y \iff \forall t_1, t_2 \in r(R), \quad \text{if } t_1[X] = t_2[X], \text{ then } t_1[Y] = t_2[Y]$$

EXAMPLE 3: Validating FDs on a Relation Instance

Consider the relation instance $R(A, B, C, D)$:

A	B	C	D
1	1	2	3
1	2	2	3
1	3	2	3
2	4	5	6
5	6	7	8

Which of the following FDs are satisfied by this relation?

1. $A \to B$
2. $A \to CD$
3. $AB \to CD$
4. $C \to D$
5. $B \to A$
6. $BD \to AC$
7. $AD \to BC$
8. $D \to B$
9. $D \to C$
10. $C \to A$

Show Solution

1. $A \to B$

   $\times$ Does not hold. For $A = 1$, there are three distinct values of B $(1, 2, 3)$.

2. $A \to CD$

   ✓ Holds. For each value of A, there is a unique (C, D) pair:

   - $A = 1 \implies (2, 3)$
   - $A = 2 \implies (5, 6)$
   - $A = 5 \implies (7, 8)$

3. $AB \to CD$

   ✓ Holds. Every (A, B) combination is distinct across all rows, so each naturally maps to a unique (C, D) pair.

4. $C \to D$

   ✓ Holds.

- $C = 2 \implies D = 3$
- $C = 5 \implies D = 6$
- $C = 7 \implies D = 8$

5. $B \to A$

   ✓ Holds. Every value of B is unique in this instance, mapping to a single A value.

6. $BD \to AC$

   ✓ Holds. Each (B, D) pair appears at most once in the table.

7. $AD \to BC$

   × Does not hold. For $(A, D) = (1, 3)$, there are three distinct (B, C) pairs: $(1, 2)$, $(2, 2)$, and $(3, 2)$.

8. $D \to B$

   × Does not hold. For $D = 3$, B takes values $1, 2$, and 3.

9. $D \to C$

   ✓ Holds.

   - $D = 3 \implies C = 2$
   - $D = 6 \implies C = 5$
   - $D = 8 \implies C = 7$

10. $C \to A$

    ✓ Holds. Each C value maps to a unique A value ($2 \implies 1, 5 \implies 2, 7 \implies 5$).

> ⚠️ **Important Distinction:**
>
> While dependencies like $B \to A$ and $C \to A$ hold in this *particular sample instance*, they cannot be declared general schema FDs unless supported by true real-world business rules. A single counterexample disproves an FD, but an instance satisfying an FD does not prove it holds universally for all future data.

3.3 Keys, Prime, and Non-Prime Attributes

3.3.1 Review of Keys

> 📖 DEFINITION 2: **Key Attribute**
>
> An attribute that uniquely identifies each entity instance.

> 📖 DEFINITION 3: **Super Key**

A set of one or more attributes (columns) that uniquely identifies a row (tuple) within a table. Any combination of columns that prevents duplicate rows is a super key.

📖 DEFINITION 4: **Candidate Key**

A **Candidate Key** is a *minimal* Super Key. It has the exact same unique identification property as a Super Key, but it contains **no redundant attributes**. If you remove any attribute from a Candidate Key, it ceases to be a key.

📝 EXAMPLE 4: **Candidate Keys vs. Super Keys**

Suppose we have a relation $R(\text{SSN}, \text{Name}, \text{Address}, \text{Phone})$ with the functional dependency:

$$\text{SSN} \rightarrow \{\text{Name}, \text{Address}, \text{Phone}\}$$

Key Classifications

1. **Candidate Key:** $\{\text{SSN}\}$
   - **Closure:** $\{\text{SSN}\}^+ = \{\text{SSN}, \text{Name}, \text{Address}, \text{Phone}\} = R$ (it uniquely identifies all attributes in the relation, so it is a **superkey**).
   - **Minimality:** No proper subset of $\{\text{SSN}\}$ can uniquely identify the entire relation. Therefore, $\{\text{SSN}\}$ is a **candidate key**.
2. **Superkey (Not a Candidate Key):** $\{\text{SSN}, \text{Name}\}$
   - **Closure:** $\{\text{SSN}, \text{Name}\}^+ = \{\text{SSN}, \text{Name}, \text{Address}, \text{Phone}\} = R$ (it uniquely identifies all attributes, so it is a **superkey**).
   - **Minimality Check:** It contains a proper subset, $\{\text{SSN}\}$, which is already a superkey on its own. Because Name is redundant for uniqueness, $\{\text{SSN}, \text{Name}\}$ fails the minimality requirement and is **not a candidate key**.

Core Concept Breakdown

$$\text{Candidate Key} = \text{Superkey} + \text{Minimality}$$

- **Superkey:** Any attribute set X whose closure covers the entire relation schema ($X^+ = R$).
- **Candidate Key:** A minimal superkey—removing any attribute from X causes $X^+ \neq R$.

3.3.2 Prime vs. Non-Prime Attributes

This distinction is critical for 2NF and 3NF normalization.

- **Prime Attribute:** An attribute that is a member of **ANY** Candidate Key.
- **Non-Prime Attribute:** An attribute that is **NOT** a member of any Candidate Key.

📝 EXAMPLE 5: **Candidate Keys & Attribute Classification**

For a relation $R(A, B, C, D)$ with candidate keys $CK_1 = \{A, B\}$ and $CK_2 = \{B, C\}$:

- **Prime Attributes:** $\{A, B, C\}$

  *An attribute is prime if it is a member of **at least one** candidate key.*

  - $A \in CK_1$
  - $B \in CK_1$ and $B \in CK_2$
  - $C \in CK_2$

- **Non-Prime Attribute:** $\{D\}$

  *An attribute is non-prime if it does **not** belong to any candidate key.*

3.4 Deriving Keys and Functional Dependencies

In database design, we are rarely handed all functional dependencies in their final form. We typically start with a set of known business rules F and must derive implied dependencies to analyze schema properties, verify rules, and discover candidate keys.

3.4.1 Armstrong's Axioms (The Inference Rules)

To derive new, sound, and complete functional dependencies from a given set F, we use **Armstrong's Axioms**. The set of all possible FDs that can be logically inferred from F is called the **closure of** F, denoted as F^+.

3.4.1.1 Primary Axioms

- **Reflexivity:** If $Y \subseteq X$, then $X \to Y$.

  (A set of attributes always determines any of its subsets.)

- **Augmentation:** If $X \to Y$, then $XZ \to YZ$ for any attribute set Z.

  (Adding the same attributes to both sides preserves the dependency.)

- **Transitivity:** If $X \to Y$ and $Y \to Z$, then $X \to Z$.

  (Functional determination propagates across intermediate attributes.)

📝 EXAMPLE 6:

Imagine a university table tracking student enrollments with columns: `StudentID`, `CourseID`, and `Semester`.

1. **Given Set (X):**
   - Let $X = \{\text{StudentID}, \text{CourseID}, \text{Semester}\}$
2. **Choose a Subset ($Y \subseteq X$):**

- Let $Y = \{\text{StudentID}\}$

3. **Result ($X \to Y$):**

- Since Y is a subset of X, we get:

$$\{\text{StudentID}, \text{CourseID}, \text{Semester}\} \to \{\text{StudentID}\}$$

Intuition: If you already know the specific combination of a student, course, and semester, you inherently know the student's ID. Knowing the whole collection of attributes guarantees you know any individual part (or combination of parts) within that collection.

📝 EXAMPLE 7:

1. **Given Functional Dependency ($X \to Y$):**

- Knowing a product's barcode (X) determines its unit price (Y).
- $\text{Barcode} \to \text{Price}$

2. **Add Attribute Z to Both Sides ($XZ \to YZ$):**

- Let $Z = \{\text{StoreLocation}\}$.
- Augmenting both sides with the store location gives:

$$\{\text{Barcode}, \text{StoreLocation}\} \to \{\text{Price}, \text{StoreLocation}\}$$

Intuition: If a barcode determines a price, then knowing the barcode *and* the store location will determine the price *and* that same store location.

📝 EXAMPLE 8:

Imagine a corporate database tracking employee assignments with columns: `EmployeeID`, `DeptID`, and `BuildingNo`.

1. **Given Functional Dependencies:**

- Knowing an employee's ID determines their department ID ($X \to Y$):

$$\text{EmployeeID} \to \text{DeptID}$$

- Knowing the department ID determines the building where that department is located ($Y \to Z$):

$$\text{DeptID} \to \text{BuildingNo}$$

2. **Result ($X \to Z$):**

- By transitivity, we can chain these together to conclude that knowing the employee's ID indirectly determines their building:

$$\text{EmployeeID} \to \text{BuildingNo}$$

Intuition: If X leads to Y, and Y leads to Z, then X transitively leads to Z because Y acts as a bridge.

3.4.1.2 Secondary (Derived) Rules

From the three primary axioms, we derive practical helper rules:

- **Union (Additive):** If $X \to Y$ and $X \to Z$, then $X \to YZ$.
- **Decomposition:** If $X \to YZ$, then $X \to Y$ and $X \to Z$.
- **Pseudotransitivity:** If $X \to Y$ and $WY \to Z$, then $WX \to Z$.

📝 EXAMPLE 9: **Union (Additive) Rule**

Given Functional Dependencies:

- Knowing an `EmployeeID` determines their `Name`:

  $$\text{EmployeeID} \to \text{Name}$$

- Knowing the same `EmployeeID` determines their `Email`:

  $$\text{EmployeeID} \to \text{Email}$$

- **Result ($X \to YZ$):**
  - By the Union rule, knowing the `EmployeeID` allows you to determine both together:

    $$\text{EmployeeID} \to \{\text{Name}, \text{Email}\}$$

Intuition: If one key can look up your name and the same key can look up your email, you can group them into a single lookup for both pieces of information.

📝 EXAMPLE 10: **Pseudotransitivity Rule**

Given Functional Dependencies:

- Knowing a `StudentID` (X) determines their current `Major` (Y):

  $$\text{StudentID} \to \text{Major}$$

- Knowing a `Major` (Y) *combined with* a specific `CourseLevel` (W) determines the maximum `AllowedCredits` (Z):

  $$\{\text{Major}, \text{CourseLevel}\} \to \text{AllowedCredits}$$

- **Result ($WX \to Z$):**
  - By Pseudotransitivity, we can substitute `Major` with `StudentID`, meaning knowing the student's ID *and* the course level gives you the allowed credits:

    $$\{\text{StudentID}, \text{CourseLevel}\} \to \text{AllowedCredits}$$

Intuition: Since X provides Y, you can feed X into the second rule wherever Y is required, bridging X and W together to reach Z.

3.4.2 Practical Tips for Spotting Keys Quickly

Before running any algorithm, experienced database designers use these **visual heuristics** to immediately narrow down the search for candidate keys. These tips come from analyzing the left-hand side (LHS) and right-hand side (RHS) of the given FDs.

Tip 1: The "Must-Have" Rule (Never on RHS)

Any attribute that **never appears on the right-hand side** of *any* FD **must** be part of **every** Candidate Key. *Why?* Because there is no way to derive this attribute from any other attribute. The only way it can appear in a closure is if you include it in the starting set.

Tip 2: The "Never a Key" Rule (Only on RHS)

Any attribute that **only appears on the right-hand side** (and never on the left-hand side) of any FD can **never** be part of any Candidate Key. *Why?* Because this attribute is always "determined by" something else. It adds no determining power. Including it in a key would make the key non-minimal.

Tip 3: The "Both Sides" Rule

Attributes that appear on **both** the LHS and RHS of various FDs are the "candidate" attributes. They *might* be part of a candidate key, depending on closure computation.

Tip 4: Single-Attribute Quick Check

Before trying combinations, always compute the closure of each individual "Must-Have" attribute and each "Both Sides" attribute. If any single attribute's closure equals R, that attribute alone is a Candidate Key.

3.4.3 The Attribute Closure Algorithm

Computing the full set of all dependencies F^+ can be exponentially large. To determine whether a specific set of attributes X can uniquely identify the entire table—or to test if a specific FD $X \to Y$ holds—we compute the **Attribute Closure** of X, denoted as X^+.

📖 DEFINITION 5: **The attribute closure**

The attribute closure X^+ is the set of all attributes in R that are functionally determined by X under the given set of dependencies F.

Algorithm: Computing Attribute Closure

1. **Initialization:** Set $X^+ = X$.
2. **Iterative Expansion:** Repeatedly search F for an FD $A \to B$ such that $A \subseteq X^+$ and $B \nsubseteq X^+$. If found, set $X^+ = X^+ \cup B$.
3. **Termination:** Repeat Step 2 until a full pass over F yields no new attributes (a fixpoint is reached).

EXAMPLE 11: **Basic Step-by-Step Expansion**

Given:

Relation Schema: $R(A, B, C, D, E, F)$

Functional Dependency Set F:

1. $A \to B$
2. $\{B, C\} \to D$
3. $D \to E$
4. $E \to F$

Compute the attribute closure of $\{A, C\}$, denoted as $\{A, C\}^+$.

Show Solution

- **Initialization:**

$$\{A, C\}^+ = \{A, C\}$$

- **Iteration 1:**
  - Check $A \to B$: LHS $A \subseteq \{A, C\}^+$. Add B.

$$\{A, C\}^+ = \{A, B, C\}$$

  - Check $\{B, C\} \to D$: LHS $\{B, C\} \subseteq \{A, B, C\}^+$. Add D.

$$\{A, C\}^+ = \{A, B, C, D\}$$

  - Check $D \to E$: LHS $D \subseteq \{A, B, C, D\}^+$. Add E.

$$\{A, C\}^+ = \{A, B, C, D, E\}$$

  - Check $E \to F$: LHS $E \subseteq \{A, B, C, D, E\}^+$. Add F.

$$\{A, C\}^+ = \{A, B, C, D, E, F\}$$

Termination:

$\{A, C\}^+ = \{A, B, C, D, E, F\} = R$.

> **Conclusion:** Since $\{A, C\}^+$ contains all attributes of R, $\{A, C\}$ is a **superkey**. Because neither $A^+ = \{A, B\}$ nor $C^+ = \{C\}$ covers R, $\{A, C\}$ is also a **candidate key**.

📝 EXAMPLE 12: Partial Closure & Unused Dependencies

Given:

Relation Schema: $R(A, B, C, D, E)$

Functional Dependency Set F:

1. $A \to B$
2. $B \to C$
3. $\{C, D\} \to E$
4. $E \to A$

Compute:

- The attribute closure of $\{A, D\}$, denoted as $\{A, D\}^+$.
- The attribute closure of $\{A, C\}$, denoted as $\{A, C\}^+$.

✅ Show Solution

Task: Compute the attribute closure of $\{A, D\}$, denoted as $\{A, D\}^+$.

Trace:

- **Initialization:**

$$\{A, D\}^+ = \{A, D\}$$

- **Iteration 1:**
  - Check $A \to B$: LHS $A \subseteq \{A, D\}^+$. Add B.

$$\{A, D\}^+ = \{A, B, D\}$$

  - Check $B \to C$: LHS $B \subseteq \{A, B, D\}^+$. Add C.

$$\{A, D\}^+ = \{A, B, C, D\}$$

  - Check $\{C, D\} \to E$: LHS $\{C, D\} \subseteq \{A, B, C, D\}^+$. Add E.

$$\{A, D\}^+ = \{A, B, C, D, E\}$$

- **Termination:**

$\{A, D\}^+ = \{A, B, C, D, E\} = R$.

Task: Compute the attribute closure of $\{A, C\}$, denoted as $\{A, C\}^+$.

Trace:

- **Initialization:**

$$\{A, C\}^+ = \{A, C\}$$

- **Iteration 1:**
  - Check $A \to B$: LHS $A \subseteq \{A, C\}^+$. Add B.

$$\{A, C\}^+ = \{A, B, C\}$$

- Check $B \to C$: C is already in the closure. No change.
- Check $\{C, D\} \to E$: LHS $\{C, D\} \not\subseteq \{A, B, C\}^+$ (missing D). Cannot apply.
- Check $E \to A$: LHS $E \not\subseteq \{A, B, C\}^+$. Cannot apply.

- **Iteration 2:**

  No new attributes can be added. Pass completes without changes.

- **Termination:**

$$\{A, C\}^+ = \{A, B, C\} \neq R$$

> **Conclusion:** $\{A, C\}$ cannot determine D or E, so it is **not** a superkey.

3.4.4 The Complete Key-Finding Algorithm

This is the systematic procedure to find all Candidate Keys, classify Prime and Non-Prime attributes, and enumerate all Super Keys.

Step 1 — Classify all attributes into three groups:

- **Group M (Must-Have):** Attributes that never appear on the RHS of any FD. These must belong to every Candidate Key.
- **Group N (Never-a-Key):** Attributes that only appear on the RHS (never on the LHS). These can never be part of any Candidate Key.
- **Group B (Both-Sides):** Attributes that appear on both LHS and RHS across different FDs.

Step 2 — Compute the closure of M:

- Compute M^+ using the attribute closure algorithm.
- If $M^+ = R$, then M is the only Candidate Key. **Stop.**
- If $M^+ \neq R$, proceed to Step 3.

Step 3 — Find Candidate Keys by extending M:

1. Systematically add attributes from Group B to M, one at a time, then in pairs, and so on.
2. For each combination, compute the attribute closure.
3. If the closure equals R (i.e., $X^+ = R$), you have found a **Super Key**.
4. **Check Minimality:** Test each attribute in your Super Key by trying to remove it. If removing any attribute still results in a closure of R, then the attribute was redundant, meaning the set is *not* minimal and therefore **not a Candidate Key**.
5. *Never* add attributes from Group N, as they cannot help form a minimal key.

Step 4 — Identify Prime and Non-Prime attributes:

- **Prime Attribute:** Any attribute that appears in *at least one* Candidate Key.
- **Non-Prime Attribute:** Any attribute that appears in *no* Candidate Key.

Step 5 — Enumerate all Super Keys (if needed):

- Every Candidate Key is automatically a Super Key.
- Every superset of a Candidate Key is also a Super Key.
- **The Complete Set of Super Keys:** All possible unions of Candidate Keys combined with any combination of Non-Prime attributes (along with combinations with each other).

EXAMPLE 13: Full Worked Example

Relation $R(A, B, C, D, E)$ with the set of functional dependencies:

$$F = \{A \to B,\ B \to A,\ AC \to D,\ D \to E\}$$

Find all Candidate Keys, classify Prime and Non-Prime attributes, and list all Super Keys.

✅ Show Solution

Step 1: Classify Attributes

First, analyze where each attribute appears across the left-hand side (LHS) and right-hand side (RHS) of the FDs:

Attribute	Appears on LHS?	Appears on RHS?	Group
A	Yes ($A \to B$)	Yes ($B \to A$)	**Group B** (Both-Sides)
B	Yes ($B \to A$)	Yes ($A \to B$)	**Group B** (Both-Sides)
C	Yes ($AC \to D$)	**No**	**Group M** (Must-Have)
D	Yes ($D \to E$)	Yes ($AC \to D$)	**Group B** (Both-Sides)
E	**No**	Yes ($D \to E$)	**Group N** (Never-a-Key)

Immediate Conclusions from Heuristics:

1. C must be in every Candidate Key (it never appears on the RHS).
2. E can never be in any Candidate Key (it only appears on the RHS).
3. We only need to test combinations of C with attributes from Group B: $\{A, B, D\}$.

Step 2: Compute the Closure of $M = \{C\}$

Iteration	Current X+	FD Applied	New X+
Start	$\{C\}$	—	$\{C\}$
1	$\{C\}$	No FD has LHS $\subseteq \{C\}$	$\{C\}$

- **Result:** $\{C\}^+ = \{C\} \neq R$. Since the closure does not cover the entire relation, we must extend M with attributes from Group B.

Step 3: Extend M and Find Candidate Keys

We systematically test adding attributes from Group B ($\{A, B, D\}$) to our core set C.

Try $\{A, C\}$:

Iteration	Current X+	FD Applied	New X+
Start	$\{A, C\}$	—	$\{A, C\}$
1	$\{A, C\}$	$A \to B$	$\{A, B, C\}$
2	$\{A, B, C\}$	$AC \to D$ (since $A, C \in X^+$)	$\{A, B, C, D\}$
3	$\{A, B, C, D\}$	$D \to E$	$\{A, B, C, D, E\}$

- **Result:** $\{A, C\}^+ = \{A, B, C, D, E\} = R$. ✅ **Super Key found!**
- **Minimality Check:**
  - Remove A: $\{C\}^+ = \{C\} \neq R$ ❌ (Cannot remove A)
  - Remove C: $\{A\}^+ = \{A, B\} \neq R$ ❌ (Cannot remove C)
- ✅ Therefore, $\{A, C\}$ **is a Candidate Key.**

Try $\{B, C\}$:

Iteration	Current X+	FD Applied	New X+
Start	$\{B, C\}$	—	$\{B, C\}$
1	$\{B, C\}$	$B \to A$	$\{A, B, C\}$
2	$\{A, B, C\}$	$AC \to D$	$\{A, B, C, D\}$
3	$\{A, B, C, D\}$	$D \to E$	$\{A, B, C, D, E\}$

- **Result:** $\{B, C\}^+ = R$. ✅ **Super Key found!**
- **Minimality Check:**
  - Remove B: $\{C\}^+ = \{C\} \neq R$ ❌
  - Remove C: $\{B\}^+ = \{B, A\} \neq R$ ❌
- ✅ Therefore, $\{B, C\}$ **is a Candidate Key.**

Try $\{D, C\}$:

Iteration	Current X+	FD Applied	New X+
Start	$\{D, C\}$	—	$\{D, C\}$
1	$\{D, C\}$	$D \to E$	$\{C, D, E\}$
2	$\{C, D, E\}$	No FD has LHS $\subseteq \{C, D, E\}$ producing A or B	$\{C, D, E\}$

- **Result:** $\{D, C\}^+ = \{C, D, E\} \neq R$. ❌ **Not a Super Key.**

(Note: We can skip larger pairs like $\{A, D, C\}$ because they would contain the valid Candidate Key $\{A, C\}$ as a subset, making them non-minimal).

Step 4: Classify Prime and Non-Prime Attributes

Attribute	In CK {A,C}?	In CK {B,C}?	Classification
A	✅ Yes	—	**Prime**
B	—	✅ Yes	**Prime**
C	✅ Yes	✅ Yes	**Prime**
D	❌ No	❌ No	**Non-Prime**
E	❌ No	❌ No	**Non-Prime**

- **Prime Attributes:** $\{A, B, C\}$
- **Non-Prime Attributes:** $\{D, E\}$

Step 5: Enumerate All Super Keys

Every Candidate Key is a Super Key, and every superset of a Candidate Key is also a Super Key. To find all Super Keys, we take our Candidate Keys ($\{A, C\}$ and $\{B, C\}$) and append **all possible combinations of the Non-Prime attributes** ($\{D, E\}$):

1. **Supersets of $\{A, C\}$:**
   - $\{A, C\}$ *(Candidate Key)*
   - $\{A, B, C\}$ *(Added B)*
   - $\{A, C, D\}$ *(Added non-prime D)*
   - $\{A, C, E\}$ *(Added non-prime E)*
   - $\{A, B, C, D\}$ *(Added B, D)*
   - $\{A, B, C, E\}$ *(Added B, E)*
   - $\{A, C, D, E\}$ *(Added D, E)*
   - $\{A, B, C, D, E\}$ *(Added all)*
2. **Supersets of $\{B, C\}$ (Unique additions):**
   - $\{B, C\}$ *(Candidate Key)*
   - $\{B, C, D\}$ *(Added non-prime D)*
   - $\{B, C, E\}$ *(Added non-prime E)*
   - $\{B, C, D, E\}$ *(Added D, E)*

- **Complete Unique Set of All Super Keys (12 total):**

$$SK = \{A, C\}, \{B, C\}, \{A, B, C\}, \{A, C, D\}, \{A, C, E\}, \{B, C, D\}, \{B, C, E\}, \{A, B, C, D\}, \{A, B, C, E$$

💡 **The Pattern:** Notice that every Super Key contains either $\{A, C\}$ or $\{B, C\}$ as a subset. The Non-Prime attributes $\{D, E\}$ act as "passengers"—appending them to any Candidate Key creates a valid Super Key, but they contribute no determining power of their own.

📝 **EXAMPLE 14:**

Consider a relation schema $R(A, B, C, D, E)$ with the following set of functional dependencies F:

$$F = \{A \to BC,\ CD \to E,\ B \to D,\ E \to A\}$$

Find all candidate keys of the relation R.

✅ Show Solution

we need to find the attribute sets whose closure includes all attributes $\{A, B, C, D\}$ and no proper subset has the same property.

Step 1: Identify Essential Attributes

Let's analyze which attributes appear on the **right-hand side (RHS)** versus the **left-hand side (LHS)** of the FDs:

- **RHS attributes:** A, B, D
- **LHS-only / Missing from RHS:** C never appears on the right side of any functional dependency. Because C is never generated by any FD, C **must be part of every candidate key**.

Step 2: Test Attribute Closures Including C

Since C must be in every candidate key, we start by testing combinations of C with other attributes.

1. Try C alone:

- $C^+ = \{C\}$ (Does not include A, B, D) $\to$ Not a key.

2. Try two-attribute combinations containing C:

- AC^+:
  - Start: $\{A, C\}$
  - Apply $AC \to D$: Add $D \to \{A, C, D\}$
  - Apply $AD \to B$: Add $B \to \{A, B, C, D\}$
  - **Result:** $\{A, B, C, D\}$ (All attributes!)
  - *Check for minimality:* Since neither A^+ nor C^+ alone gives all attributes, AC **is a candidate key**.
- BC^+:
  - Start: $\{B, C\}$
  - Apply $BC \to A$: Add $A \to \{A, B, C\}$
  - Apply $AC \to D$ (since we now have A and C): Add $D \to \{A, B, C, D\}$
  - **Result:** $\{A, B, C, D\}$ (All attributes!)
  - *Check for minimality:* Since neither B^+ nor C^+ alone gives all attributes, BC **is a candidate key**.
- CD^+:
  - Start: $\{C, D\}$
  - Apply $CD \to B$: Add $B \to \{B, C, D\}$
  - Apply $BC \to A$ (since we now have B and C): Add $A \to \{A, B, C, D\}$
  - **Result:** $\{A, B, C, D\}$ (All attributes!)
  - *Check for minimality:* Since neither C^+ nor D^+ alone gives all attributes, CD **is a candidate key**.

Conclusion

The candidate keys for relation R are:

1. AC
2. BC
3. CD

3.5 Types of Functional Dependencies

Not all functional dependencies are created equal. Some represent structural flaws (anomalies) in the database design, while others are mathematically trivial. Understanding these types is the exact mechanism used to define **Second Normal Form (2NF)** and **Third Normal Form (3NF)**.

Table 30: *Types of FDs*

Term	Mathematical Definition	Meaning & Impact
Trivial FD	$X \to Y$ where $Y \subseteq X$	Always true by definition. Carries no new information.
Non-trivial FD	$X \to Y$ where $Y \not\subseteq X$	The meaningful FDs that define actual business rules.
Full FD	$X \to Y$ and no proper subset of X determines Y	The ideal state. Y depends on the *whole* key.
Partial FD	$X \to Y$ and a proper subset of X determines Y	**Red flag for 2NF.** Y depends on only *part* of a composite key.
Transitive FD	$X \to Y \to Z$ (where $Y \not\to X$)	**Red flag for 3NF.** Z depends on a non-key attribute.

3.5.1 Trivial Functional Dependency

📖 DEFINITION 6: **Trivial Functional Dependency**

An FD $X \to Y$ is trivial if Y is a subset of X ($Y \subseteq X$).

📝 EXAMPLE 15:

Consider an employee table where we know both the employee's ID and their name.

- **FD:** $\{\text{EmpID}, \text{Name}\} \to \text{EmpID}$ (Since EmpID is part of the left-hand side set, this is trivial).

EmpID	Name	Department
101	Ahmed	Computer Science
102	Mahmoud	Engineering

If you already know that an employee has ID `101` and is named **Ahmed**, stating that $\{\text{EmpID}, \text{Name}\}$ determines EmpID tells you nothing new. You already had the ID.

3.5.2 Non-Trivial Functional Dependency

📖 DEFINITION 7: **Non-Trivial Functional Dependency**

An FD $X \rightarrow Y$ is non-trivial if Y is not a subset of X ($Y \not\subseteq X$). *If X and Y share no common attributes at all, i.e., $X \cap Y = \emptyset$, it is called **completely non-trivial**).*

📝 EXAMPLE 16:

Consider an employee table:

- **FD:** $\text{EmpID} \rightarrow \text{Department}$ (Knowing the employee ID uniquely determines their department).

EmpID	Name	Department
101	Ahmed	Computer Science
102	Mahmoud	Engineering

The attribute on the right (Department) is completely separate from the left (EmpID). This is a genuine business rule used to design the database.

3.5.3 Full Functional Dependency

📖 DEFINITION 8: **Full Functional Dependency**

An FD $X \rightarrow Y$ is a full functional dependency if it is non-trivial, and removing *any* attribute A from X breaks the dependency (meaning no proper subset of X determines Y).

📝 EXAMPLE 17:

Imagine a university enrollment relation where a student takes a course across a specific semester. The Primary Key is a composite key: $\{\text{StudentID}, \text{CourseID}\}$.

- **FD:** $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}$

StudentID	CourseID	Grade
201 (Ahmed)	CS101	A
201 (Ahmed)	MATH202	B
202 (Sara)	CS101	A

Can you know **Ahmed**'s Grade by looking at $\mathrm{StudentID}$ alone? No, because Ahmed takes multiple courses with different grades. Can you know the Grade by looking at $\mathrm{CourseID}$ alone? No, because many students take CS101 and get different grades. **Grade** requires *both* attributes together to be uniquely determined.

3.5.4 Partial Functional Dependency

📖 DEFINITION 9: **Partial Functional Dependency**

An FD $X \to Y$ is a partial dependency if a *proper subset* of X can already determine Y.

📝 EXAMPLE 18:

Using the same enrollment table, suppose we also store the student's name:

- **FD:** $\{\mathrm{StudentID}, \mathrm{CourseID}\} \to \mathrm{StudentName}$

StudentID	CourseID	StudentName
201	CS101	Ahmed
201	MATH202	Ahmed
202	CS101	Sara

- **The Red Flag:** $\mathrm{StudentName}$ depends *only* on $\mathrm{StudentID}$, which is just a proper subset of the composite key $\{\mathrm{StudentID}, \mathrm{CourseID}\}$.
- **The Problem:** If **Ahmed** changes his name, you have to update it across multiple rows. If you miss one, your database becomes inconsistent. This violates **2NF**, and the fix is to move $\mathrm{StudentName}$ into a separate table where $\mathrm{StudentID}$ is the sole primary key.

3.5.5 Transitive Functional Dependency

📖 DEFINITION 10: **Transitive Functional Dependency**

An FD $X \to Z$ is transitive if there is an intermediate attribute set Y such that $X \to Y$ and $Y \to Z$ (where $Y \not\to X$, and Y is not a candidate key).

EXAMPLE 19:

Consider an employee table where EmpID is the primary key:

- **FD 1:** $\mathrm{EmpID} \rightarrow \mathrm{DeptID}$ (An employee belongs to a department ID)
- **FD 2:** $\mathrm{DeptID} \rightarrow \mathrm{DeptName}$ (A department ID determines the department name)
- **Transitive FD:** $\mathrm{EmpID} \rightarrow \mathrm{DeptName}$ (Through DeptID)

EmpID	Name	DeptID	DeptName
101	Ahmed	D01	IT
102	Khaled	D02	HR
103	Yousef	D01	IT

- **The Red Flag:** $\mathrm{DeptName}$ is a non-prime attribute, but it depends on DeptID, which is *also* a non-prime attribute (just a regular column, not a candidate key for this table).
- **The Problem:** If Department `D01` changes its name from **IT** to **Computer Science**, you have to update every single employee belonging to that department. This violates **3NF**, and the fix is to extract department data into its own table ($\mathrm{DEPARTMENT}$).

4 Normalization

Normalization is the systematic approach of effectively designing a database so that it minimizes data redundancy and inconsistency. It is generally done through a series of steps called **Normal Forms (NF)**.

Each normal form builds on the previous one—a relation in a higher NF automatically satisfies all lower NFs:

$$1NF \subset 2NF \subset 3NF \subset BCNF \subset 4NF \subset 5NF$$

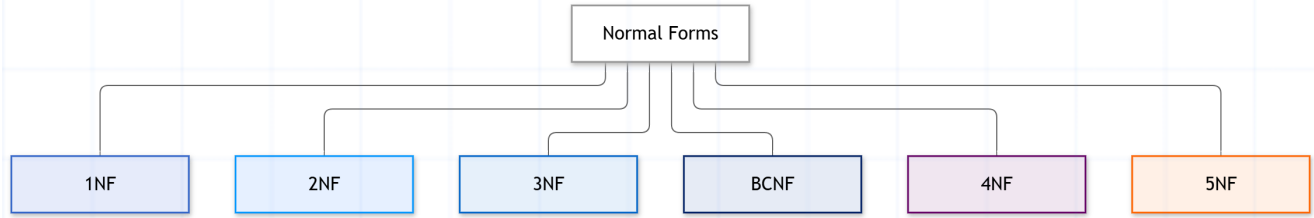


Diagram 59: Hierarchy of Database Normal Forms

Normal Form	Key Requirement	What it Eliminates
1NF	All attribute values are atomic	Multi-valued / repeating groups
2NF	1NF + no partial FDs on any candidate key	Redundancy from partial dependencies
3NF	2NF + no transitive FDs (with prime-attr exception)	Transitive dependencies via non-key attributes
BCNF	Every FD determinant is a superkey	All non-trivial FD-based redundancy
4NF	BCNF + no non-trivial MVDs	Multi-valued dependency redundancy
5NF	4NF + no non-trivial join dependencies	Decomposition artifacts

4.1 The Origins of Database Tables

Before applying normalization rules, it is essential to understand where database tables originate. Tables in a normalized relational system typically come from one of two design paths:

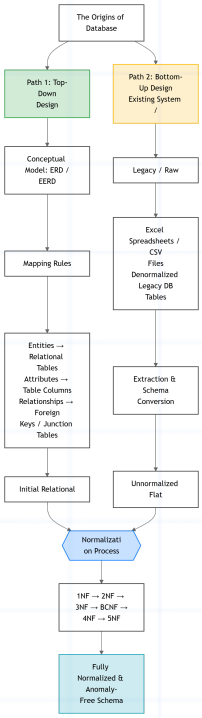


Diagram 60: Top-Down (ERD) vs. Bottom-Up (Legacy/Flat files) database design approaches leading into the normalization process (1NF through 5NF).

4.2 First Normal Form (1NF)

A relation is in 1NF if:

- Every attribute contains only **atomic** (single-valued, indivisible) values
- There are no repeating groups or multi-valued attributes
- There is a **primary key** that uniquely identifies each row

📝 EXAMPLE 20: **Violating 1NF**

Consider an unnormalized student table where a student can enroll in multiple courses, and those courses are stored together inside a single comma-separated cell:

StudentNo	StudentName	Courses
S1	Ahmed	DS, Algorithms
S2	Hussein	Databases

✅ Show Solution

The Issue: The `Courses` attribute is **not atomic** because it holds multiple values in a single cell. This directly violates 1NF and prevents standard relational querying (like indexing or joining on individual course identifiers).

To bring the relation into 1NF, we split the multi-valued field into separate rows so that each cell holds exactly one atomic value:

StudentNo	StudentName	CourseNo	CourseName
S1	Ahmed	CS101	Data Structures
S1	Ahmed	CS102	Algorithms
S2	Hussein	CS301	Databases

- **Atomicity Achieved:** Each cell now holds a single, indivisible value.
- **Primary Key:** The composite key $\{StudentNo, CourseNo\}$ uniquely identifies each row in the flattened relation.

> ⓘ **Note on Redundancy**
>
> While this table is now successfully in 1NF, the `StudentName` attribute is repeated for student `S1`. This structural redundancy is systematically addressed by moving to **Second Normal Form (2NF)**.

4.3 Second Normal Form (2NF)

A relation schema R is in Second Normal Form (2NF) if and only if:

- It is already in **1NF**.
- Every non-prime attribute is **fully functionally dependent** on every candidate key.

Understanding Partial Dependencies

- **Scope:** 2NF is only relevant when a relation has a **composite primary key** (composed of two or more attributes). A relation with a single-attribute primary key is automatically in 2NF if it is already in 1NF.
- **Partial Dependency:** This occurs when a non-prime (non-key) attribute is functionally dependent on only a *proper subset* of a composite candidate key, rather than the entire key.

Second Normal Form eliminates these partial dependencies by breaking them out into separate relations.

EXAMPLE 21: Violating 2NF

Consider the 1NF student enrollment table where the composite key is $StudentNo, CourseNo$:

StudentNo	StudentName	CourseNo	CourseName	InstructorNo	Grade
S1	Ahmed	CS101	Data Structures	I1	A
S1	Ahmed	CS102	Algorithms	I2	B+
S2	Hussein	CS301	Databases	I3	A-

Functional Dependencies Present:

- $StudentNo, CourseNo \rightarrow Grade$ (Full Dependency: both attributes are required to determine the grade).
- $StudentNo \rightarrow StudentName$ (**Partial Dependency**: `StudentName` depends only on `StudentNo`, not on `CourseNo`).
- $CourseNo \rightarrow CourseName, InstructorNo$ (**Partial Dependency**: course attributes depend only on `CourseNo`, not on `StudentNo`).

Show Solution

To achieve 2NF, we decompose the table by extracting the attributes involved in partial dependencies alongside the respective subset of the key:

1. **Students Table** (Key: $StudentNo$)
   - Stores attributes dependent solely on the student identifier.

StudentNo	StudentName
S1	Ahmed
S2	Hussein

2. **Courses Table** (Key: $CourseNo$)

- Stores attributes dependent solely on the course identifier.

CourseNo	CourseName	InstructorNo
CS101	Data Structures	I1
CS102	Algorithms	I2
CS301	Databases	I3

3. **Enrollments Table** (Key: $StudentNo, CourseNo$)

- Stores the relationship and attributes fully dependent on the entire composite key.

StudentNo	CourseNo	Grade
S1	CS101	A
S1	CS102	B+
S2	CS301	A-

Result: All non-key attributes now depend on the *entire* primary key of their respective relations. Partial dependencies are eliminated.

4.4 Third Normal Form (3NF)

A relation schema R is in **Third Normal Form (3NF)** if and only if it is already in **2NF** and, for every non-trivial functional dependency $X \rightarrow A$, at least one of the following conditions holds:

- X is a **superkey** of R.
- A is a **prime attribute** (i.e., A is a member of some candidate key of R).

Core Rules & Key Concepts

- **Eliminates Transitive Dependencies:** A transitive dependency occurs when a non-key attribute determines another non-key attribute, forming a chain like $Key \rightarrow Y \rightarrow Z$ (where Y is not a superkey and Z is a non-prime attribute). 3NF systematically removes these structures to prevent update anomalies.
- **The Prime Attribute Exception:** Unlike Boyce-Codd Normal Form (BCNF), 3NF permits a non-key attribute to appear on the right-hand side of a dependency as long as that attribute belongs to a candidate key. This specific rule allows 3NF to preserve functional dependencies during decomposition where BCNF might otherwise fail.

EXAMPLE 22: **Violating 3NF**

Suppose we have a `Courses` relation schema that extends our 2NF design by also embedding instructor details:

$Courses(CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation)$

- **Candidate Key:** $\{CourseNo\}$
- **Functional Dependencies:**
  1. $CourseNo \rightarrow \{CourseName, InstructorNo\}$ (Key determinant)
  2. $InstructorNo \rightarrow \{InstructorName, InstructorLocation\}$ (Transitive dependency)

Why It Violates 3NF

- The functional dependency $InstructorNo \rightarrow \{InstructorName, InstructorLocation\}$ features a determinant ($InstructorNo$) that is **not a superkey** of the relation.
- Furthermore, the attributes on the right-hand side are **non-prime attributes** (they do not belong to any candidate key).
- This creates a transitive chain: $CourseNo \rightarrow InstructorNo \rightarrow \{InstructorName, InstructorLocation\}$.

✅ Show Solution

To bring the relation into 3NF, we decompose it by extracting the transitive dependency into its own relation:

1. **Courses Table** (Key: $CourseNo$)
   - Retains core course information and references the instructor via a foreign key.

CourseNo	CourseName	InstructorNo
CS101	Data Structures	I1
CS102	Algorithms	I2
CS301	Databases	I1

2. **Instructors Table** (Key: $InstructorNo$)

- Isolates attributes that depend directly on the instructor identifier.

InstructorNo	InstructorName	InstructorLocation
I1	Dr. Ali	Building A
I2	Dr. Sara	Building B

Result: Transitive dependencies are successfully eliminated. Updating an instructor's location now requires modifying a single row in the `Instructors` table, preventing data inconsistency.

4.5 Solved Examples

📝 EXAMPLE 23:

Consider a relation schema $R(A, B, C, D, E, F)$ with the following set of functional dependencies F:

$$F = \{AB \to C,\ D \to E,\ B \to F,\ A \to D\}$$

1. **Find all candidate keys** of the relation R.
2. **Determine the highest normal form** that relation R satisfies (1NF, 2NF, 3NF, or BCNF). Justify your answer.
3. If the relation is not in 3NF,**decompose it** into 3NF relations.

✅ Show Solution

To determine whether the given relation $R(A, B, C, D)$ is in Second Normal Form (2NF), let's first find the candidate keys, identify any partial dependencies, and then perform the decomposition if necessary.

Step 1: Find the Candidate Keys of R

Given the functional dependencies $FD = \{AB \to CD,\ B \to C\}$:

1. **Test attribute closures:**
   - Let's check AB^+:
     - Start:$\{A, B\}$
     - Apply $AB \to CD$: Add $C, D \to \{A, B, C, D\}$
     - **Result:**$\{A, B, C, D\}$. Thus,AB **is a candidate key**.
   - Let's check AB subsets (A^+ and B^+):
     - $A^+ = \{A\}$ (cannot determine B, C, D)
     - $B^+ = \{B, C\}$ (using $B \to C$)

Since no proper subset of AB can determine all attributes,AB **is the only candidate key** for relation R.

- **Prime attributes (part of a candidate key):**A, B
- **Non-prime attributes:**C, D

Step 2: Check for 2NF

Definition of 2NF: A relation is in 2NF if it is in 1NF and **no non-prime attribute is fully functionally dependent on a proper subset of any candidate key** (i.e., no partial dependencies).

Let's check the given FDs against this rule:

1. $AB \to CD$: This is a full dependency since the entire key AB determines CD.
2. $B \to C$:
   - Here,B is a **proper subset** of the candidate key AB (i.e.,$B \subset AB$).
   - C is a **non-prime attribute**.
   - Therefore,$B \to C$ is a **partial dependency** (a non-prime attribute is determined by part of the candidate key).

Conclusion:**Relation R is NOT in 2NF** because of the partial dependency $B \to C$.

Step 3: Convert R into 2NF

To eliminate partial dependencies and convert the relation into 2NF, we decompose R into smaller relations:

1. **Extract the partial dependency into its own relation:**
   - The determinant B and the attribute it determines (C) form the first decomposed relation:
     - $R_1(B, C)$ with FD:$B \to C$ (where B is the primary key of R_1).
2. **Create the remaining relation with the candidate key and remaining attributes:**
   - The original candidate key attributes plus any attributes not involved in the partial dependency form the second relation:
     - $R_2(A, B, D)$ with FDs:$AB \to D$ (where AB is the primary key of R_2).

Final Result

The relation R is decomposed into the following 2NF relations:

- $R_1(B, C)$ with primary key B
- $R_2(A, B, D)$ with primary key AB

📝 **EXAMPLE 24:**

Consider a relation schema $R(A, B, C, D, E)$ with the following set of functional dependencies F:

$$F = \{AB \to C,\ C \to D,\ D \to B\}$$

1. **Find all candidate keys** of the relation R.
2. **Determine whether R is in 3NF**. If not, decompose it into 3NF.

✅ Show Solution

Step 1: Find the Candidate Keys

1. Test attribute closures:
   - AB^+:
     - Start:$\{A, B\}$
     - Apply $AB \to C$: Add $C \to \{A, B, C\}$
     - Apply $C \to D$: Add $D \to \{A, B, C, D\}$
     - Apply $D \to B$: Add B (already there)
     - *Wait, what about E?* Notice that E **never appears on the right-hand side** of any functional dependency in F. Because E cannot be generated by any FD,E **must be part of every candidate key**.
   - Let's re-test ABE^+:
     - Start:$\{A, B, E\}$
     - Apply $AB \to C \to D \to B \to C \ldots \to$ Generates $\{A, B, C, D, E\}$.
     - **Result:**$\{A, B, C, D, E\}$. Thus,ABE **is a candidate key**.

2. Are there other candidate keys?

- Since $D \to B$ and $C \to D$ and $AB \to C$, we can substitute:
- Let's check ACE^+:
  - Start:$\{A, C, E\}$
  - Apply $C \to D$: Add $D \to \{A, C, D, E\}$
  - Apply $D \to B$: Add $B \to \{A, B, C, D, E\}$
  - **Result:**$\{A, B, C, D, E\}$. Thus,ACE **is also a candidate key**.
- Let's check ADE^+:
  - Start:$\{A, D, E\}$
  - Apply $D \to B$: Add $B \to \{A, B, D, E\}$
  - Apply $AB \to C$: Add $C \to \{A, B, C, D, E\}$
  - **Result:**$\{A, B, C, D, E\}$. Thus,ADE **is also a candidate key**.

Candidate Keys:ABE,ACE, and ADE.

- **Prime attributes:**A, B, C, D, E*(Every attribute is part of at least one candidate key!)*
- **Non-prime attributes:** None.

Step 2: Check for 3NF

Definition of 3NF: A relation is in 3NF if, for every functional dependency $X \to Y$, at least one of the following holds:

1. $X \to Y$ is a trivial functional dependency ($Y \subseteq X$).
2. X is a **superkey** for R.
3. Every attribute in $Y - X$ is a **prime attribute** (part of a candidate key).

Let's test each FD in F:

1. $AB \to C$:AB is not a superkey, but C **is a prime attribute**→ Satisfies 3NF.
2. $C \to D$:C **is NOT a superkey**, and D **is a prime attribute**→ Satisfies 3NF!*(This is the crucial edge case: even though C is not a superkey, $3NF$ allows this dependency because D is prime).*
3. $D \to B$:D **is NOT a superkey**, and B **is a prime attribute**→ Satisfies 3NF.

Conclusion: Relation R**is already in 3NF** because every non-trivial dependency has a right-hand side consisting of prime attributes. No decomposition is needed.

📝 **EXAMPLE 25:**

Consider a relation schema $R(W, X, Y, Z)$ with the following set of functional dependencies F:

$$F = WX \to Y,\ Y \to Z,\ Z \to X$$

1. **Find all candidate keys** of the relation R.
2. **Determine the highest normal form** that relation R satisfies (1NF, 2NF, 3NF, or BCNF). Justify your answer.

✅ Show Solution

Step 1: Find the Candidate Keys

1. Analyze the attributes to see which ones never appear on the right-hand side (RHS) of any FD:
   - RHS attributes: Y, Z, X.
   - Notice that W **never appears on the right-hand side** of any functional dependency. Therefore, W **must be part of every candidate key**.
2. Test attribute closures starting with W:
   - W^+:
     - Start: $\{W\}$
     - No FD has W alone on the LHS. Thus, $W^+ = \{W\}$ (Not a key).
   - WX^+:
     - Start: $\{W, X\}$
     - Apply $WX \to Y$: Add $Y \to \{W, X, Y\}$
     - Apply $Y \to Z$: Add $Z \to \{W, X, Y, Z\}$
     - **Result:** $\{W, X, Y, Z\}$ (All attributes!) $\to WX$ **is a candidate key**.
   - **Alternative path using $Z \to X$:**
     - What about WZ^+?
     - Start: $\{W, Z\}$
     - Apply $Z \to X$: Add $X \to \{W, X, Z\}$
     - Apply $WX \to Y$: Add $Y \to \{W, X, Y, Z\}$
     - **Result:** $\{W, X, Y, Z\}$ (All attributes!) $\to WZ$ **is also a candidate key**.

Candidate Keys: WX and WZ.

- **Prime attributes:** W, X, Z (attributes that belong to at least one candidate key). Wait, let's check Y:
  - Does Y belong to a candidate key? Let's check WY^+:
    - Start: $\{W, Y\}$
    - Apply $Y \to Z$: Add $Z \to \{W, Y, Z\}$
    - Apply $Z \to X$: Add $X \to \{W, X, Y, Z\}$
    - **Result:** $\{W, X, Y, Z\} \to WY$ **is also a candidate key**!
  - **All attributes (W, X, Y, Z) are prime attributes** because every single one of them participates in at least one candidate key (WX, WZ, or WY).

Step 2: Determine the Highest Normal Form

Let's check each FD against the normalization rules:

1. $WX \to Y$:
   - WX is a candidate key (superkey) $\to$ Satisfies BCNF, 3NF, and 2NF.

2. $Y \rightarrow Z$**:**

   - Y is **not a superkey**.
   - However, let's check the right-hand side (Z):Z **is a prime attribute** (part of candidate key WZ).
   - Under the 3NF definition, a dependency where the LHS is not a superkey is still **allowed in 3NF** as long as the RHS attributes are prime. Therefore,$Y \rightarrow Z$ **satisfies 3NF**, but violates BCNF.

3. $Z \rightarrow X$**:**

   - Z is **not a superkey**.
   - However, the right-hand side (X) is **also a prime attribute** (part of candidate keys WX and WZ).
   - Therefore,$Z \rightarrow X$ **satisfies 3NF**, but violates BCNF.

Conclusion:

- The relation **is in 3NF** because every non-trivial dependency has a right-hand side consisting of prime attributes.

Chapter6 Storage & Indexing

1 Storage Hierarchy

Whenever a database uses data, that data must exist in memory. Accessing this data is relatively fast, but once the data becomes very large, it becomes impossible to fit all of it within memory. Disks are used to cheaply store all of a database's data, but they incur a large cost whenever data is accessed or new data is written.

> **Note**
>
> all execution happens ultimately by reading **pages** from disk into memory. The number of page reads is the primary cost metric for query optimization — and indexes exist to reduce that count dramatically.

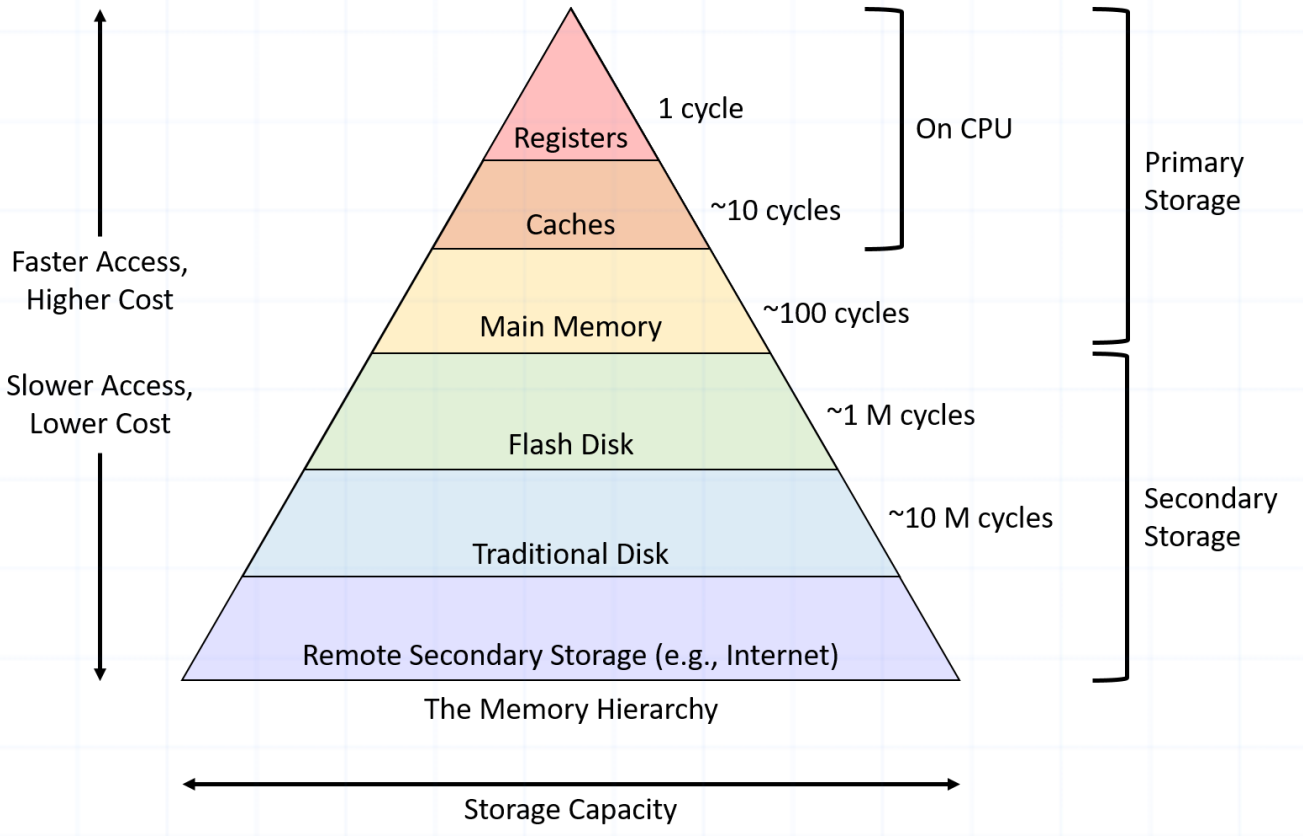


Figure 11: Memory Hierarchy

The pyramid above illustrates how storage is organized in a computer system — and it maps directly onto how a DBMS manages data:

- **Registers & CPU Cache (L1/L2/L3)** — Extremely fast but tiny (kilobytes). The CPU works here. A DBMS has no direct control over this layer; the CPU uses it automatically when processing query operators.
- **Main Memory / RAM** — Fast (nanoseconds), limited in size (gigabytes). This is where the DBMS **buffer pool** lives. All query execution happens on data loaded here — a row can only be read, compared, or joined once it is in RAM.

- **SSD / Flash Storage** — Slower than RAM (microseconds), larger (terabytes), persistent. Many modern databases use SSDs as their primary storage layer, greatly reducing I/O latency compared to spinning disks.
- **HDD / Magnetic Disk** — Slowest (milliseconds per read), cheapest per GB. Traditional databases were designed around this level. Because a single disk read is **~100,000× slower** than a RAM access, minimizing disk I/Os is the central goal of every DBMS storage and indexing decision.
- **Network / Cloud Storage** — Slowest tier, used for backups, archival, and distributed databases (e.g., Amazon S3, Google Cloud Storage).

2 Why We Need a Buffer Manager

> 📖 DEFINITION 1: **The Buffer Manager**
>
> The **Buffer Manager** is the core database management system (DBMS) module responsible for bridging the performance gap between fast main memory (RAM) and slow secondary storage (disks/SSDs).
>
> Because database sizes frequently exceed available RAM, and because the CPU can only execute operations on data loaded into memory, the buffer manager acts as an intermediary, maintaining a pool of memory pages known as the **database buffer** (or buffer pool).

Whenever a database uses data, that data must exist in **memory (RAM)** — because the CPU can only work with data that is loaded there. Accessing data in RAM is fast (nanoseconds), but RAM is expensive and limited in size. A typical database can be hundreds of gigabytes or even terabytes, and it is simply **impossible to fit all of it in memory at once**.

Disks (HDDs or SSDs) solve the capacity problem — they are cheap and can store enormous amounts of data permanently. But there is a catch: accessing data on disk is **thousands of times slower** than accessing data in RAM.

2.1 Core Responsibilities of the Buffer Manager

- **Space Allocation:** Allocates and manages buffer space in main memory to temporarily store data blocks while they are actively being used.
- **Hit vs. Miss Management:**
  - If a requested data block is already in the buffer, it immediately returns its memory address (avoiding costly disk access).
  - If the block is missing, it fetches the page from disk into the buffer pool.
- **Page Eviction & Replacement:** When the buffer is full and a new page must be loaded, it decides which older blocks to evict using replacement algorithms (such as LRU).
- **Dirty Page Synchronization:** Tracks whether pages in memory have been modified ("dirty blocks"). Before evicting a modified page, it writes the updated data back to disk to ensure durability; otherwise, unmodified pages are simply discarded.

2.2 How It All Works Together

The diagram below traces what happens when you run a SQL query — from the moment the query arrives to when results are returned (and how modified data is synced back to disk).

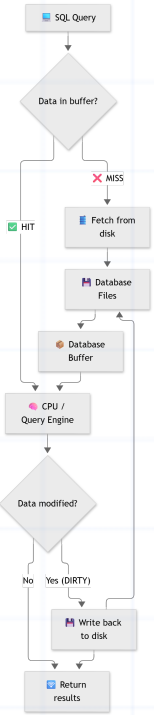


Diagram 61: Buffer Manager workflow diagram

Key Takeaways

- The **first time** you query data → it comes from **disk** (slow)
- The **second time** you query the same data → it comes from the **buffer in RAM** (fast — no disk access!)
- If you **modify** data → the change happens in the buffer first, then gets **written back to disk** to make it permanent

The Page is the Unit of I/O

All disk reads and writes transfer exactly **one page** (typically 4–16 KB, configurable). Reading a single 10-byte row forces a full page to come into RAM. Query cost is measured in **number of page I/Os**, not rows accessed.

3 Records and Page Layout

The basic unit of data for relational databases is a **record** (row). These records are organized into **relations** (tables) and can be modified, deleted, searched, or created in memory.

Every table in a relational database is stored on disk. But disk and memory work differently — the **CPU can only process data that is in memory (RAM)**, and disk stores data in fixed-size chunks. To bridge this, three concepts stack on top of each other: a table is stored as a file, the file contains fixed-size pages, and each page holds one or more records.

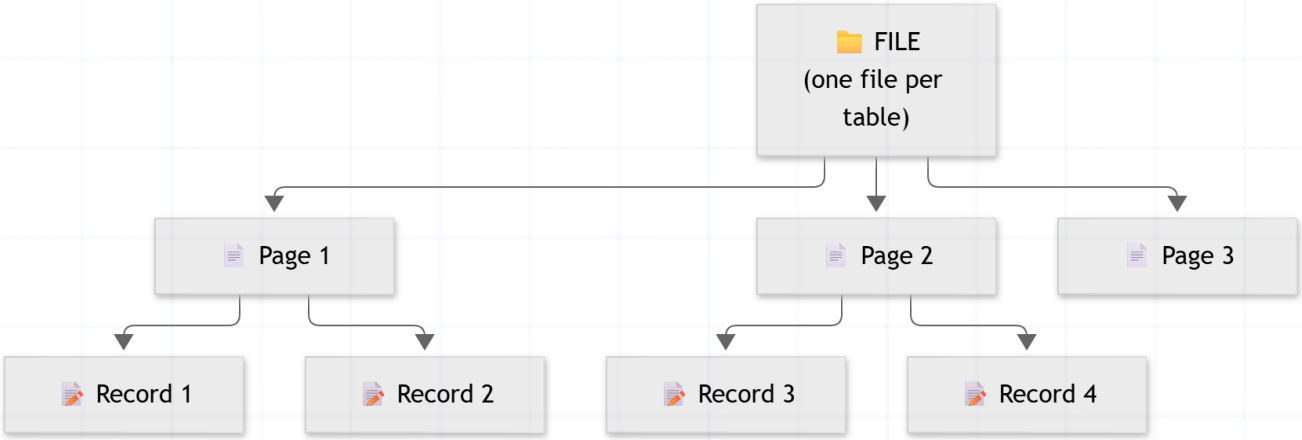


Diagram 62: The three-level storage hierarchy: a table is stored as a file, the file contains fixed-size pages, and each page holds one or more records.

> **Note**
>
> - A **record** is one row (the database's smallest unit of meaning)
> - A **page** is a fixed-size block (the disk's smallest unit of transfer, typically 4–16 KB)
> - A **file** is a sequence of pages that stores all records of one table

When the DBMS decides how to store a table, it must answer four questions — in this exact order:

1. **What type of file?** — heap or sorted?
2. **How are pages organized in the file?** — linked list or page directory?
3. **How are records organized on each page?** — packed, bitmap, or slotted?
4. **How is each record formatted?** — fixed-length or variable-length?

3.1 Choosing File Types

The **file type** determines how pages (and the records inside them) are ordered. The DBMS picks the type that gives the **lowest I/O cost** for the table's typical workload.

There are two main types of files: Heap Files and Sorted Files. For each relation, the database chooses which file type to use based on the I/O cost of the relation's access pattern. **1 I/O** is equivalent to **1 page read from disk** or **1 page write to disk**, and I/O calculations are made for each file type based on the insert, delete, and scan operations in its access pattern. The file type that incurs less I/O cost is chosen.

> **Note**
>
> In a relational database management system (DBMS), the file type (Heap File vs. Sorted File) is **not determined dynamically at runtime** for every single operation, nor is there always a single static system default.
>
> When a table (relation) is created using a statement like `CREATE TABLE`, the database designer or the system's storage engine decides how the relation's data will be physically stored.
>
> - **Default Behavior:** Many standard relational databases (like PostgreSQL, MySQL/InnoDB, or SQLite) use **Heap Files** as the default storage structure for standard tables because they provide fast, low-overhead insertions.
> - **Sorted/Clustered Files:** If a user or the database administrator explicitly creates a clustered index or specifies an ordered storage method (such as an index-organized table or a sorted table), the DBMS will store the records in a **Sorted File** structure based on a chosen key.

COPYRIGHT

Fundamentals of Database Systems: A Comprehensive Course — Relational Databases & NoSQL

© 2026 Hussein Younis. All rights reserved. Computer Engineering Department.

This work is an original educational resource authored by Hussein Younis. All text, figures, diagrams, interactive widgets, and source code are the intellectual property of the author unless otherwise attributed. Third-party works cited throughout the text (including Elmasri & Navathe, *Fundamentals of Database Systems*, and the official MongoDB documentation) remain the property of their respective authors and publishers and are used here under fair-use / academic quotation principles.

The book is distributed for **educational use** within the Database Systems course. Readers may read, study, and quote short excerpts in academic work with proper citation. Redistribution, derivative works, translations, and commercial use require prior written permission of the author.